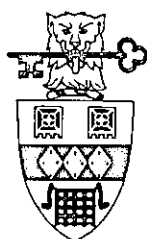


VECTOR

APL '86 REVEALED

- Free APL computer.
- Keyword APL on the Sinclair QL.
- Goodbye to Von Neumann.
- Review of APL on PC, AT, and Sage.
- Graphics revisited.
- Reading spreadsheets from APL.
- £50 prize competition.



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

Vol.1 No.4 April 1985

CONTRIBUTIONS

All contributions to VECTOR should be sent to the Editor at the address given on the inside back cover. Letters and articles are welcomed on any topic of interest to the APL community. These do not need to be limited to APL themes nor must they be supportive of the language. Articles should be submitted in duplicate and accompanied by as much visual material as possible, including a photograph of the author. Unless otherwise specified each item will be considered for publication as a personal statement by its author, who accepts legal responsibility that its publication is not restricted by copyright. The provision of camera-ready or machine-readable copy is encouraged: please contact the Editor beforehand. Program listings should indicate the computer system on which they have been run. APL symbols should be displayed on a separate line and not embedded in narrative. Except where indicated, items published in VECTOR may be freely reprinted with appropriate acknowledgement.

MEMBERSHIP

<i>Category</i>	<i>Fee p.a.</i>	<i>VECTOR copies</i>	<i>Passes</i>
Nonvoting student membership	£ 5	1	1
UK Private membership	£ 9	1	1
Overseas private membership	£ 16	1	1
Supplement for airmail (not needed for Europe)	£ 8		
Corporate membership	£ 75	10	5
Sustaining membership	£325	neg	5

The membership year runs from 1st May to 30th April. Applications for membership should be made on the form at the end of the journal. Passes are required for entry to some Association events and for voting at Annual General Meetings. Applications for student membership will be accepted on a recommendation from a course supervisor. Overseas membership rates cover VECTOR surface postage and must be paid in UK £.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive multiple copies of VECTOR and are offered group attendance of Association meetings. Partaking individuals need not be identified but a contact person should be nominated for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in the editorial section of the journal and opportunities to inform APL users of their products via seminars and articles.

ADVERTISING

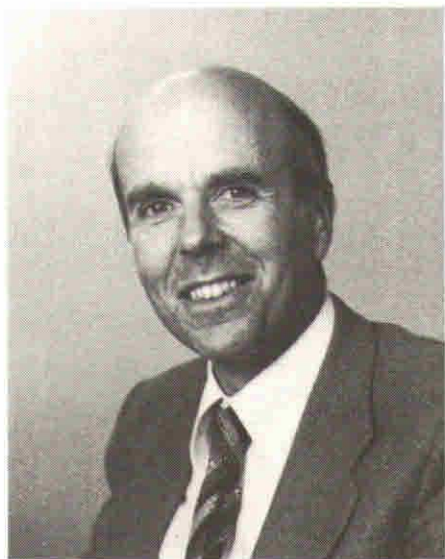
Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20 mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £200 per page. A6 and A7 sizes are offered pro-rata subject to layout constraints. Deadlines are:

Advertisement booking:	3rd Friday in April, July, October & January.
Camera-ready copy:	1 week later. Distribution: 2 weeks later.

Advertisements should be booked with and sent to David Preedy at the address on the inside back cover.

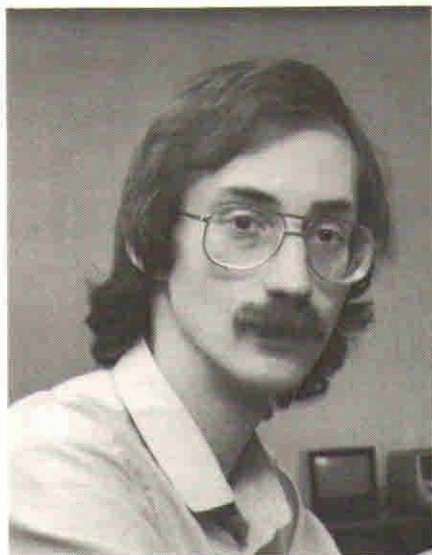
CONTENTS

		Page
EDITORIAL: Your Association needs you	Robert Bittlestone	3
APL NEWS		
General correspondence		5
British APL Association news	David Ziemann	6
Free APL computer	David Ziemann	8
International APL news — APL86	Goacher & Ziemann	18
British Computer Society news	Philip Goacher	31
News from sustaining members	David Preedy	33
APL product guide	David Preedy	37
Book Reviews	Robert Bittlestone	51
Software Review	Adrian Smith	54
RECENT MEETINGS		
		61
The Royal Overseas League	Adrian Smith	63
APL and GKS	Richard Nabavi	63
Duck à la Carte	Peter Donnelly	65
Getting the best out of GDDM	Adrian Smith	65
GENERAL ARTICLES		
		73
Reinventing Man: Goodbye to Von Neumann	Robert Bittlestone	75
QL/APL — A new approach to popularising APL	David Eastwood	81
The information centre as a strategy for the future	Fred Perkins	90
Steps towards a better Basic — Part 3	Anthony Camacho	95
Case Study: A practical introduction to expert systems — Part 2	Christopher Harvey	98
TECHNICAL SECTION		
		103
Technical Editorial	Barman & Ziemann	105
Technical Correspondence		107
Surely There Must be a Better Way	Dick Bowman	112
Prize Competition	Barman & Ziemann	117
Keyword QL/APL — an (occasionally) technical review	David Ziemann	121
Moving data from 1-2-3 to APL	Michael Carmichael	130
A DIF file interface	Romilly Cocking	149
When domino is not sufficient	Sykes & Ansell	155



VECTOR 4 was edited and assembled by Dave Ziemann and Jonathan Barman. Most of the articles were available on diskette which made it possible to produce VECTOR 4 in time for APL 85, and hence the photos of editors tapping away at keyboards.

Special thanks are due to Anthony and Sylvia Camacho and to Dick Bowman who spent the Easter holiday proof reading.



A APOLOGY: Please correct an INDEX ERROR by applying the following fix to your copy of VECTOR Vol.1 No.3:

VECTOR3[68 101]+VECTOR3[101 68]

EDITORIAL: YOUR ASSOCIATION NEEDS YOU

By Robert Bittlestone

Bringing VECTOR into existence and nurturing its post-natal twelve months has been most enjoyable. But it's time for others to take part. So please don't assume that the post of Journal Editor, or indeed any of the other positions in the British APL Association, are already taken up for 1985/86. Write to Anthony Camacho (address inside rear page) now to nominate yourself. Browbeat your colleagues, or better still come to the A.G.M. and raise your hand in person. An illustrious future awaits you, while for ourselves there is the heady and hard-won prospect of early retirement.

This fourth issue of VECTOR brings us to the end of a full year of the new format of the journal. Six hundred pages and about three megabytes of copy later (including the blanks), your editorial team is a little older, a little wiser, and just a little wearier. We look forward with some relish to handing over this particular Olympic torch to our successors following the elections at the forthcoming Annual General Meeting of the British APL Association. Should, however, exhaustion cause us collectively to predecease those who follow us, perhaps these reflections on our experiences this last year may not come amiss. They may bear superficial resemblance to the Ten Commandments, but they are intended as words of guidance rather than coercion.

1. Never publish distribution dates in advance.
2. Never make announcements about the periodicity of a journal which is maintained by the unpaid work of its editorial team.
3. It is unrealistic to expect contributors to a computer journal to deliver their copy in computer-legible form.
4. The greater a contributor's insight into computing, the lower the probability that he or she can write in English.
5. The average number of entries to a competition is inversely proportional to the length of its rules.
6. Bribes such as prize money have no effect on swelling the ranks of competition entrants. Glory through print is its own reward.
7. Books should be accepted for review only from those publishing houses which regard a delay of a year before the review appears as fast work.
8. The expansion of a journal's scope does not of itself increase its circulation. There is no substitute for house-to-house inquiries.
9. After five years there is still nothing to match WORDSTAR.
10. The world-wide demise of the APL symbol set will continue to be exacerbated by the difficulty of typesetting APL code.

I would like to acknowledge the impressive contributions made throughout the year by the VECTOR editorial and production team. Following my own lapse into catatonia as a result of producing Issue 1, David Preedy masterminded Issue 2 effortlessly (at least in comparison), Adrian Smith single-handedly commuted from York to London to sign off Issue 3 (rattling off the presses as I pen these lines), while David Ziemann and Jonathan Barman have taken collective responsibility for Issue 4. Finally, especially not to be forgotten is Paul Whelan of Copyprint, whose appearances at midnight half way down the A3 festooned in galleys have become reassuring by their ephemerality.

Give your Information Center users the world.

An Information Center should be a company-wide resource taking advantage of modern technologies through the integration of hardware, software and data communications.

There are many communications companies in the world. There are many software companies. But for full integration of communications and software, there is only one I.P. Sharp.

With I.P. Sharp's products for the Global Information Center, your users enjoy:

- IPSANET...one of the world's most advanced private communications networks providing secure error-free transmission anywhere your company operates.
- Integrated Software...a library of end user productivity tools to reduce your applications backlog. Products for today's decision making.
- Worldwide User Support...through I.P. Sharp offices in 22 countries.
- Online Public Data...access to over 100 numeric data bases to complement corporate data and help your organization make informed decisions.
- Integration of PC's and mainframes...with a single application development language, SHARP APL.

Ready to install, ready to run, and already proven on an international scale by I.P. Sharp clients, the Global Information Center gives you the most advanced implementation of the Information Center concept.

Why not let them have the world?

Call your nearest I.P. Sharp office in one of the cities listed below and we'll show you how your Information Center users can have the world.



I.P. Sharp Associates
Products for the
Global Information Center

London, England (European Headquarters)
I.P. Sharp Associates Limited,
Heron House, 10 Dean Farrar Street,
London SW1H 0DX.
Telephone: (01) 222 7033. Telex: 8954178 SHARP

Aberdeen, Amsterdam, Atlanta, Bristol, Brisbane, Brussels, Calgary, Canberra, Chicago, Copenhagen, Coventry, Dallas, Denver, Dublin, Düsseldorf, Edmonton, Frankfurt, Halifax, Helsinki, Hong Kong, Houston, London, Los Angeles, Madrid, Melbourne, Mexico City, Miami, Milan, Montreal, New York City, Newport Beach, Oslo, Ottawa, Palo Alto, Paris, Rochester, Rome, San Francisco, San Jose, Seattle, Singapore, Stockholm, Sydney, Taipei, Toronto, Vancouver, Victoria, Vienna, Washington, Wayne, White Plains, Winnipeg, Zurich

GENERAL CORRESPONDENCE

This section of VECTOR is reserved for communications of a general nature. Letters which require a high technical knowledge of APL or contain APL code usually appear in the Technical Section: please note the requirements stated on the inside front cover. The editor reserves the right to edit letters unless the author states that a letter is to be published in full or not at all.

From Ole Immanuel Franksen

26th November 1984

Sir: Just a short note to thank you for the copy of the last issue of VECTOR (Vol.1, No. 2), containing my presentation at APL84 of the book Mr. Babbage's Secret as well as the description of its contents. It may interest you to know that Prentice-Hall, Inc., has taken over the international rights of the book outside Scandinavia, and that they will publish a new edition in the early spring of 1985.

In closing permit me to express my appreciation of the publication by the British APL Association of an European journal devoted to APL. I feel convinced that this initiative will arouse much interest among Scandinavian APL users.

Sincerely yours,

Ole Immanuel Franksen
Electric Power Engineering Department,
Technical University of Denmark,
Building 325, DK-2800 Lyngby.

From Dick Bowman

12th February 1985

Sir: What a distressing rule to find, that Committee members are prohibited from entering VECTOR competitions; may I suggest a revised rule, that Committee members are actually obliged to supply entries (whether a subrule prohibiting them from winning anything would then be required I leave to the readers' discretion).

Yours sincerely,

Dick Bowman,
CEGB,
85 Park Street,
London SE1.

(Ed: The remainder of Dick's communication can be found in the Technical Section of this issue of VECTOR.)

BRITISH APL ASSOCIATION NEWS

Compiled by David Ziemann

Situations Filled and Vacant

The Association has filled two committee places and lost one; the role of Education Officer has been accepted by Dick Gray, and Geoff Kemmish becomes the new Technical Officer.

The position of Publicity Officer is again vacant following Mr Christopher Beatty's resignation. This post requires an ability to handle Press contacts and a gift for phrasing announcements about Association affairs in an interesting and news-worthy manner. Duties include (a) handling all Press matters, including press releases, (b) liaising with sustaining members over matters of general APL market interest, (c) assisting the Journal Editor in promoting the circulation of VECTOR, and (d) helping the Activities Officer to publicise Association meetings.

Activities Programme 1985/6

Forthcoming Association activities include the following:

- September 13 — Either Consultants Questiontime
or ADRS Workshop
- October 18 — Large APL Systems
There are increasing numbers of multi-person long term development projects within the APL community;
What do we have to learn from the mainstream DP business and what in turn can we teach them?
- November 15 — Compilers and Interpreters
APL compilers are becoming a reality; what goes on inside them and what impact does this have on the application developer?
- January 17 — Joint Meeting with OR Society
Who don't know about it yet. OR is an area which has had a long history of using APL as a tool; what's going on and what can we both do to make OR use of APL even more successful and widespread?
- March 21 — APL System Design
Beyond wet fingers and pieces of string which design techniques are most useful to the APL developer; how much discipline is useful and how much is restrictive?

- May 16 — AGM + Relational Databases etc.
Besides your regular opportunity to vote with your feet; an examination of the impact which developments such as APL2's link with mainline DB2 databases may have on the future use of APL.
- July 7 - 11 — APL86
Of which you will read much more elsewhere.

Other topics are being considered; hopefully we may be able to put on a special discussion/debate about 'Marketing APL' (is APL getting its fair share of the computing cake and what can/should we do to increase our slice of the action?). Anyone wishing to contribute to this debate is invited to contact the Activities Officer.

Contrary to the suggestion made elsewhere we are not planning to run a repeat of 'APL84 In Focus' during 1986 for the benefit of all those people who can't make the trip to Manchester.

The venue will continue to be the Royal Overseas League (London W1) with all meetings scheduled to begin at 2pm; don't forget the opportunity which this gives you to meet with other APL users and implementors both before and after the meeting proper.

Once again I remind everyone of the APL Publications bookstall which is a regular feature of London events and the opportunity which this gives you to purchase all that hard-to-find APL literature. A full list of available publications with prices can be found towards the end of the British APL Association News Section.

Please note that this programme is subject to revision; further details of each event will be sent to British members nearer to the date.

FREE APL COMPUTER

by David Ziemann

Well, that should have got your attention, but here's the double bluff - it's true. The British APL Association has recently acquired a second-hand IBM 5100 APL micro-computer, including peripherals, and we're going to give it away - to a deserving cause, of course. The kind donors of this equipment are Quaker Oats Ltd., of Southall in London, whose computing requirements have far outstripped the capabilities of the old, but trusty 5100.

The complete package, which cannot be split, includes the following equipment:

- IBM 5100 APL computer
- IBM 5103 dot-matrix printer
- IBM auxilliary tape cassette drive
- NEC monitor
- IBM UK to US voltage converter
- Assorted manuals and tape cassettes

The only catch is that this 5100 is a US machine and therefore requires a UK to US voltage converter to operate in this country. The converter is a heavy cube of metal, but shouldn't present a big problem to someone keen to use APL.

The IBM PC (otherwise known as a 5150) is a direct descendant of the 5100, a machine that was way ahead of its time when it was built back in 1975. It is a dedicated turn-on-and-go APL machine which runs full APL SV via a 370 emulator in read-only-memory. The machine has a built-in screen, APL keyboard and tape cassette drive which uses standard data tape cartridges, and onto which APL workspaces can be saved, and data files created. The auxilliary drive can be attached to provide the system with a second on-line tape drive. The ease with which standard composite-signal monochrome monitors can be attached to and slaved from the 5100 make this an ideal system for APL training, or even just for one or two people to use as a dedicated machine on which to learn APL.

Written applications from those wishing to provide a permanent home for the equipment should be addressed to the Chairperson of the British APL Association, and must be received by 31st July 1985. Although commercial organisations will also be considered, we would particularly like to see the machine used within an educational group or institution, especially where awareness and use of the APL language can be promoted. Applicants should therefore state their affiliations, and indicate how the computer would be used.

The responsibilities of owning a complete micro-computer system are many, and casual applications are discouraged. Maintenance and repairs to the equipment will be the responsibility of the recipient - Quaker Oats and the British APL Association unfortunately cannot undertake to provide support of any kind.

ANNUAL GENERAL MEETING

The Annual General Meeting of the British APL Association will be held in the St. Andrews Hall at the Royal Overseas League starting at 2 pm on Tuesday 28 May 1985.

The following papers consist of:

- (a) A reminder about nominations for the committee
- (b) The Agenda
- (c) The minutes of the 1984 AGM
- (d) The draft Regulations for the Association
- (d) The secretary's report on the committee meetings

a) Nominations for the Committee

To summarise the notice in Vector 3:

Nominations for the committee are to be sent to the secretary by the end of April 1985 and those received in time will be circulated to all members. Nominations can be accepted at the AGM until the chairman declares the list closed. Nominees (even if only nominated at the AGM) should be individual members of the Association, proposed and seconded by any class of member and nominations should include a brief description of their suitability and their intentions should they be elected.

b) Agenda for the AGM

The Agenda for the AGM is as follows:

- 1 Minutes of the 1984 AGM
(These are printed below so that they may be taken as read if there are no objections at the meeting).
- 2 Chairman's report
The draft Regulations for the Association
(This is printed below)
- 3 Votes on the motion "The new Regulations be adopted" (discussions will be restricted to amendments that have been passed for discussion by the committee and circulated to members; proposed amendments should be sent to the secretary)
(The secretary will represent those unable to attend if they send him their — brief — representations beforehand)
- 4 Officers' reports (Secretary, Treasurer, Activities Officer, Journals Officer, Education Officer, Technical Officer)
- 5 Call for final nominations and listing of candidates
- 6 Elections — conducted by the chairman
- 7 Close of AGM followed by the ordinary meeting.

Note: The new officers take office at the close of the AGM and handovers are made at the first committee meeting after the AGM. Both old and new committees are to attend that meeting.

c) Minutes of the 1984 AGM

Minutes of the AGM of the APL Society held in the Read Lecture Theatre, Imperial College, on 20 January 1984

1 Officers' Reports

The Chairman reviewed the year: the Society is seven years old, prosperous and growing (though not as fast as we would like). It has held five meetings this year and a major three day conference, published two magazines and conference proceedings and a third magazine is expected from the printer within a week. A task force was set up in November to recommend how the Society should be improved and its proposals were circulated to all members for discussion today.

The acting secretary reported on behalf of Gordon Sutcliffe, that the five (well attended) meetings were:

January: Phil Chastney on Hunting the Snark

March: Three speakers on implementations of VS APL, APL *PLUS and Dyalog APL

May: A joint meeting with the Local Authorities APL User Group to hear six speakers on subjects connected with APL and education.

November: A workshop on Idioms in APL

The APL Business Technology 83 conference and exhibition was held at Loughborough in September: over 200 people attended and 43 papers were read. Most of the major APL vendors exhibited.

The treasurer reported that when all debts were paid the Society would have a surplus this year of £7333, of which £5193 was attributable to APL Business Technology 83. Nearly six thousand pounds was owed to the Society. The accumulated surplus from last year was £1182 so the Society's assets at the end of its financial year (31 October 1983 — the Society's financial year will now be aligned to the B.C.S. financial year) were £8515. The balance sheet was shown in a simplified form on an overhead projector slide.

The membership secretary reported approximately 450 members of whom about 50 lived abroad.

- 2 The chairman went through the proposals for the improvement of the Society which were slightly amended by the morning committee meeting. The new committee is to have eight officers and no limit on the number of ordinary members who may be co-opted: a student grade is to be considered: the committee may recommend a representative to the APL standards committee and work towards establishment of standards for representation of APL in ASCII. There was no dissension. The proposals are therefore accepted and the new committee is to act on them.
- 3 The elections were held by displaying the nominations for each post in turn, asking for further nominations, reading out a brief note about each candidate and then taking a vote. Voting was by show of hands.

The following officers were elected:

Chairman	Phil Goacher
Secretary	Anthony Camacho
Treasurer	Mel Chapman
Activities	Dick Bowman
Promotion	Fred Perkins
Journal	Robert Bittlestone
Technical	Romilly Cocking
Education	Chris Beattie

No ordinary members were elected: the following offered their services or were nominated:

David Allen, Peter Cyriax, Ron Fuss, Marc Griffiths, Chris Hogan,
Valerie Lusmore, Tony Moore, Dominic Murphy, David Preedy,
Adrian Smith, Chris Smith, Prof. Barrie Weatherill, Stan Wilkinson.

Anyone wishing to help who preferred not to be on the committee was asked to let the secretary have their name address and area of interest at the end of the meeting.

- 4 The Chairman explained that the Society's year is to be changed to match the BCS accounting year and so the newly elected committee will hold office for a little over a year.
- 5 The Society's business was concluded at 3.05 p.m. by which time there were over 80 people present.

Anthony Camacho 21 January 1984

Signed P S Goacher (Chairman) 31 January 1984

d) **Draft Regulations for adoption at the AGM**

BRITISH APL ASSOCIATION

DRAFT REGULATIONS

1. NAME

The Group shall be called The British APL Association.

2. AFFILIATION

The Association is and shall be affiliated to The British Computer Society (BCS) as a Specialist Group.

3. OBJECTS

- (a) To promote the use of APL;
- (b) To develop awareness and competence in APL;
- (c) To represent the interests of Association members with other bodies;
- (d) To contribute to the development of the language and definition of international standards.

4. CONSTITUTION

The British APL Association shall consist of:

- (a) President and Vice Presidents;
- (b) Chairman, Secretary and Treasurer;
- (c) Other elected officers;
- (d) Individual fee paying members;
- (e) Individual fee paying student members;
- (f) Corporate members;
- (g) Sustaining members.

5. MEMBERS IN GOOD STANDING

A member in categories 4(d), 4(e), 4(f), or 4(g) is in good standing with the Association by virtue of having paid all membership fees due to the Association for the current year. Memberships which are not renewed in the first four months of a new financial year (by 31 August) are deemed to have lapsed.

6. APPOINTED OFFICERS

- (a) A President may be appointed at the discretion of the Management Committee.
- (b) A maximum of four Vice Presidents may be appointed in recognition of service to the APL community at the discretion of the Management Committee.
- (c) Appointed Officers may serve for a period of one year with up to two consecutive renewals. Non-contiguous periods of office are permitted with no limit.
- (d) Appointed Officers shall have the right to attend free of admission charge all General Meetings of the Association and all events staged by the Association.

7. ELECTED OFFICERS

- (a) The Elected Officers shall be Chairman, Secretary and Treasurer who must be individual fee paying members in good standing.
- (b) Other officers may be elected to fill posts created by the Management Committee.
- (c) It is the duty of Elected Officers to attend meetings of the committee.
- (d) Election: The officers shall be elected by the Annual General Meeting (AGM) and shall serve from the end of the meeting at which they are elected until the end of the AGM following.
- (e) Casual Vacancy: A vacancy occurring during the term of office may be filled by an appointment by the Management Committee.

8. MANAGEMENT

- (a) The affairs of the Association shall be managed (subject to the control of the AGM) by a Management Committee comprising:
 - (i) Appointed Officers
 - (ii) Elected Officers
- (b) Co-option: The Management Committee may co-opt members as required, normally to hold office until the next AGM.
- (c) The General Committee of the Association shall comprise the Management Committee together with members of Sub-Committees and Working Parties which from time to time be set up by the Management Committee.
- (d) Meetings: The Management Committee shall meet at least four times in its year of office and frequently enough to carry out the business of the Association properly.
- (e) Notice: At least 14 days notice of the place, date and time of meetings shall be given to each member of the relevant committee.
- (f) Quorum: The business of the Association may be transacted by not less than half the members of the Management Committee.
- (g) In the absence of the Chairman, the Management Committee shall elect one of its members to take the chair for the meeting.
- (h) Voting: In determining a question by vote of the majority of the members present, each having one vote, the Chairman of the meeting shall have a second or casting vote.
- (i) Standing Committees: The Management Committee is empowered to set up Standing Committees to deal with matters that require continuity. A member of the Management Committee will sit on each Standing Committee.
- (j) Sub-Committees and Working Parties: The Management Committee may set up at any time sub-committees and working parties responsible to the Management Committee which shall appoint a Chairman and provide appropriate terms of reference.

9. ANNUAL GENERAL MEETING

- (a) Each year the Association shall hold an AGM in May.
- (b) Notice: The Secretary shall send, at least 28 days before, notice of the date, time and place of the AGM, together with the Agenda, to all members of the Association. For this purpose a notice printed in the official publication of the Association shall be considered sufficient.
- (c) All members have the right to attend the AGM for which there shall be no attendance charge.
- (d) Agenda: The following items shall be included:
 - (i) Minutes of the previous AGM;
 - (ii) Minutes of any Extraordinary General Meeting held since the previous AGM;
 - (iii) Chairman's Report;
 - (iv) Management Committee Officers' Reports;
 - (v) Audited Statement of Accounts;
 - (vi) Proposals for Alterations to Regulations;
 - (vii) Proposals for Alterations to Fees;
 - (viii) Election of Officers;
 - (ix) Ratification of Auditors.
- (e) Nominations: Any member in good standing is entitled to nominate another individual member in good standing for any elected office on the Management Committee. Such nominations may be proposed and seconded at the meeting if not previously received by the Secretary.
- (f) Voting: Every question at an AGM shall be decided by a majority of the votes cast. Individual members of the Association each have a single vote. The accredited representative of each corporate and sustaining member have five votes which may be split as desired. If the vote is tied, then there shall be a recount; if this fails to resolve the matter the candidate with the least votes is removed and the vote retaken, this process continuing until there is a decisive vote — or it cannot be continued, in which case the Chairman has a casting vote.

10. EXTRAORDINARY GENERAL MEETING

- (a) An Extraordinary General Meeting (EGM) shall be convened on a resolution of the Management Committee or within five weeks of receipt of the Secretary of a requisition signed by no less than one quarter of the current membership in good standing stating the business to be transacted at the meeting.
- (b) Regulation 7, except subparagraph (d) and (e) shall apply.
- (c) An EGM shall transact only such business as is specified in the resolutions or requisitions concerning it.

11. FINANCE

- (a) The Association shall follow the BCS Financial Guidelines as issued from time to time insofar as they apply to Specialist Groups.
- (b) Bank Account: In accordance with the BCS Financial Guidelines, the Association shall have at least one Account (Account A) at Lloyds Bank, Langham Place Branch, used for normal running expenses. Other accounts at the branch may be used for special events or investment funds.
- (c) The financial year shall start on 1 May each year when membership fees become due.
- (d) The Chairman is responsible to the BCS for submitting draft budgets, recording ongoing expenditure and capital expenditure separately for each year by 30 November in the preceding year.

- (e) The Chairman is responsible for making available to the BCS a receipts and payments statement at the end of every financial year (30 April) in respect of the Association's normal operations and special events; this statement to be included in the BCS annual accounts subject to audit by the BCS auditors.
- (f) All cheques drawn on the Association's bank accounts must be signed by any two of Chairman, Secretary and Treasurer.
- (g) The accounts of the Association shall be audited each year by an auditor appointed by the Committee.
- (h) All income and property of the Association from whatever source derived shall be applied solely to the promotion of the objects of the Association.
- (i) No member of the Association shall receive payment for his services as a member.

12. DISSOLUTION

In the event of the winding up or dissolution of the Association any surplus assets remaining after discharge of liabilities shall automatically vest in the BCS.

13. THE BRITISH COMPUTER SOCIETY

- (a) The Association shall be governed by the regulations of the BCS as these apply to Specialist Groups of the BCS. Where it is considered that a regulation of the Association is in conflict with a BCS regulation governing Specialist Group activities, the BCS rule shall apply.
- (b) The Chairman of the Association must be a Fellow, Member or Associate Member of the BCS.
- (c) Other elected officers of the Association should normally be members of the BCS.
- (d) The Chairman of the Association is ex officio a member of the BCS Specialist Groups Board and is thereby eligible, if nominated and elected, to be one of three representatives of the Specialist Groups Board appointed to the Council of the BCS.
- (e) The Chairman or his representative is expected to attend four Specialist Group Board meetings each year.
- (f) The Association must advise the Vice President (Specialist Groups) of those members of the Management Committee of the Association who are not members of the BCS.
- (g) All members of the Management Committee shall abide by the Code of Conduct relating to members of the BCS.
- (h) The Association may use the BCS name to enhance the reputation of its own activities but must not bring the BCS into disrepute.
- (i) No member of the Association may speak on behalf of the BCS without proper authority from the BCS.

e) Secretary's Report on Committee Meetings

There have been eight committee meetings since the 1984 AGM. There will be one more on the morning of the 1985 AGM. Members attendances have been as follows:

	10 Feb 84	9 Mar 84	11 May 84	13 Jul 84	12 Sep 84	16 Nov 84	10 Jan 85	15 Mar 85
Chairman: P.S. Goacher	P	P	P	P	P	P	P	P
Secretary: A.J. Camacho	P	P	P	P	P	P	P	P
Treasurer: M. Chapman	P	P	P	P	P	P	P	P
Activities: D. Bowman	P	P	P	P	P	P	P	P
Vector: R.G.A. Bittlestone	P	P	P	P	A	A	A	A
Promotion: 1 F.J. Perkins	P	A	R	—	—	—	—	—
2 C. Beattie	—	—	—	—	P	A	A	R
Education: 1 C. Beattie	P	P	A	PR	—	—	—	—
2 D. Gray	—	—	—	—	—	—	—	P
Technical: 1 R.J. Cocking	A	R	—	—	—	—	—	—
2 G. Kemmish	—	—	—	—	P	A	A	A

P = present

A = apologies

R = resigned

Committee meetings have usually occupied a whole morning or afternoon. Their efficiency was improved when Dick Bowman's suggestion was acted on (that officers reports are circulated before the meeting). He has done this regularly for activities and APL 86. My aim in the summaries of the year's committee meetings that follow is to reflect to the members how the committee has managed the Association's affairs; many people (especially those on the Vector and Activities working parties) have done a great deal of work for the Association this year which will not be mentioned below.

10 Feb 84

The main work was on the plans for Activities and Vector. The accounts were to be completed and proposals for promotions, technical work and education were discussed. Over a score of volunteers were to be approached for help. Work also began on the takeover by the BCS admin of the chore of membership record maintenance.

9 Mar 84

Romilly Cocking resigned through pressure of work. The secretary's draft of the Association's aims was approved. The Treasurer's draft budget was agreed. The Activities officer reported working party progress and suggested "APL 84 In Focus". New membership grades and 'sustaining' memberships were discussed. Plans for Vector 1 were approved.

11 May 84

Fred Perkins resigned in view of a projected move to Toronto. The Chairman reported final agreement to our new name "The British APL Association". The Treasurer had sorted out a great deal of outstanding mail. The Activities officer reported "APL 84 In Focus" was arranged and on the outline of the winter programme. The Journal officer reported that Vector 1 would be ready for APL 84 in Helsinki. A working party on APL 86 was formed (PSG, MC & DB). Treasurer and Chairman agreed to spend two days sorting out the membership records and Secretary to reply to three dozen outstanding queries.

13 Jul 84

The Treasurer reported progress on subscription reminders, debt collection and audit: MC and PSG were voted thanks. The Activities officer reported "APL 84 in Focus" was a success (the working party was congratulated) and proposed "What's New for 85". The Journal officer reported that Vector 1 just got to Helsinki and that Vector 2 was in hand. Christopher Beattie resigned as Education officer and was co-opted as Promotions officer. Geoff Kemmish was co-opted as Technical Officer. The APL86 subcommittee had advertised APL86 at Helsinki.

21 Sep 84

The Secretary produced the Association's advertising leaflet. His proposal for formalising the special relationship between the Association and APL Publications was agreed. The Chairman reported that Vector 2 was being printed and undertook to arrange publicity through the BCS for Association activities. He also offered to draft a constitution. The APL86 subcommittee's recommendation of the UMIST venue was accepted (the conference had outgrown the accommodation at Oxford or Cambridge).

[Secretary's Note:

The following people now (March 85) comprise the APL86 Organising Committee:

Dick Bowman, Chairman
Sarah Swindells, (BISL) minutes secretary and acting admin.
Phil Goacher, Publicity
Mel Chapman, Finance
Christine McCree, Exhibition
Geoff Kemmish, Technical
Anthony Camacho, Programme]

10 Jan 85

The Activities officer reported the change in venue for March and subsequent meetings and the consequent variation in dates. The Chairman of APL86 reported a Call for Papers had gone into Vector 3 (a brochure is to be part of Vector 4) and various arrangements for representation at Seattle (for which Vector 4 is essential — the absence of the Journal officer was regretted). The Secretary reported that a notice about elections for the 1985-6 committee had gone into Vector 3.

15 Mar 85

The Secretary offered the AGM arrangements and agenda which were approved together with the Chairman's draft Regulations. The Treasurer's draft budget for 1985-6 and the new membership rates were accepted. The Secretary is to publish these in Vector.

The Activities officer reported a profit on 'Whats New for 85'. His proposal to send out a mailshot to canvass support for a workshop on ADRS was approved. The Secretary was asked to book the Royal Overseas League rooms for the rest of 1985 and provisionally for spring 1986.

The Journal officer: The Committee was most disturbed to have no representative or report from Vector in spite of previous appeals and minutes.

The Technical Officer was in the USA and made no report.

The Chairman of APL86 reported that means of overcoming the large debt incurred during run up to APL86 were being investigated. The major publicity drive begins at APL85.

Anthony Camacho, Secretary, 13 March 1985 E & OE

BOOKLIST

LIST OF BOOKS STOCKED AND ON SALE: NOTE SPECIAL PRICES FOR MEMBERS

Description	Invoice £	Cash with order £	British APL		UK Post & pack £
			Invoice £	Cash £	
APL: An Interactive Approach, Gilman & Rose	20.71	20.21	19.50	19.00	2.65
APL: An Introduction, H. Peele	11.20	10.80	10.40	10.00	2.25
Introducing APL to Teachers, K. Iverson	.80	.75	.70	.65	.40
Introduction to APL for Scientists & Engineers	.80	.75	.70	.65	.40
An APL Notebook, Barrie Wetherill	1.90	1.60	1.30	1.00	.50
A Source Book in APL, A Falkoff & K. Iverson	8.75	8.28	8.20	6.96	1.80
APL: Design Handbook for Commercial Systems, Smith	12.25	12.00	10.35	10.00	1.50
APL and Insight, P. Berry and G. Bartoli	4.50	4.20	3.60	3.45	.55
APL in Practice, STSC	20.00	18.00	16.00	15.00	2.50
APL The Language & its Usage, R. Polivka & S. Pakin	29.95	28.95	27.95	26.95	2.70
APL Quote-quad the Early Years	31.50	31.00	30.00	29.60	2.70
APL Business Technology '83 Proceedings	11.20	10.80	10.40	10.00	2.70
APL SV Reference Card	.30	.28	.25	.23	SAE
Designing APL Systems Toronto Procs 82 Vol 2	6.50	6.00	5.50	5.00	1.80
FinnAPL Idiom Library	11.20	10.80	10.40	10.00	1.30
Sharp APL Reference Book	1.30	1.20	1.10	1.00	.50
Sharp APL Reference Manual, P. Berry	10.00	9.70	9.30	9.00	2.70
Yale University Idiom list	1.50	1.40	1.30	1.00	.50
APL in Exposition, K. Iverson	.95	.90	.85	.80	.50
Algebra, K. Iverson	7.95	7.65	7.40	7.10	2.30
Solutions to Algebra, J. Iverson	2.00	1.80	1.70	1.50	.50
Calculus in a New Key, D. Orth	6.85	6.60	6.35	6.10	1.70
Elementary Analysis, K. Iverson	5.40	5.20	5.00	4.80	1.50
A Case Study in BASIC, APL and Functional Prog.	4.80	4.60	4.40	4.30	1.00
Resistive Circuit Theory, R. Spence	10.30	10.00	9.55	9.20	2.80
Star Map, P. Berry & J. Thorstetsen	3.50	3.20	3.10	2.90	.50
Reliable Software Through Composite Design, Myers	9.90	9.65	9.30	9.00	1.30
APL Lapel Pin with gripping backplate	2.00	1.90	1.70	1.60	SAE
Whizzbangs Volume II, R. Sykes	6.90	6.60	6.30	6.00	1.80
Whizzbangs Volume I, R. Sykes	6.90	6.60	6.30	6.00	.40
Information Centres Toronto Procs 84	8.00	7.50	7.00	6.00	1.80

PLEASE ORDER DIRECT FROM APL PUBLICATIONS

APL Publications: L.R. Hollingbery, 9 Koh-I-Noor Avenue, Bushey, Herts, WD2 3EJ
Telephone: 01-950-1044 evenings 1800 to 2100.

This list replaces all previous lists.

Prices are in pounds sterling and are as at 1st May 1985

Terms: sterling cheque with order; invoices payable on receipt of books.

Non-UK postage is extra: please specify AIRMAIL or SURFACE MAIL.

Save postage: a selection of books is usually on sale at APL Association meetings.

INTERNATIONAL APL NEWS

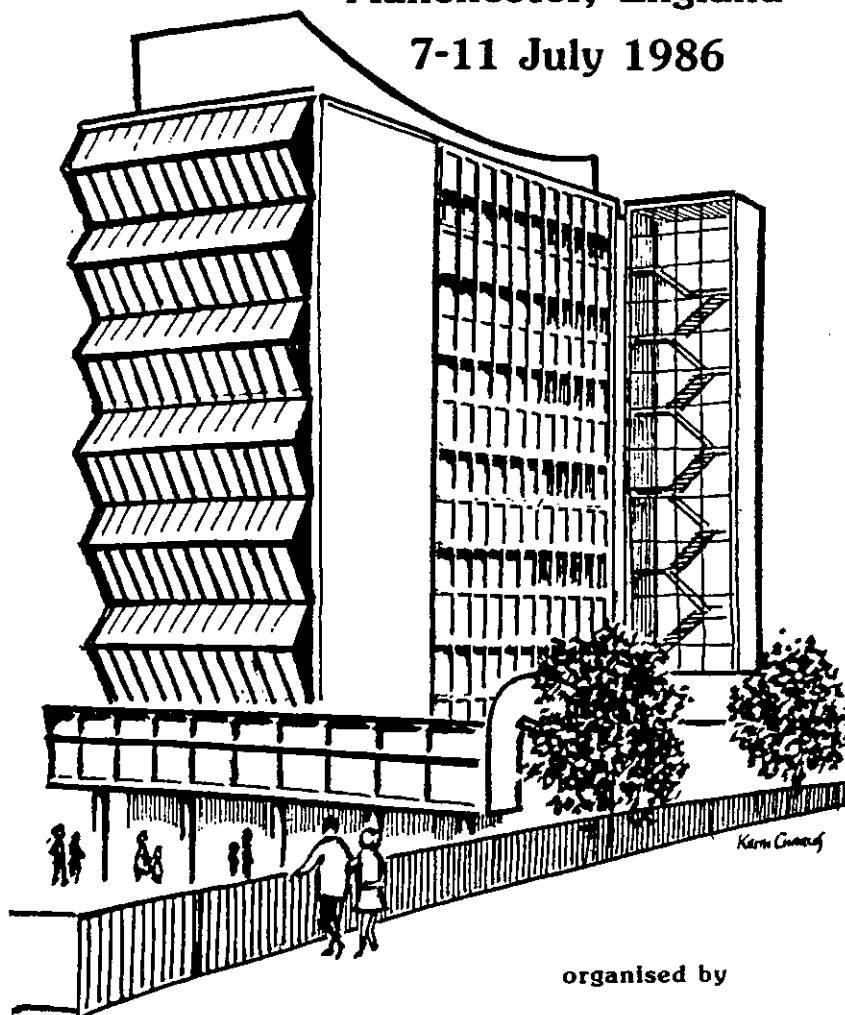
compiled by David Ziemann

This column is open to APL Associations outside the UK; please write to us with your news and we shall be pleased to broadcast extracts for you. A substantial number of VECTOR subscriptions are from overseas, and we hope in this way to keep APL users in touch with each other internationally.

APL86

Manchester, England

7-11 July 1986



organised by

The British APL Association



The British
Computer Society

acm

Association for
Computing
Machinery

APL86 — Call for Papers

APL86 will be the first of the annual series of international conferences on the development and applications of APL to be held in England. There will be contributed and invited papers and a simultaneous exhibition by suppliers of APL-based services and products.

The conference will emphasise practical aspects of APL, and 'live' presentations to accompany submitted papers will be encouraged by the provision of a wide range of computing and audio-visual equipment.

As is traditional at the APL Conferences, a number of distinguished speakers from both within the APL community and other areas, such as Expert Systems, will be invited to present papers, and lead longer discussion sessions, based around 'live' presentations.

The conference will also include a number of tutorial sessions, of interest to both newcomers to APL and those with more experience in the use of the language.

The conference will have three main themes:

APL in Action — including sessions on the use of APL in Personal Computing, Information Centres and Education. Other sessions currently planned include, for example, Full-screen and Graphics orientated systems, applications of APL in Expert Systems and programmer productivity tools.

APL and its Environment — including sessions covering, for example, the interactions between APL and its host Operating Systems, APL as a compiled language, APL-dedicated hardware configurations and the use of APL as an 'engine' in systems constructed largely in other languages.

The APL Notation — including sessions covering, for example, experience with the ISO Standard and with 'second-generation' APLs, the influence of theory on further extensions, the integration of the 'second-generation' extensions and APL as a teaching notation.

The Organising Committee is keen to encourage those who might not previously have considered submitting a paper to such a conference to share their experiences with others in the APL community.

Submission of Papers

Those intending to submit papers on one of the conference themes must make at least a preliminary submission by 15 June 1985; including a title, an abstract and an indication of the nature of the accompanying 'live' presentation and the equipment that would be required.

Full draft papers, for consideration by a panel of referees, must be submitted by 1 November 1985. Three copies of the paper, which is to be in English and should not exceed 6000 words, must be submitted, together with an abstract and a list of keywords for subject identification and classification, and a fuller description of the 'live' presentation. One or more referees may request that the presentation is made as part of the judging process.

Notification of the referees' decisions will be sent out on 10 January 1986. Authors of papers accepted must provide final copies of their papers suitable for photo-reproduction by 4 April 1986.

All papers accepted and received by the final submission deadline will be published in the Conference Proceedings. Other papers will be considered for publication in VECTOR in the four issues following the conference.

It is intended that prizes will be awarded for the 'best' papers and presentations given to the conference.

For further details please contact the BISL Conference Department (APL86). Preliminary submissions may also be made through the I P Sharp Mailbox — APL86.

Important Dates

Preliminary Submission	15 June 1985
Submission Deadline	1 November 1985
Acceptance Deadline	10 January 1986
Final Version	4 April 1986

The 1986 International Conference on APL and its applications will be held from 7 — 11 July 1986 at the University of Manchester Institute of Science and Technology, better known as UMIST.

This venue is situated at the centre of England's second city. Its facilities for speakers, exhibitors and delegates are excellent and with the many interesting places to visit in the area APL86 promises to be a memorable event.

This preliminary notice about APL86 includes the call for papers and initial information for exhibitors. The conference will comprise tutorial sessions, contributed papers, invited speakers, exhibitor presentations and a full social programme including a conference banquet.

If you are not already on our mailing list to receive further information about the conference or exhibition please contact:

BISL Conference Department (APL86)
The British Computer Society
13 Mansfield Street
London W1M 0BP
England

APL86 Local Organising Committee

Chairman	Dick Bowman
Programme	Anthony Camacho
Publicity	Phil Goacher
Technical	Geoff Kemmish
Exhibition	Christine McCree
Finance	Mel Chapman
Administration	BISL (British Computer Society)

Manchester and UMIST

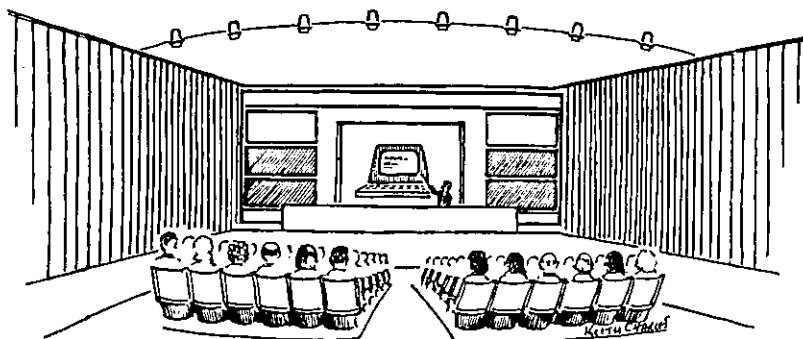
Situated in the North West of England, Manchester is within easy reach, having excellent road and rail access and a major international airport.

The University has a long tradition of computing and is where S.E. Williams developed the Williams Tube in 1946 — a small prototype computer which was the first stored program computer to operate.

The UMIST campus, located in Manchester City Centre is a mixture of old architectural styles and purpose built modern buildings. The modern complex being used for APL86 includes a conference and exhibition centre, restaurant and hall of residence — delegate accommodation is therefore available either on site or at one of the major hotels nearby.

The conference facilities at UMIST are excellent. The main lecture theatres are sited around the exhibition floor and a number of smaller theatres are available for meetings arranged during the conference.

Lunches and dinners will be provided in the large adjoining restaurant. UMIST also offers a wide range of sports facilities and a club bar for which delegates will become members during their stay.



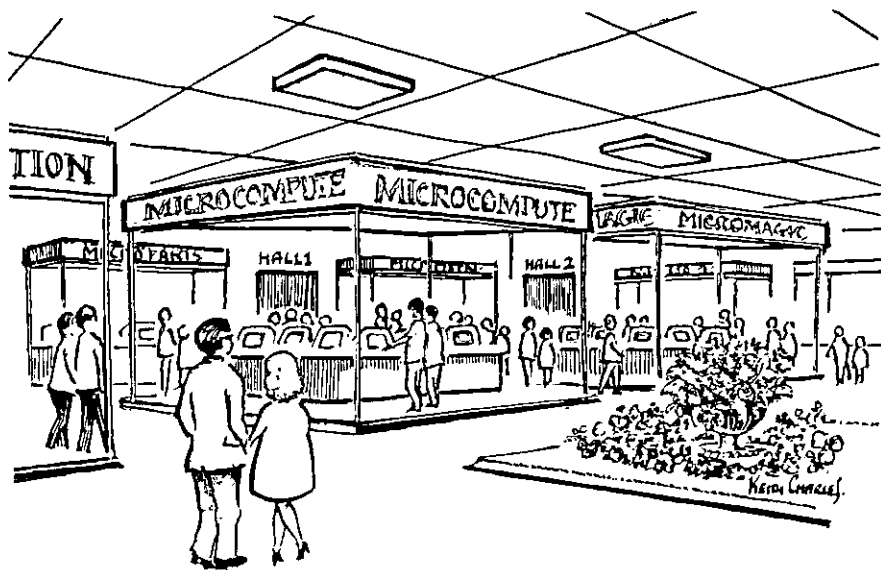
Exhibition

Throughout the Conference a comprehensive exhibition of APL hardware, software, applications and literature will be open to all conference delegates. The exhibition areas are ideally located immediately outside the lecture theatres and beside the cafeteria and bar.

Exhibitors will have an opportunity to give formal presentations on the products they offer through product forum sessions within the conference programme. These will be scheduled at a time when no technical papers are being given to ensure that all conference delegates have the opportunity to attend. Lecture theatres with 500, 300 or 150 seats are available for these one hour presentation sessions.

The British APL Association invite you to participate in APL86 by taking exhibition space, product forum sessions and a hospitality suite. We feel this will give you an excellent opportunity to display your services to the many delegates that this conference will attract from all over the world.

To book, or obtain further details about the exhibition, please contact the BISL Conference Department (APL86).



Out and About

Manchester City Centre has the largest range of entertainment and attractions in the North West of England. For the serious music lover, Manchester is the home of the world famous Halle Orchestra — but all musical tastes are catered for locally. Museums, art galleries, theatres and an exciting night life can be found close by in the City, and whatever your tastes in food, there is a restaurant to meet them.

Manchester has a fine range of shops and minutes from UMIST there is the Arndale Centre, the largest under-cover shopping precinct in Europe.

Within easy reach of Manchester are the Lake District, Peak District, North Wales and the Roman town of Chester — all providing a different view of the area. Visits can also be made to the famous Wedgwood potteries or to the stately home Chatsworth, with its magnificent grounds and interesting history.

Day trips to London are also possible using the Inter-City rail service.

Pre- and post-conference tours will also be arranged for those delegates who wish to see more of England than is possible during the conference week. A programme for accompanying persons will also be available.



For APL86 Information

To: BISEL Conference Department (APL86)
The British Computer Society,
13 Mansfield Street,
London W1M 0BP,
England.

Name _____

Address _____

Please send further delegate information ☐

Please send exhibitor information ☐

Regular APL86 information will be issued to all on the mailing list.

We can fly
more people to
more conferences
than any
other airline.



British Airways is the official airline for this conference. We are the world's largest international airline and in the best position to offer you the most comprehensive range of services from Concorde through to Economy travel. For further details, contact your local British Airways office or appointed Travel Agent.

BRITISH AIRWAYS

The world's favourite airline. 

CIPS APL Special Interest Group (Toronto)

Do you have an interest in using the programming language APL, and live in the Toronto, Canada area?

The CIPS APL SIG (Toronto) is a special interest group (SIG) associated with the Canadian Information Processing Society (CIPS). This APL SIG enables APL professionals, users, vendors, consultants, and academics in the Toronto area to meet each other and to exchange ideas and information on new developments, applications, and problem solving techniques based on APL.

There is no charge for attending meetings. Membership in the SIG, however, is available for \$15 (Canadian). Membership entitles you to receive advance mailing notice of our monthly meetings, to obtain our membership directory, to obtain copies of minutes of meetings and APL utilities (as available), and free admission to our annual APL micro-computer show held each fall.

Other publications of the group include the APL—NAPLPS (Telidon) Toolkit (a completely documented set of APL functions to interface APL and NAPLPS, available for \$5), and the soon-to-be published first edition of our APL Utility Library.

We were formed in September, 1981, and currently have about 100 members.

We have a wide and varied program. For example, our program in 1984-1985 included meetings on APL and UNIX, Business Graphics, APL2 (with James Brown as speaker) SHARP APL (with Ken Iverson and Bob Bernecky as speakers), our new Birds of a Feather meeting, local area networks, and Q'NIAL (with Michael Jenkins as speaker).

Requests for publications and membership should be sent to the following address:

Toronto APL SIG
P.O. Box 917, Station "Q"
Toronto, Ontario, Canada.
M4T 2P1

We also have a SHARP mailbox: TSIG.

Specific inquiries (between now and September, 1985) may also be sent to Richard Levine, APL Product Support Consultant, Xerox Canada Inc., 703 Don Mills Road, North York, Ontario, Canada. M3C 1S2 (416-429-6750).

Meetings**29 April 1985, Toronto**

Monthly Meeting of Canadian Information Processing Society Special Interest Group on APL. *Topic:* Mike Jenkins, President of NIAL Systems Limited, on Q'Nial; and annual general meeting. Ontario Institute for Studies in Education. *Contact:* Richard Levine (416) 429-6750.

Q'Nial is a powerful 5th generation programming language combining ideas from APL, Lisp, and structured and functional programming. Q'Nial is a product of research in Computer Science at Queen's University, Kingston, Ontario, Canada.

27 May 1985, Toronto

Monthly Meeting of Canadian Information Processing Society Special Interest Group on APL. *Topic:* Local area networks.

Ontario Institute for Studies in Education. *Contact:* Richard Levine (416) 429-6750

Ed: The information above was sent to us by the Toronto APL SIG.

International APL Organisations

We are publishing with permission an updated version of the list of organisations appearing in APL Quote Quad, Vol 14 No. 1. The VECTOR Editor would be pleased to receive information from any of these groups for publication.

Austria

APL-Club Austria
Dr. Siegfried Streit, Chair
Wiener Stadtwerke
Mariannengasse 4-6
A-1095 Vienna
Austria

Belgium BACUS

An affiliate of the Computational and
Applied Mathematics Group

Belgian APL-CAM Users Society
Joseph De Kerf, Chair
Roelenberg 72
B-2570 Duffel
Belgium

Telephone: 015/314724
IPSA mailbox: KERF

Canada: Toronto CIPS APL SIG

An affiliate of the Canadian Information
Processing Society

Toronto Special Interest Group on APL
Richard Levine, APL Co-ordinator
Xerox Canada Inc.
703 Don Mills Road
Don Mills, Ontario M3C 1S2
Canada

Telephone: (416)429-6750, Ext. 2684
(after 4 p.m.)
IPSA mailbox: RLEV; group box: TSIG

Finland FinnAPL

Finnish APL Association
Gustav Tollet, Chair
P.O. Box 1005
SF-00101 Helsinki 10
Finland

France

A subsidiary of the Association Française
de Cybernétique, Économique, et
Technique

Group APL, AFCET
Gilles A. Martin, Chair
CISI
35 Boulevard de Brune
F-75014 Paris
France

Germany APLCG

APL-Club of Germany e.V.
Prof. Dr. Wolfgang Stucky
Postfach 6380
D-7500 Karlsruhe 1
Federal Republic of Germany

Italy

A subsidiary of the Associazione Italiana di
Calcolo Automatico

APL Club Italia
Giorgio Faconti, President
CNUCE
Via S. Maria 36
I-56100 Pisa
Italy

Telephone: (50)45245

Mexico

Sociedad Mexicana de APL, A.C.
Erick Alvarado, President
Teleinformática de México, S.A.
Arenal 40
México 20, Dist. Federal
Mexico

Telephone: (905)550 80 33

Netherlands APLWG

A subsidiary of the System-Programming
Section of the Netherlands Society for
Informatics (NGI)

Werkgroep APL
Dhr. Bram Kornaat, Secretary
Rijks Computer Centrum,
Fauststraat 1
NL-7323 BA Apeldoorn
Netherlands

Telephone: (55)778822

Portugal

Associacao Portuguesa da Linguagem
APL
José M. Henriques
Rua Pedro de Barcelos 14
Encosta do Restelo
P-1400 Lisboa
Portugal

Sweden SIG-APL

A subsidiary of the Swedish Society for
Informatics

APL Special Interest Group
Bo Lundmark
Sperry AB
P.O. Box 1138
S-171 22 Solna
Sweden

Switzerland SAUG

Swiss APL User Group
Herr Th. Formanek, President
Postfach
CH-3000 Bern 29
Switzerland

Telephone: (31)624980

United Kingdom

A Specialist Group of the British Computer
Society

British APL Association
Philip Goacher, Chairman
British Computer Society
13 Mansfield Street
London W1M 0BP, England

Telephone: 01 637 0471

USA: Dallas-Fort Worth SWAPL

Southwest APL Users' Group
Don Hatfield, Secy/Treas.
2809 Apple Valley
Garland, Texas
USA 75043

USA: Denver DIGAPL

Denver Interest Group for APL
P.O. Box 3931
Englewood, Colorado
USA 80155

USA: Houston

Houston APL Special Interest Group
Mary Hall, President
Charter Oil Company
P.O. Box 87535
Houston, Texas
USA 77287/7535

Telephone: (713)640-4724

USA: New York City NY/SIGAPL

New York Chapter, SIGAPL
P.O. Box 524
New York City, New York
USA 10025

USA: Rochester

Rochester Chapter, ACM SIGAPL
Robert Pullman, President
P.O. Box 92487
Rochester, New York
USA 14692

Telephone: (716)244-4732

USA: San Francisco APLBUG

APL Bay Area Users' Group
Dick Foss, Treasurer
Crocker National Corporation
West Tower, 29th Floor
One Montgomery Street
San Francisco, California
USA 94104

USA: Seattle

UW APL Users Group / SIGAPL
J.R. Atkins
University of Washington DH-05
Seattle, Washington
USA 98155

USA: Washington

Washington Chapter, ACM SIGAPL
Lawrence Dusold
Dept. of Health, Education and
Welfare HFF-146
200 C Street SW
Washington, District of Columbia
USA 20204

Telephone: (202)245-1413

APL Quote Quad Subscriptions

As an information service to VECTOR readers and as a courtesy to the editors of "APL Quote Quad" for allowing us to reprint some of their news material, we are relaying the details of how to subscribe to their journal.

APL Quote Quad is an informal quarterly publication containing articles, algorithms, announcements, book reviews, correspondence, problems, and other materials of interest to users of the programming language APL. Subscription is a benefit of SIGAPL membership, but is also available to non-members. Address inquiries to:

Fred Aronson
Coordinator of SIGs
Association for Computing Machinery
11 West 42nd Street
New York, New York
USA 10036

Telephone: (212)869-7440
Telex: 421686 ACMUI

The price of annual subscriptions to 'APL Quote Quad' is US\$30.00.

BRITISH COMPUTER SOCIETY NEWS*by Philip Goacher*

The BCS now has a Royal Charter, and the transition to the new Chartered body will be completed in July this year.

One of the issues which this has indirectly raised is the status of specialist group members such as the British APL Association membership. In most specialist groups there are some BCS members and some non-members. This can cause administrative difficulties where the BCS members are able to enjoy BCS privileges not available to their non-member colleagues in the same specialist group. One example is the BCS Computer Newsletter, a 10 issues per year newsletter free to all BCS members only - an ideal medium for contacting the BCS membership but useless to specialist groups as only part of their membership receive copies.

To overcome this and other problems it is proposed that all specialist group members (not already members of the BCS) should automatically qualify to be a form of affiliate member of the Society, one of the benefits of which is to receive a copy of the Newsletter. Other benefits will also accrue, such as being able to get discounts from various suppliers of computer equipment through the buying power of a 30,000 membership.

The proposals are at present in the early stages of discussion, but it is hoped to introduce the scheme before the start of the 1986/87 financial year - in May 1986.

I would like to repeat that those Association members interested in corporate membership of the British Computer Society - as Members or Associate Members - should apply to:

Pat Harris
Membership
The British Computer Society
13 Mansfield Street
London W1M 0BP



THE BRITISH COMPUTER SOCIETY

Specialist Group on Expert Systems

EXPERT SYSTEMS 85 (ES85)

EXPERT SYSTEMS 85, the fifth Annual Technical Conference of the British Computer Society Specialist Group on Expert Systems will be held at the University of Warwick from 16th—19th December 1985.

Two parallel themes are planned — one on the technical and theoretical aspects of expert systems and the other focussing on practical applications and invited tutorial material.

There will be reports from the Alvey Directorate and a number of invited talks, as well as an exhibition of commercial and scientific software and hardware.

This annual event is rapidly assuming international significance and is one you should not miss. For further details please contact:

BISL Conference Department (ES85)
The British Computer Society
13 Mansfield Street
London W1M 0BP
Telephone: 01-637 0471

NEWS FROM SUSTAINING MEMBERS

Compiled by David Preedy

"Sustaining Membership" of the British APL Association is available to any company trading in APL products or services. It provides a tangible way for such companies to express their commitment to APL and to promote increased interest and activity in APL and in the Association.

As well as receiving public acknowledgement for their sponsorship of the Association, sustaining members receive bulk copies of VECTOR for distribution amongst their customers and are offered the opportunity to submit news material for this section of the Journal. They are also able to submit announcements for Association meetings (subject to the approval of the Activities Officer) and are invited to inform APL users of their products via seminars and articles.

The Committee of the British APL Association would like to acknowledge the generous support of the following organisations that have become Association Sustaining Members.

APL People	3 Cross Lane Close, Orwell, Royston, Herts., SG8 5QW. Tel. 04626-77375
APL *PLUS Ltd.	Aston Science Park, Love Lane, Birmingham, B7 4BJ. Tel. 01-493 5096
CACI Inc. International	Oriel House, 26 The Quadrant, Richmond, Surrey, TW9 1DL. Tel. 01-940 3606
Cocking & Drury Ltd.	16 Berkeley Street, London, W1X 5AE. Tel. 01-493 6172
Dyadic Systems Ltd.	30 Camp Road, Farnborough, Hants., GU14 6EW. Tel. 0252-547222
Inner Product Ltd.	Eagle House, 73 Clapham Common South Side, London, SW4 9DG. Tel. 01-673 3354
MetaTechnics Systems Ltd.	Unit 216, 62 Tritton Road, London, SE21 8DE. Tel. 01-670 7959
MicroAPL Ltd.	Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London, SW8 5BP. Tel. 01-622 0395
I.P.Sharp Associates	10 Dean Farrar Street, London, SW1. Tel. 01-222 7033

As usual, we have a wealth of news from our sustaining members since the last issue of VECTOR. The increasing size and apparently continuous activity of this core of the British APL community reflects a growing and more active interest in APL in the U.K. and overseas.

APL People

The APL People support team has been hard at work extending the communications and graphics software on the IBM PC using APL*PLUS/PC. They believe they have the cheapest PC to mainframe communications system in Europe.

The graphics software now works at a resolution of 700 by 700. It will also give 70 lines of text on screen or plotter at very low cost.

APL People's modelling team is supporting a client on a large transportation project in Chicago. They have found IP Sharp's network very useful to keep the communications in civilised hours - although much of the work is being done on PCs.

The demand for APL staff is still exceeding the supply of practitioners. Because of this the company is developing APL aptitude tests for trainees and they plan to validate these tests with the help of experienced APLers.

APL*PLUS Limited

APL*PLUS are pleased to announce that they now offer the STATGRAPHICS package, which provides interactive software for Data Analysis, Data Management and Statistical Graphics on micros and mainframes. STATGRAPHICS is an integrated system of over 350 functions, covering all major areas of statistical analysis, and is designed to provide a flexible, user-friendly environment for serious statistical computing. A key characteristic is the extensive integration of graphics directly into the statistical procedures. Graphical output can be generated by simply pressing selected function keys, and the graphics displayed on the screen can be easily dumped to a printer or routed to a pen plotter. Options are selected by direct editing of full-screen panels and the system has been designed to interface easily with popular micro packages and to encourage the addition of user-written procedures.

Cocking & Drury Limited

Cocking & Drury are looking forward to their new role as distributors for APL*PLUS products in the UK. Several APL*PLUS mainframe installations are imminent, and they are currently carrying out an internal evaluation of UNIX based supermicros for use with APL*PLUS/UNIX. The company is continuing to expand its consulting and education activities, and will also be announcing a number of new products over the next few months. Among these is an APL-to-DIF file interface, which is currently undergoing beta-testing at selected customer sites. Anyone wishing to keep up-to-date with Cocking & Drury's activities should ask to be included in their newsletter mailing list - contact Liz Swann at the London office.

Dyadic Systems Limited

Dyadic has announced an implementation of Dyalog APL for the IBM/PC AT. Dyalog APL for the PC AT runs under Xenix, is multi-user and supports large workspaces. Indeed when using the virtual workspace option, up to 3 users may have active workspaces whose combined size far exceeds the amount of memory (a maximum of 3 megabytes) installed. The implementation for the PC AT is a complete one containing all the features of Dyalog APL found on larger systems.

Dyadic has also announced a session manager which provides a full-screen front end to Dyalog APL. It records input/output in a session log and allows lines to be recalled for editing and resubmission. The session manager is a run-time option which may be selected whenever Dyalog APL is invoked.

Inner Product Limited

Whilst retaining its presence in the consulting and general APL support market, Inner Product has been extending its range of software packages which run under APL*PLUS. Three major new products are now available; two of these - a LOTUS 123/APL interface and a dBaseIII/APL interface - both serve to extend the use of APL for passing data between different proprietary packages; the third, CEMAS, is a package for analysing the European Monetary System and Common Agricultural Policy of the E.E.C.

MetaTechnics Systems Limited

MetaTechnics Systems announce the release of MetaPack, a comprehensive Utilities Package for APL*PLUS/PC. MetaPack comprises MetaScreen (an AP124 screen management system), together with NI (numeric input and editing), Browse (screen-based viewing of reports and text files), Toolbox (APL utilities collection) and MetaWS (a program filing system for large APL systems). Interestingly, many APLers have expressed the most interest in the Assembler versions of several functions; MAKEMAT & MAKEVEC convert delimited vectors into matrices and vice-versa, and IOTA & MEMBERSHIP provide table look-up facilities. MetaScreen will continue to be available as a product in its own right.

Overseas readers will be interested to know that MetaTechnics are currently talking to possible dealers and distributors in various market areas and have already agreed for Sim-Corp, Cybex AB and Metaplan AB to become dealers for their product range in Scandinavia.

In conjunction with Volvo of Sweden, MetaTechnics Systems have developed a system linking APL*PLUS/PC to the IEEE interface bus for controlling laboratory instruments and data-logging equipment. The package requires the use of the TECMAR IEEE 488 interface card.

Continuing with a Scandinavian flavour, MetaTechnics have agreed dealership terms with Andreas Buch, the Norwegian developer of ADAPTA DLS. This is a unique system for scheduling production and purchasing in process manufacturing industries, such as food, drink and chemical manufacturing. It has been found to save process manufacturers an average of 1-2% of their annual turnover. ADAPTA DLS is written in APL and will run on the IBM/PC XT.

MicroAPL Limited

For some time now, MicroAPL have been aware of the increasing need of end-users and programmers alike to operate in a total APL environment. To this end, they are launching a new concept for all APL users called I.A.S. - the Integrated APL Service. I.A.S. is a brand new approach to APL programming. It draws together the often disparate elements of an APL system into a comprehensive corporate structure.

I.A.S. consists of a number of APL modules which can be combined into an APL service of any size, from a single-user system to a company-wide APL network linked to a mainframe. The modules are all designed to work together and include, amongst others, micros and supermicros, terminals and printers, plotters, modems, networking, utility software, applications software, on-and off-line links to both mainframe data and other software (such as Lotus 123), maintenance, support, consultancy and training.

Customers who adopt the I.A.S. strategy are assured of a planned, expandable APL service which integrates mainframe APL systems and corporate data, high-powered APL systems based on MicroAPL's Mirage machines, and smaller PC's such as the QL and IBM/PC into a coherent, distributed processing system. I.A.S. is primarily intended for organizations already using APL, although APL newcomers can benefit from the system by planning their growth into I.A.S. from the start.

APL PRODUCT GUIDE

Compiled by David Preedy

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

The fourth release of the Guide contains several additions of new products and suppliers who slipped through our net in earlier passes. We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry — see address on inside back cover.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to VECTOR readers, the product list has been divided into the following groups:

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL Visual Display Units
- APL character set printers
- APL-based packages
- APL Consultancy
- Other services
- Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the VECTOR working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'

APL PRODUCT GUIDE

Company	Product	Prices £	Details
COMPLETE APL SYSTEMS			
Alan Pearman	IBM PC & PC/XT	1995 – 4425	Authorised IBM PC dealer supplying complete systems including peripherals.
C.A.C.I.	APL Machine	poa	APL-dedicated array processor from Analogic. Many configurations.
Cocking/Drury	MicroAPL SPECTRUM SAGE II SAGE IV	6000 – 35000	Supplied as part of a turnkey system. See MicroAPL entry.
Gen. Software	Myriade	poa	Ti computer with APL and APL operating system
Inner Product	IBM PC	2000 – 6000	IBM PCs supplied for turnkey applications
MBS Rentals	IBM PC & PC/XT	2200 – 5000	For purchase or rental
	COMPAQ	poa	Portable IBM PC compatible
M.B.T.	MBT 10/40	poa	UNIX/68000 based multi-user APL system
	FORTUNE 32:16	poa	FORTUNE 68000 system enhanced for APL
	TORCH	poa	68000/Z80 multiprocessor
MetaTechnics	—	3000 +	Details on application
MicroAPL	Sinclair QL with QL/APL SPECTRUM	from 434 11000 – 15000	Fully expandable APL system with colour graphics. Expandable multi-user APL computer using Motorola 68000. Standard configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	SAGE IV	8500	Multi-user APL computer, 1 Mb RAM, 12/18 Mb disc.
APL INTERPRETERS			
Alan Pearman	APL *PLUS/PC	420 – 695	See APL *PLUS entry
APL *PLUS	APL *PLUS/PC (Release 4)	695	Full feature interpreter for IBM PC, PC/XT, PC/AT & compatibles: CORONA, COMPAQ COLUMBIA, WANG, OLIVETTI, EAGLE, AJ, ERICSON, ITT
	Upgrade	95	

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL INTERPRETERS (continued)			
APL*PLUS	APL*PLUS/UNIX	poa	Interpreter for Unix systems: WICAT, CADMUS, CALLAN, FORTUNE 32:16, HP9000/500
	APL*PLUS/1000	poa	Enhancements to IBM's VSAPL. Runs under VM/CMS or MVS/TSO.
Burroughs	APL/700	3150	Runs on Burroughs B5000, B6000, B7000 and A9 mainframes.
Cocking/Drury	APL*PLUS/PC Release 4.1	695	Discount on multiple orders.
Dyadic	Dyalog APL	1000 - 8000	2nd generation portable APL for UNIX systems eg. VAX, HP9000, Fortune, MC68000, Zilog, Perkin-Elmer, Perq etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
IBM UK Product Sales	IBM PC APL	poa	With event-handling, & APs for full-screen I/O, disks, diskettes, asynch. comms.
Inner Product	VIZ::APL	250 - 350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See APL*PLUS entry
M.B.T.	Dyalog APL MBTAPL	poa	See Dyadic Systems entry
	VIZ::APL	poa	Enhanced Dyalog APL for MBT hardware. Customized for TORCH hardware
MetaTechnics	APL*PLUS Release 4	695	Discount on quantity.
MicroAPL	APL.68000	1000 +	Full implementation with component files, error trapping etc. for SPECTRUM, SAGE & other MC68000-based computers.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL INTERPRETERS (continued)			
MicroAPL	QL/APL(keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL(APL chars)	113	VSAPL compatible APL for QL with many extra features.
I.P. Sharp	Sharp APL	25000	For IBM mainframes
	Sharp APL/PC	55	For IBM PC or PC/XT
APL VISUAL DISPLAY UNITS			
Farnell	Tandberg TDV2221	1498	Ergonomic design APL terminal, 50-19200 baud, 15" anti-reflex screen, low profile keyboard.
	Tandberg TDV2271	1685	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata Elite 3045A	400	Second-hand.
Lynwood	Alpha	1995	Full APL character set, 16-bit microprocessor with pixel-addressable monochrome graphics.
	Alpha Colour	2985	As above with 8-colour graphics.
MBS Rentals	Lynwood Alpha	1995	See Lynwood entry.
	Lynwood Alpha Col.	2985	See Lynwood entry.
	Concept/APL	1491 - 1663	See Shandell entry.
	Concept GVT/APL +	2220 - 2393	See Shandell entry.
	(All above for purchase or rental)		
M.B.T.	various		Contact MBT for details

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL VISUAL DISPLAY UNITS			
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1450	With monochrome graphics
	Tektronix 4105	4595	High resolution colour graphics supporting APL character set on MicroAPL hardware.
Shandell	AVT-APL + GVT-APL +	1200 +	ANSI 3.64; DEC compatible; full overstrike chars; 4/8 pages; 2/3 comms. ports; 80/132 cols; windowing; 12" screen; 46 PF keys; Tek 4013 graphics compatible.
	HDS 200	1200 +	As above plus 15" screen; viewports; smooth scroll; NVM storage; 55 PF keys; Tek 4014, VT640/DQ640 & Visual 500 graphics compatible.
Sigmex	Sigmex 5684	from 6091	High-res colour graphics (768x512x4)
	Sigmex 5688	from 7096	High-res colour graphics (768x512x8)
	Sigmex 5472	from 6649	High-res monochrome (1536x1024x2)
	Sigmex 5484	from 10500	High-res colour (1536x1024x4) (Hard-copy output devices available for all these models.)
APL PRINTERS			
Alan Pearman	Epson FX series NEC Spinwriter PRISM 80 PRISM 132		Prices range from 289 to 1995 over all models.
Datatrade	Datasouth DS180	1395	180 cps matrix printer with 2K buffer, 9x7 dot matrix and APL option.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1180	40 cps., letter quality

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL PRINTERS (continued)			
MBS Rentals	Epson FX series	438-1100	
	Datasouth DS180	1395	See Datatrade entry.
	Phillips GP300	2251	Matrix with letter & draft quality & APL.
	TI745 + APL	1390	Portable dot-matrix.
	DEC LA120	2516	120 cps dot-matrix.
	DEC LA12C	1444	Portable dot-matrix.
	AJ833	2595	Daisy-wheel
	(All above for purchase or rental)		
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL daisy-wheel printer
MicroAPL	Datasouth DS180	1545	See Datatrade entry
	Phillips GP300	1798	Matrix printer with letter and draft quality and APL.
APL PACKAGES			
APL *PLUS	Info Centre	12000 -	Tools for full-screen entry, display and analysis. Interfaces to other proprietary IC products Complete Materials Management, Factory Planning & Forecasting package. Comprehensive package for IBM PC and compatibles. Utilities for IBM PC;IRMA 3278 comms, full-screen, RAM disk, report generator Interactive Statistical Graphics
		20000	
	CMCS	30000 -	
		100000	
	Finance/Statistics	275	
	Tools Vol. 1	poa	
C.A.C.I.	STATGRAPHICS	poa	Language-driven data extraction, and foil preparation. VM/CMS VSAPL, AP126. PF-key driven personal information manager VM/CMS VSAPL, AP124
	PILGRIM	20000	
	PIM	3000	
Cocking/Drury	ARMS 2	3000 -	Applications Generator
		15000	
	AFM	10000	High performance shared file system.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL PACKAGES (continued)			
Cocking/Drury	CALL/AP	3000	Non-APL program execution.
	FORMAT	2250	Enhanced report formatting
	WSAIDS	795	Workspace documentation and development aids.
E&S	PROTOPAK consisting of:		Packages for prototyping management information systems — PC & mainframe
	RMS	550-2500	Relational databases.
	AMS	550-2500	Multi-dimensional arrays.
	RAMS	750-3500	Combined RMS & AMS.
	BMS	750-3500	Dynamic financial modelling & forecasting
	FMS	250	Full-screen handler for IBM PC. (AP124-based)
	CMS	poa	Communications package.
Gen. Software	PROPS	from 500	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
Holtech	CASH	3500 – 10000	Accounting package and hotel management system on MicroAPL SPECTRUM & SAGE CPUs.
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBASE III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3500	EEC monetary and agrimonetary analysis.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL PACKAGES (continued)			
M.B.T.	RHOMBUS	poa	Integrated Office System
	HASLEMERE	poa	Hotel Accounting System
MetaTechnics	MetaScreen	195	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	395	Comprehensive utilities package for APL*PLUS/PC. Includes MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Project development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc.
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
Packer	Budworth Genealogy System	30	Record-keeping, documenting, sorting & pedigree-chart printing for professional & amateur family historians. Uses IBM/PC APL.
Parallax	ExecuCalc	\$5000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.
	ExecuPlot	\$5000	Mainframe-based colour graphics system with micro links.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL PACKAGES (continued)			
I.P. Sharp	ACT	poa	Actuarial system
	APS	poa	Financial modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/ reporting
	MAGIC	poa	Time series analysis/ reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail
	MICROCOM	poa	Mainframe to micro link
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL — Info centre product
	XTABS	poa	Survey Analysis

APL CONSULTANCY (prices quoted are per day unless otherwise marked)

Alan Pearman	Consultancy & Courses	25	Per hour, onsite and offsite
APL People	Consultancy	poa	All levels, most APL systems
Attikale	Consultancy	160	Management reporting systems & Information Centres.
C.A.C.I.	Consultancy	poa	All aspects, specializing in Decision Support Systems.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Cocking/Drury	Consultancy	120-150 140-200 185-300 275-400	Junior consultant Consultant Senior consultant Managing consultant

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL CONSULTANCY (continued)			
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming and training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
Gen. Software	Consultancy	from 100	
H.M.W.	Consultancy	100-250	System design consultancy, programming.
Inner Product	Consultancy & Training	200	On-site micro/mainframe APL, PC/DOS & Assembler
M.B.T.	Consultancy & Courses	poa	
MetaTechnics	Consultancy	poa	Management Information & Production Management Systems & Process Control.
	MetaSys	poa	Prototyping APL-Hybrid Custom Programming Service.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Packer	Consultancy	150	Specialising in Microcomputer & Operational Research applications.
Parallax	Consultancy & Training	\$750 poa	Introductory APL, APL for end-user & Advanced Topics in APL.
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL CONSULTANCY (continued)			
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
OTHER PRODUCTS			
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short or long term projects, supervised.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms. software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

APL PRODUCT GUIDE (continued)

VENDOR ADDRESSES

<u>Company</u>	<u>Contact</u>	<u>Address & Telephone</u>
Alan Pearman Ltd.	Alan Pearman Maria Pearman	96-98 Chester Road, Huntington, Chester CH3 6BT. 0244-46024
APL People	Valerie Lusmore	3 Cross Lane Close, Orwell, Royston, Herts SG8 5QW. 04626 77375
APL *PLUS Ltd.	Barrie Webster John Ward	Aston Science Park, Love Lane, Birmingham, B7 4BJ. 021-359 5096
Attikale Ltd.	Peter Hurdley	51 Brookside, Paulton, Bristol BS18 5YR. 0761-415427
Burroughs Machines Ltd.	M.J. Fennel	Heathrow House, Bath Road, Hounslow, Middlesex TW5 9QL 01-750 1400
CACI Inc. International	Paul Thornett	Oriel House, 26 The Quadrant, Richmond, Surrey, TW9 1DL. 01-940 3606
Anthony Camacho		2 Blenheim Road, St. Albans, Herts., AL1 4NR. St. Albans 60130
Cocking & Drury Ltd.	Romilly Cocking	16 Berkeley Street, London W1X 5AE. 01-493 6172
Datatrade Ltd.	Tony Checksfield	38 Billing Road, Northampton, NN1 5DQ. 0604-22289
Delphi Consultation Ltd.	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. 03677-384
Dyadic Systems Ltd.	Peter Donnelly	30 Camp Road, Farnborough, Hants. GU14 6EW. 0252-547222
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. 0689-24741

APL PRODUCT GUIDE (continued)

VENDOR ADDRESSES (continued)

Company	Contact	Address & Telephone
Farnell International Instruments Ltd.	R. Fairbairn or Roger Attard	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. 0937-61961 Davenport House, Bowers Way, Harpenden, Herts. 05827-69071
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middlesex UB5 4QS. 01-864 9537
H.M.W. Programming Consultants Ltd.	Ken Jackson	142 Feltham Hill Road, Ashford, Middlesex TW15 1HN. 07842-41232
Holtech Ltd.	Jan Bateman	'O' Block 4th Floor, Metropolitan Wharf, Wapping Wall, London E1 9SS. 01-481 3207
IBM UK Product Sales Ltd	Stephen Voller	PO Box 32, Alencon Link, Basingstoke, Hants., RG21 1EJ. 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. 01-673 3354
Lynwood Scientific Developments Ltd.	Gareth Wokes	Park House, The High Street, Alton, Hants GU34 1EN. 0420-87024
M.B.S. Rentals	Robert Notley	119/120 High Street, Eton, Windsor, Berkshire. 07535-68171
MetaTechnics Systems Ltd.	Steve Margolis	Unit 216, 62 Tritton Road, London, SE21 8DE. 01-670 7959
MicroAPL Ltd.	Richard Nabavi Paul Thornton	Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London SW8 5BP. 01-622 0395
Modern Business Technology Ltd. (M.B.T.)	Michael Branson	P.O. Box 87, Guildford, Surrey GU4 8BB. 04868-23956
S.H. Packer		Cleveland House, Little Budworth, Tarporeley, Cheshire, CW6 9BP. 082-921 439

APL PRODUCT GUIDE (continued)

VENDOR ADDRESSES (continued)

<u>Company</u>	<u>Contact</u>	<u>Address & Telephone</u>
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. 212-475-4001
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Road, Camberley, Surrey, GU15 1LB 0276-20789
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. 02407-2027
I.P. Sharp Associates Ltd.	David Weatherby	10 Dean Farrar Street, London SW1 01-222 7033
Sigmex Ltd.	Alan Taylor	Sigma House, North Heath Lane, Horsham, W.Sussex, RH12 4UZ. 0403-50445

BOOK REVIEWS

Compiled and reviewed by Robert Bittlestone

Books are welcomed for review on any topic likely to be of interest to the APL user, and should be sent to the Editor at the address on the inside back cover.

LISP: A Gentle Introduction to Symbolic Computing; Touretzky, Harper & Row 1984. £11.95 from Harper & Row, Tavistock Street, London WC2.

Question: what's the difference between LISP and APL? Answer: they're both over twenty years old, but LISP is getting more popular. It certainly shows that it takes more than a funny vocabulary to put people off a language. One hundred zillion Chinese can't be wrong.

If you don't know LISP already then you can certainly learn it with this book: the introduction is indeed gentle and sprinkled with wry humour. The first few chapters present LISP processing using a functional style that would work almost as it stands as a description of APL functions with arguments that return explicit results. Then we move off into the lisping world of CARs and CDRs (the legacy of bygone development days, Content Address Register and Content Decrement Register, as near to the "1 Take" and "1 Drop" of APL as makes no difference).

There's a good bibliography and some real worked examples later on. Read it, and you'll probably agree that APL could do it all with its eyes shut, and APL2 with no eyes at all. So where did we go wrong? I still blame our silly symbols. Even if technology is now no longer a penalty for implementing special character sets, it certainly used to be — and by now it's too late.

Fuzzy Sets and Decision Analysis; Zimmermann, Zadeh & Gaines, North-Holland 1984. Available from Elsevier Science Publishers B.V., P.O. Box 1991, Amsterdam.

The interplay between techniques and applications in a discipline such as Operational Research is an interesting one. Students tend to start by imagining that the techniques are all that matters, often because of the difficulty of presenting real applications in an academic environment. Later, in the heady confrontation with "trouble up at t'mill" they may well over-react, praying that a blight may descent on the professors who made it all sound so neat, tidy and mathematically elegant. But in what I believe to be the final phase of this particular enlightenment, they tend to acknowledge the stimulus to creative thinking that comes from the exploitation of a genuine new technique, initially for its own sake.

In this context, fuzzy set theory ranks with catastrophe theory, the laser and the linear induction motor as having been described as a "solution looking for a problem". However, patience is its own reward; the laser gets Reagan's Star Wars as its prize; the rest — perhaps the best — is yet to come.

A brief reminder of what fuzzy sets are all about may be in order. Conventional mathematics allows us to make precise statements. For example: "John is over 180 cm tall". Whether such a statement is true or false is a separate issue; the statement itself is intentionally exact. Fuzzy set theory relaxes this unnatural tyranny by introducing the concept of "degree of membership", a real number varying over the interval 0 to 1. This allows us to recast this statement as "John belongs to the set of over 180 cm tall people with degree of membership 1". More interesting things start happening once the degree of membership drops below 1.



COCKING & DRURY LTD.
THE APL PROFESSIONALS

Enhance your APL Information Centre

An enhanced APL will improve programmer productivity — but what if you can't afford to be cut off from future IBM Information Centre products?

Cocking & Drury have a range of powerful enhancements to VS APL that will not commit your company to a non-IBM strategy.

- * Multi-user APL filing system
- * Enhanced report formatting
- * High speed printing
- * Mailbox facilities
- * Execution of non-APL programs from APL

**For further information call us on 01-493 6172 or write to
16 Berkeley Street, London, W1X 5AE**

Almost before the ink was dry on the definitive 1970 article on the subject by Zadeh (for that is he) and Bellman (inventor of dynamic programming, not to be confused with COBOL), the academic hounds were unleashed and promptly tore into the authors with the accusation that beneath the fuzzy emperor's clothes were the unmistakable pyjamas of probability theory. This has made for a nice controversy ever since. However, with the coming of expert systems, genuine interest has reawakened in using fuzzy set techniques to design routines for user dialogue. These ought to allow people to respond adverbially rather than by interjection: it's always more helpful to reply "Probably" to the computer's question "How old are you?"

Given this degree of topicality it is a pity to report that this collection of papers contains not one useful practical example of the application of the technique to a real problem. The nearest we get is an abstruse discussion of predicting television revenues using fuzzy sets by a pair of American researchers. Admittedly the book doesn't try to be a do-it-yourself guide, but as a selection of papers one might reasonably have expected a balance between the academic and the practical. Since all that is left to comment on are the integral signs, I must declare myself incompetent. Nevertheless, I refuse to be intimidated by Zimmermann's approach in one of his own papers: "Proof: The proof is almost trivial...". There follow several paragraphs of algebraic formulation terminated by an academically witty exclamation mark (!).

Powers of Ten: About the Relative Size of Things in the Universe; P. & P. Morrison, Scientific American Books Inc. 1982.

Available from Freemans, Beaumont Street, Oxford.

This is one of the most delightful books I have ever seen, and "seen" is the operative term. The main feature of the book is a series of 42 photographs, each forming a square grid and related to its successor by an order of magnitude along each dimension. For example, the photograph of a square kilometre of Chicago viewed from the air is followed by a closeup of its centre, 100 metres on each side. The area of each successive photograph is therefore one-hundredth of its predecessor. This sounds unremarkable, almost pedestrian; but now extend the concept to a starting square with sides of 1 billion light years (10 to the 25 metres), expanded by 42 steps through to one of 0.1 fermis (10 to the minus 16 metres).

We gain many realisations from this wonderful book. Not the least is some sense of the recursive nature of the Universe. The photograph of stellar nebulae looks so like that of the electrons of the carbon atom. To speculate that our whole Universe is contained within some quark in another dimension of matter is a popular theme with science fiction writers, but if you look through this book you may even believe it. Worse still, you may wake up with the nightmare, as I did, that this recursion is closed: the quark that contains our Universe is the one that we plot in a bubble chamber.

As an aid to humility, notice also that the first photograph of a recognisable Earth occurs as 100 thousand kilometres away (10 to the 8 metres) while the smallest frame recognisable to the non-specialist is that of a human hand in Chicago, with area 10 by 10 centimetres (10 to the minus 1 metres). This range of 10 orders of magnitude out of the 42 depicted in this voyage is modest indeed; and for all of us save the astronauts, our lives are limited to a mere seven of these.

This is excellent science at work, and one suspects a labour of love by the Morrison duo. Philip Morrison teaches Physics and Astronomy at MIT and reviews books for Scientific American. Phyllis Morrison teaches children and their teachers (sic) science and art in classroom and museum. I wonder what the Morrises chat about after watching Dynasty. Well, maybe they're the 0.00002% of Americans who don't. This particular trip must make Denver Carrington look like a vicar's tea party in Nether Wallop. So buy the book and I will buy it back from you myself if you're underwhelmed.

SOFTWARE REVIEWS

IBM PC/APL, APL*PLUS/PC and PC/AT, M68000.APL on Sage IV

by Adrian Smith

Introduction

What follows is by no means an exhaustive review; more a case of simple opportunism when we found ourselves with a MicroAPL Sage and an IBM PC/AT on adjacent desks. The 'benchmarks' were the result of at most 10 minutes thought, and should obviously be extended if you are seriously considering any of the above as a vehicle for some particular system. There may also be other overriding factors, such as the speed of the file systems, which I have not been able to try out.

First Impressions

It is very hard indeed to like the IBM PC/APL. I don't quite know why, but it just doesn't present a particularly friendly face! Partly it is annoying to be forced into installing the 8087 co-processor (so far two of ours have fried); partly it is the use of the graphics characters; partly the lack of 'add-ons' to make use of the PC. True, there is a selection of auxiliary processors, but they are very limiting; for example you cannot use colour in your screen design (AP205), and there is no real file system.

There were also some pretty obvious bugs - IBM did get them fixed quite quickly, but I can't believe anyone else could possibly have been making serious use of the system for any length of time. Conclusion - this APL is really only worth considering in the absence of something better. It is included in the benchmarks for completeness (and maybe to embarrass STSC into doing something about booleans!) rather than for genuine comparison.

M68000.APL is by far the most aesthetically satisfying of the set. Similarly the architecture of the Sage IV is little short of brilliant; together they should prove a winning combination, but there are a couple of very serious reservations. The first is price - you won't get much change out of £9,000; the second is the total lack of the kind of 'programmer support' which is offered almost to excess by the remaining APL in the group - APL*PLUS/PC.

Basically I can find a lot in this APL to object to in principle, and a lot to like in practice! The use of two-byte integers; the restriction on object size; the ultimate limit to workspace; the shoals of quad-functions: all these ought to be serious gripes. So far I have found the sheer practical useability of STSC's APL to be so great that I can live with an awful lot of these boddies, rather than take the path of true righteousness offered by MicroAPL and Phil van Cleave.

The Benchmarks

The benchmarks used (and the APL objects required to run them) are listed below. They were designed in part to test the relative speed of the chips, and in part to investigate the short-cuts taken by the various interpreters. I would stress again that the only true benchmark is the system you intend to run; treat all such abstract tests with a hefty dose of scepticism.

APL OBJECTS REQUIRED ...

=====

V+1000p4 5 6 0

M+100 10p2.5 6.7 0 0 8 -5.7

S+10000p'THE CAT SAT ON THE MAT '

S[999+16]+ 'ADRIAN'

I+100?1000

B+10000pV=6

FF+800 5p'AASHJ RHF KJDSHJ '

FF[601;]+ 'EJD..'

V FN N;CT;X

[1] CT+0

[2] Lp:-(N<CT+CT+1)+Exit

[3] X++/ 2 3 p16

[4] +Lp

[5] Exit: 'Done'

V

=====

BENCHMARKS USED ...

=====

Type	Bench Mark	PCAPL	PCPLUS	PCPLUSX	ATPLUS	SAGE
INT	+ / V + V	520	370	390	160	188
INT	+ / V [I]	180	70	60	50	17
REAL	V + . * V	1180	480	440	160	125
REAL	+ / V * 2	2450	2080	4270	1200	1361
REAL	X + + / M	228	490	770	238	177
REAL	V + . * , M	1250	1190	2750	770	720
CHAR	+ / S = 'T'	12350	3840	3790	1954	1485
CHAR	'FAT' € S	100	40	60	50	48
CHAR	X + X € 'FAT'	3650	2940	2810	900	985
SS	□SS(S; 'ADRIAN')	-	-	-	-	275
SS	(S □SS 'ADRIAN') / 1pS	-	1640	2040	600	-
BOOL	X + BVBA ~ B	130	3940	4950	1900	25
BOOL	A / B	3550	150	170	100	115
BOOL	X + v \ B	6150	270	270	160	145
FN	FN 10	400	490	390	160	85
FN	FN 100	2850	3940	3800	1200	915
FN	FN 1000	26850	36440	35600	12000	9085
IP	X + FFA . = 'EJD..'	7050	80	60	60	485
IP	X + FFA . = 20 5p 'EJD..'	139850	34940	35050	13500	10485
=====						
PCAPL	: IBM APL on IBM PC		PCPLUS	: APL*PLUS on IBM PC		
PCPLUSX	: ... without 8087		ATPLUS	: APL*PLUS on PC/AT		
SAGE	: Sage IV with M68000.APL					
=====						

The results are also shown graphically on the following pages.

The first obvious comment is on the complete lack of relationship between IBM's APL and APL * PLUS. Basically there is very little in it on average; but it is a pretty meaningless average! However I do wish that STSC had taken booleans seriously; OK they've optimised the reductions to death (unlike IBM), but some of us rather like to use APL to solve logical problems!

It is also interesting to note that APL * PLUS could well be faster without the 8087 co-processor. It obviously depends heavily on the type of processing you do, but my guess is that most commercial APL will be better off without it. Of course it only really makes sense to compare the 8087-less PC with the AT; the magic number is 0.361 (i.e. the AT is almost precisely 3 times as fast) with very little variability. This has the useful side effect that you can set up systems on the PC and be absolutely confident that you will get a predictable speed-up when you move them to the AT.

On balance, the Sage remains the fastest processor of the set, particularly when it comes to function execution. The string-search algorithm is pretty sneaky too, and the lack of an optimised inner product hardly matters. However I was quite surprised to see the AT running it so close on simple arithmetic and character handling. For a full system the AT would probably feel faster, as APL * PLUS drives the screen much the quicker of the two.

The Hardware

There is no doubt about it; the IBM/AT has got the keyboard absolutely right! I like the PC keyboard anyway, and the few minor quibbles have been well corrected for the AT. The screen (assuming you want colour) remains a let-down; until there is a decent one available you really ought to stick to monochrome for serious APL work. Maybe one of the optional graphics boards would be a good idea? Unfortunately I haven't had a chance to try out any of these.

We ran our Sage with an Insight VDT-1, complete with MicroAPL keyboard. This is perfectly workable, but has nothing like the quality feel of the AT. In general I would say that it rather let the Sage down, but of course it is at the cheaper end of the MicroAPL range. Not really a fair comparison.

What about that APL character generator? You can now run APL * PLUS on a colour screen in graphics mode rather than swopping the chip. In my view a non-starter; you lose all the nice scrolling and editing, and it burns nasty little holes in your retina. In order of preference, I would run my APL on:

1. Monochrome monitor with character ROM fitted.
2. Colour monitor as above.
3. Mono monitor without ROM using symbols (you soon get the hang of 'square-root' for 'RHO'), rather than keywords.
4. Colour monitor as above.
5. Either, running keywords.
6. Colour monitor in graphics mode.

I also object to messing up my keyboard with stickers; experienced APLers will probably find that they know the majority of the key positions anyway; I found it quite good discipline to have to learn some of the odder ones! Incidentally, why have they moved 'Goto' to lower-case, rather than leaving it on upper-case 'Assign'? This is about the only difference from the standard IBM layout and it is very annoying!

The APL Itself

You pay your money and takes your choice! All the implementations are full APLs, with odd quirky bits round the edges. IBM is as near as dammit straight VS APL, with APs to play tunes, drive the screen and so on. STSC have gone quad-function crazy, but their interpreter seems to fit happily enough in a 256K PC, so who cares. You only need to learn the ones you want, after all. APL 68000 has some helpful oddities too (such as quad.BOX to do the vector to matrix bit); it really depends what sort of things you like, and what sort of systems you want to write. Basically they are all very good APLs.

I suppose the most obvious difference lies in the way they drive the screen. Naturally IBM play very much by the rules; everything goes via the PC BIOS and you drive the screen through a cut-down AP124. Much the same applies to the Sage, only this time you have a usefully augmented AP124 to go at. Both these work very well, and have the big advantage that mainframe code will run with only minimal modification.

APL *PLUS completely reverses this approach; it merrily leapfrogs the BIOS and splats everything straight to the screen from memory. This leaves you with the job of re-writing your full-screen code to use the various quad-functions, but has the benefit of being extremely fast once you have done so. As a typical example, my 'DRAW' routines (Design Handbook, Chapter 10) run rather faster on APL *PLUS/PC than on the Sage; in fact with a bit of tuning I suspect they will push mainframe APL pretty close! This is simply due to the ultra-fast screen I/O - the APL code itself runs no faster than you would expect from the table of benchmarks.

The Environment

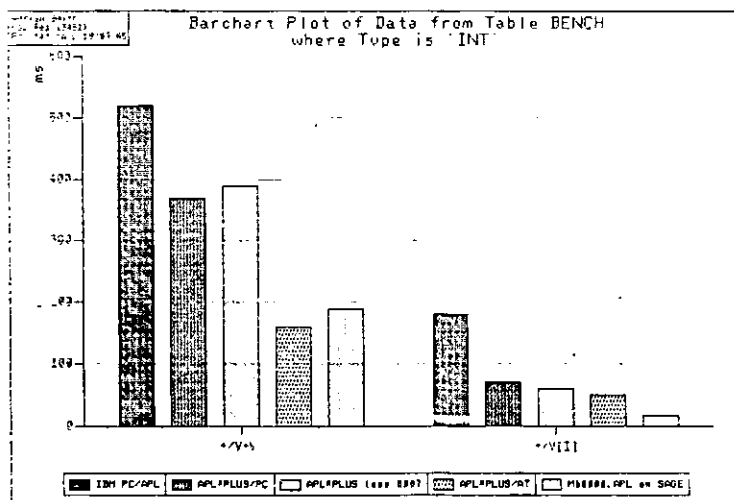
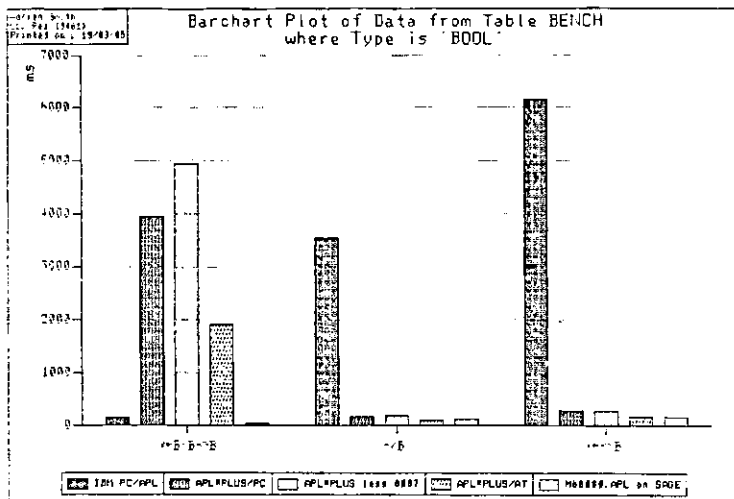
This is the one area where APL *PLUS really scores over M68000.APL. The inbuilt editor is first class, and the use of function keys and on-screen scrolling make for an ideal APL workstation. Some of the key definitions are a bit weird (like having to hit ALT-INS to toggle insert mode!) but you learn them pretty fast and at least STSC have been consistent - the same keys do the same things in the editor as in terminal mode.

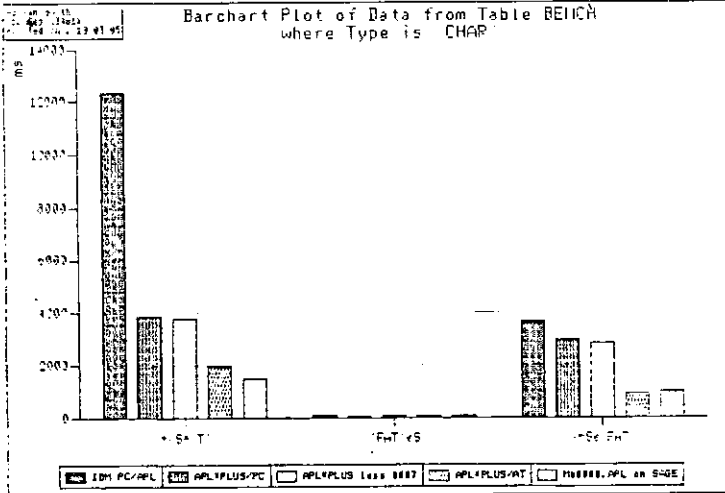
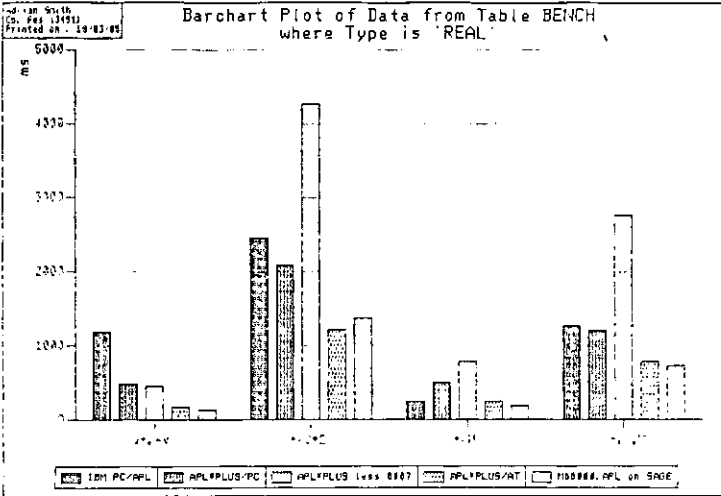
With the Sage you are basically back in teletype mode; you can get your last line back for modification (using all those funny DEL commands most of us forgot years ago) but there is very little by way of programmer support. You need to pay extra to get access to the operating system files, and to get any sort of full-screen editing. I don't see how MicroAPL can go on charging £250 each for two essentials that APL *PLUS throw in for nothing!

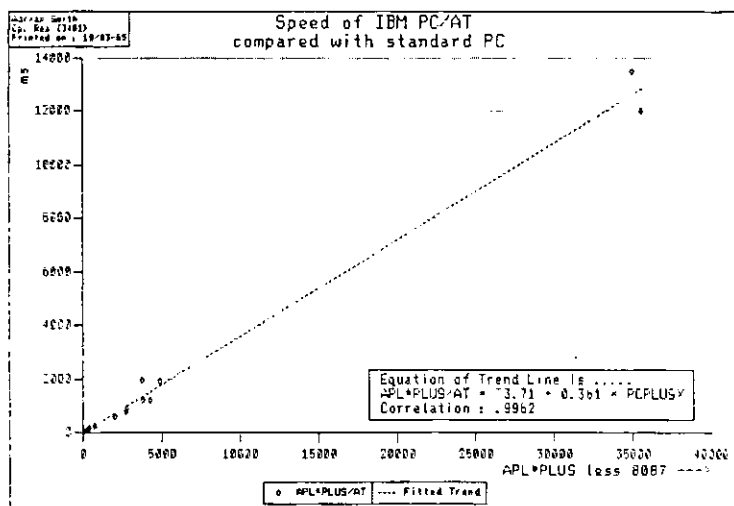
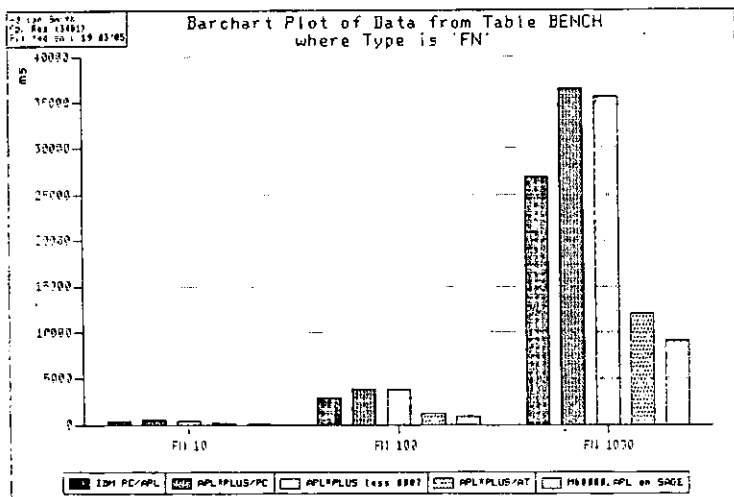
The operating system of the AT is standard PC DOS, and is hence very familiar to a vast number of PC users. STSC use normal DOS files to cover their component file system; in our limited experience this seems quite adequate, although I have heard rumours to the contrary. The Sage is a multi-user system, with a very much more complex operating system called Mirage. To a large extent this is tuned specifically for APL processing, so the native file system is very fast and extremely powerful. The drawback is the someone has to learn enough Mirage to support your APL user's; it is by no means straightforward, and has some interesting traps such as a 'Copy' which goes from right to left!

Summary

The AT is pretty cheap (say £5,500 for an all-up system) and provides a natural upgrade path from the PC. It is the obvious choice for stand-alone commercial APL processing. The Sage is a super system in principle, but lacks many of the friendly features that one is getting to expect on micros. It may be the only choice if you need multiple users or really big workspaces. It begins to look overpriced. IBM's PC/APL will not run on the AT, and it is not really worth looking at even on the PC; sorry.







RECENT MEETINGS

This section of VECTOR is intended to document the seminars delivered at recent meetings of the Association, particularly for those members who work outside London and often find it hard to spare the time to attend.

We are dependent on speakers for their willingness to provide us with a written version of their seminars, and we would remind them that "a picture's worth 1000 words". Copies of slides and transparencies will enhance their articles.

The Activities officer (details on inside back cover) will respond enthusiastically to offers from individuals to contribute seminars and supporting papers.

MetaPack... Everything you need for APL on the PC.

MetaPack is the utilities package that has been designed especially for users of APL★PLUS/PC.

MetaPack comprises a number of programs, each designed to help you build better systems in less time.

MetaPack will increase your productivity as an APL systems builder, and enhance the function and appearance of your systems.

The **MetaPack** programs are:

- MetaScreen** — the popular API 24 based full-screen management system. The speed-critical modules have been written in assembler, making **MetaScreen** ideal for running mainframe applications on the PC.
- NI** — for full-screen input and editing of APL numeric variables.
- Browse** — an attractive and sophisticated method of viewing reports or text files on the screen by horizontal or vertical scrolling.
- ToolBox** — an invaluable collection of APL utilities, including table-lookup and vector-matrix conversion functions written in assembler.
- MetaWS** — the program filing system for easy management and documentation of large APL systems.

MetaPack programs have been written in a combination of APL, C and Assembler. This means that your systems can benefit from **MetaPack's** advanced features yet still run with realistic execution times.

The programs may be used either as stand-alone utilities or as an integrated part of your systems.

Whether you're a professional APL systems builder or an end user who has discovered the power of APL, you will find **MetaPack** an indispensable aid to building better systems.

MetaPack — £495 **MetaScreen** (available separately) — £245
For IBM PC XT AT, COMPAQ & Compatibles

APL★PLUS/PC is a registered trademark of STSC, Inc.

MTS

MetaTechnics Systems Ltd

Unit 216V, 62 Tritton Road, London SE21 8DE, 01-670 7959

The Systems Builders

Introductory Notes

by Adrian Smith

This issue of VECTOR includes one paper held over from the 1984 programme (Fred Perkins on the Information Centre - see General Papers), and my notes on the first 1985 meeting (APL and Graphics). This was our first visit to the Royal Over-Seas League, and I personally liked the new venue very much. The staff were extremely helpful, and the general level of technical support (and availability of tea) were a significant improvement on Imperial. I just hope they can keep it up!

The subject of Graphics, and the way APL systems get access to graphical output, is a growing and lively issue. Consequently I was not surprised to see a turnout well into the eighties, and to get a strong feeling of enthusiasm from those present.

APL and Graphics March 15th 1985

The Royal Over-Seas League

Three papers were presented at this meeting. We began with Richard Nabavi, who explained the background to GKS, and why we should start thinking seriously about its relationship with APL. Richard was followed by Peter Donnelly of Dyadic, who gave an action-packed demonstration of a new graphics package for their Unix-based APL. Finally I rounded off the afternoon with some slides taken from a number of typical mainframe systems, under the general heading of 'Getting the best from GDDM'.

All the notes are mine - because of the very tight production schedule for VECTOR-4, I will not have time to check them with the individual speakers; apologies in advance for the inevitable mis-representations.

APL and the Graphical Kernel Standard (GKS)

Richard Nabavi (MicroAPL)

... with the emphasis on the word *Standard*.

It is well known that standards imply:

- some reduction in facility.
- poorer performance.

... but at least your applications are portable. Of course this leads to an ambivalent attitude on the part of suppliers (notably IBM) who find it much harder to lock people in to a 'standard' system. APL itself is highly standard (and hence APL systems are very easy to move), but the same is not true when you start doing graphics. This problem also applies to FORTRAN, PASCAL etc., so it was from here that the motivation for GKS first came.

What is GKS?

GKS is an abstract model covering around 300 graphics concepts (e.g. 'POLYLINE'). In many ways it is similar to GDDM, and in the same way one finds that different languages get at the facilities in different ways (called the 'language binding' in the standard). Outside the standard are various agreed bindings for FORTRAN, PASCAL etc. ... what is needed is an agreed binding for APL (just as AP126 is the default standard for access to GDDM).

Why bother? There are several good reasons for defining such a standard:

- it will persuade software developers to take APL seriously. In particular anyone developing
- graphical software will be encouraged to include APL in their list of target languages.
- it will make APL more independent of particular hardware.
- the hardware itself is becoming increasingly GKS oriented.

So, having decided that the whole exercise is worthwhile, what kind of APL binding should we use?

What is the Form of the Binding?

There are essentially three options, none of them a clearcut winner. Firstly, you could use quad-functions. The snag here is that there could be around 300 of these! Even with some cunning use of dyadic calls to rationalize things, this is still a whopping patch to the interpreter.

Secondly, you might go via an auxiliary processor and shared variables. This has the benefit of familiarity (at least to regular GDDM users), and is potentially very efficient. It resides outside the APL interpreter, so space is much less of a problem than in option-1. However the ISO standard for APL does not specify shared variables! This pretty well rules this one out, as it would be very illogical for an ISO-conforming APL to be unable to access a standard GKS binding!!

The third option is to leave it all to user-defined functions (which are part of the ISO standard!). The merit of this approach is its universality, and the fact that no patches are needed to the APL interpreter. However it is likely to be much less efficient, and you will probably end up with rather a lot of functions. GKS is *extremely* comprehensive, and covers all sorts of CAD/CAM requirements as well as more familiar graphics.

Underneath the covers these functions will doubtless consist of incomprehensible strings of quad.ARBOUTs and quad.WPUTs; the point is that they can be written once, and then forgotten about. As with GDDM there will probably be at least two levels of APL software buffering the application developer from the real guts of the GKS binding.

Where Next?

A good start would be a UK initiative to get things going; hopefully Seattle will provide a forum to start real discussions on GKS. VECTOR is always available to publish reactions to this proposal; for more details see VECTOR (Vol.1 No.2) or any of a range of literature on GKS. Beware - some of this is not for the faint-hearted!

Duck à la Carte

Peter Donnelly (Dyadic Systems Ltd)

Anyone who attempts the simultaneous use of two video projectors is either brave, or foolhardy, or most likely both! Peter survived the afternoon with only one brief onset of mains ripple, and the temporary loss of one blue gun (miraculously restored by the naked lady). Congratulations!

We were given a very entertaining guided tour through CISIGRAPHE-C, which has been developed over 10 years by Dyalog's French distributor. Originally written for VS APL as a Graphpak replacement, it embodies a lot of GKS principles, and extends GKS into three-dimensional drawing. The interface from APL is by 'external functions' rather than an auxiliary processor, and from the demonstration it was clearly easy to use and very powerful.

Peter took us from simple line drawings (hence the 'Duck' of the title) through more complex shading and use of multiple windows to some very sophisticated business graphics. As with Graphpak, these are written as APL functions which unscramble '... gimme a piechart ...' into the necessary graphics primitives. There were some very nice features, particularly the ability to use a cross-hair cursor to enter information directly onto the graphics image.

My one reservation about the demonstration was the speed of the business graphics. Unlike the simple line drawing (effectively instantaneous) the four sample charts took 42, 90, 16, 44 seconds respectively. They were pretty complex, but even so you would be unlikely to get a serious business chart in under 30 sec; maybe this package is really aimed at the CAD/CAM user where the inbuilt 3D and fast vector drawing (an entire circuit diagram could be held in a single [n;3] matrix) would be ideal?

Getting the Best out of GDDM

Adrian Smith (Rowntree Mackintosh)

Basically, this was a slide-show to illustrate the range of use to which Rowntrees are putting GDDM. The slides began with simple alphanumeric screens, demonstrated the effective use of both field and character colours, and then moved on to simple line-drawings and finally 'business graphics'. Several slides are reproduced (unfortunately not in colour!) on the following pages. Three points were stressed:

- you must integrate graphical output with conventional alphanumeric screens. There is nothing 'special' about graphics.
- you should aim for consistent results whatever route you take. Whether CICS-PL/1, Chart or APL, the end-product should look the same.
- you should make the full power and range of GDDM available to your APL systems.

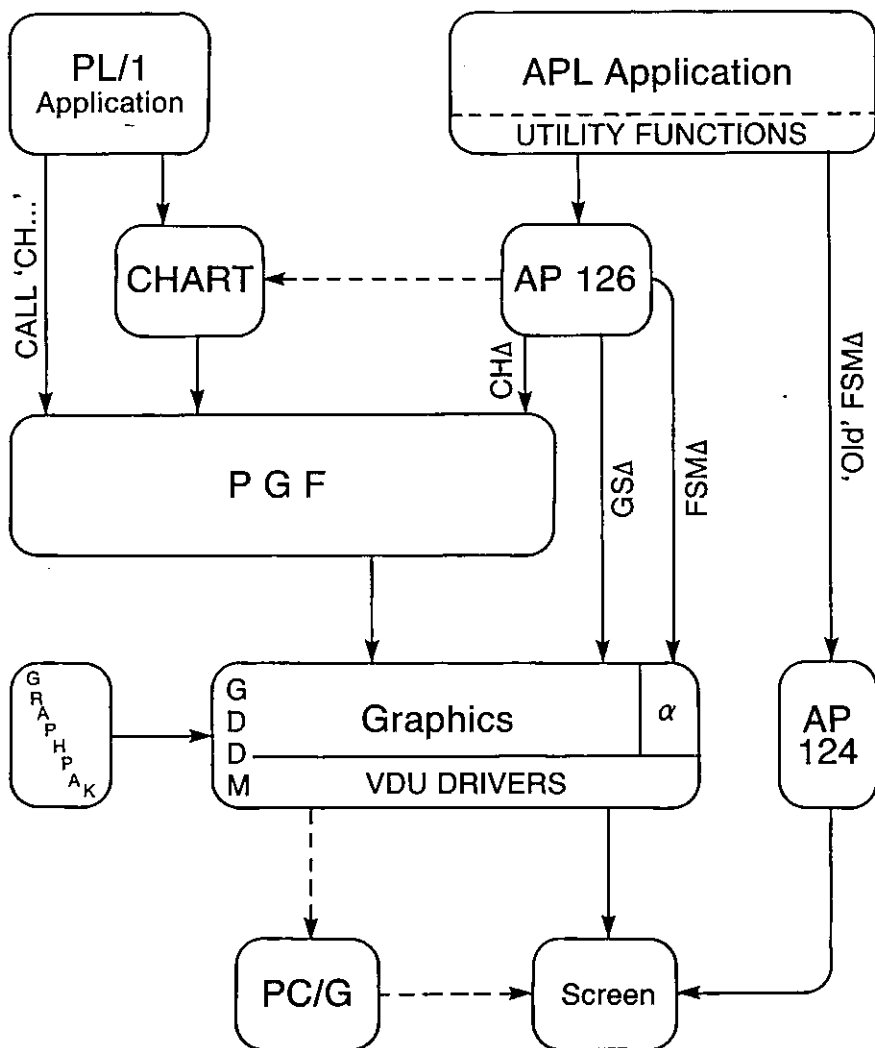
A fourth (tongue in cheek?) point concerned the correct way to approach APL Graphpak; a barge-pole of at least 12ft is recommended!

Which Way to GDDM?

Figure 1 shows the enormous variety of routes from APL to the physical screen. The first recommendation is to cross out AP124; AP126 is around 30% cheaper even for simple alphanumeric, so it makes obvious economic sense to convert to it. In particular if you want to mix alpha and graphics on the same screen (a pretty fundamental requirement in my view) you must send them both through AP126.

How Many Routes to GDDM?

Figure 1



The second recommendation is to ignore APL Graphpak. This goes in at the GDDM level; for reasons about to be explained this is terribly expensive, and a most inefficient use of APL. It also gives APL graphics a 'house style' which is instantly recognisable, but is different from everyone else: probably a bad idea in the long term.

One possible route (GDDM Release-3 upwards) is to go via the Interactive Chart Utility (ICU for short); Tim Perry of APL People has used this method with great success. For presentation graphics (not a major use of APL at Rowntrees) this is probably a very reasonable option; however I personally dislike it on two grounds:

- ICU takes over the whole screen; most of our applications require some alpha fields as well as graphics.
- ICU restricts the options available to you. It is pretty difficult (?impossible) to superimpose different chart types, to put rings round points, and generally to 'tart up' the finished product.

There is also a possible 'not invented here' element in my aversion to ICU - readers must judge for themselves.

My own favourite route from APL to graphics is as follows:

- make sure all your screen-design and screen-access utilities handle graphics naturally, not as a special case.
- push everything through AP126, which splits the requests into three types
 1. Alphanumeric (including field and character colouring). Character colouring is often cheaper than the same effect achieved using multiple fields, as long as you only set a few positions to be different from the background.
 2. Drawing lines, shading areas, and writing text at arbitrary sizes, angles, directions. This is handled directly by GDDM.
 3. Business graphics, with detailed control over axes, titles, labelling, chart notes and so on. This goes into PGF, which uses efficient compiled code to turn it into the basic GDDM instructions appropriate to type 2.
- provide a consistent set of APL cover functions to handle the AP126 requests. I suggest FSM... CH... GS... to cover routes 1,2,3.

The Benefits of PGF

The main advantage of PGF is its speed; a factor of 5 at least over APL Graphpak. This implies CPU timings in the 150 - 400ms band (IBM 3083 running MVS/VSPC) for the typical range of business graphs, giving response times of between 2 and 5 sec on local screens. There is also rather less code to clutter up your workspace - Graphpak comes out at 3600 + lines of APL (some 110K); the PGF route requires a mere 543 lines (20K) of which about half is comments. In fact many of the functions are quite trivial - see for example the listings of GS.delta.DRAW and CH.delta.PIE.

GSADRAW MAT

```

[1]  #DRAW A SET OF LINES FROM GIVEN STARTING POINT.
[2]  # <MAT> IS A 2-COLUMN MATRIX OF X/Y CO-ORDINATES
[3]  #      OR A VECTOR WITH AN EVEN LENGTH.
[4]  *** Saved on 17/05/83 at 09.45.

[5]  →(2=ρρMAT)+Move
[6]  #RESHAPE TO TWO-COLUMN MATRIX.
[7]  MAT+((10.5×ρ,MAT),2)ρMAT
[8]  #GENERATE A MOVE TO FIRST SET OF CO-ORDINATES
[9]  Move:CTLs+528,MAT[1:]
[10] 'FAILED TO MOVE CURSOR' FSMAERR CTLs
[11] MAT+ 1 0 ↓MAT
[12] CTLs+530,(1+ρMAT),.MAT
[13] 'FAILED TO DRAW LINE' FSMAERR CTLs

```

v

CHAPIE MAT

```

[1]  #FORM DATA INTO MULTIPLE PIE CHARTS, EACH
[2]  #ROW OF MAT SPECIFIES THE SEGMENTS OF A SINGLE PIE.
[3]  *** Saved on 19/04/83 at 09.56.

[4]  MAT+(-2+ 1 1 ,ρMAT)ρMAT
[5]  CTLs+796,(ρMAT),,MAT
[6]  'FAILED TO DRAW PIE CHART(S)' FSMAERR CTLs

```

v

A further benefit is that PGF provides a style of graphics indistinguishable from end-user efforts (via ICU) or mainstream CICS-PL/1 through ICU or PGF. At Rowntrees we have on occasion taken users' prototypes and rebuilt them in APL extremely accurately (adding on the odd extra feature that couldn't be done with ICU!).

One final thought on implementation; as far as possible it pays to stick to the syntax and mnemonics used by FORTRAN, COBOL and PL/1 in devising the cover functions. I have found PGF to be an excellent package, and the documentation is first class. It saves a lot of time, effort and expense if you can simply hand someone the 'PGF Application Programmers Guide' (SC33-0148) and say "... for CALL CHSET please read CH.delta.SET"

R←BIZ2;XDATA;YDATA;PF

```
[1]  AAN APL COPY OF THE PL/1 DEMO PROGRAM ON PAGE 126 OF
[2]  ATHE PGF MANUAL.
[3]  A
[4]  ATTHIS DRAWS AN EXTREMELY BASIC GRAPH OF TWO SETS OF NUMBERS.
[5]  AWITH SOME SIMPLE ANNOTATION AND HEADINGS.
[6]  A
[7]  A** Saved on 19/03/85 at 11.28.

[8]  XDATA← 3 8 12 13 16 19 22 24
[9]  YDATA← 2 8 p 8 11 13 14 12 10.4 9 7.2 , 7 9 14.1 16 13.5 11 8 6.4
[10] MAT←XDATA,QYDATA

[11] AESTABLISH SCREEN FORMAT (DESIGNED BY 'DRAW') ....
[12] F$MAFMT 'FMTPLT32'
[13] ASET OUT CHART LAYOUT .....
[14] 'CHART' CHASET 'KBOX,NBOX,CBOX,CURVE'
[15] 'VMAR' CHASET 7 1
[16] 'HMAR' CHASET 9 6
[17] 0 0.5 CHAKEYPOS 'BC'

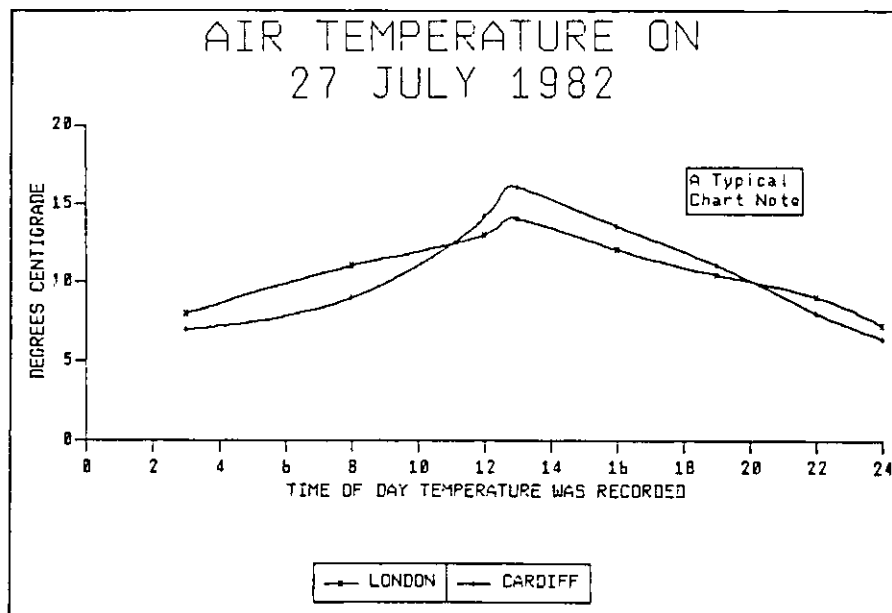
[18] ADEFINE ATTRIBUTES FOR HEADING, LABELS, .....
[19] 'HEAD' CHAATT 'RED.2.5'
[20] 'LABELS' CHAATT 'BLUE'
[21] 'TITLES' CHAATT 'YELLOW'
[22] 'BOX' CHAATT 'CYAN'

[23] ADO HEADING, KEYS AND AXIS TITLES .....
[24] CHAHEAD 'AIR TEMPERATURE ON;27 JULY 1982'
[25] CHAKEY 2 7 p 'LONDON CARDIFF'
[26] 'X' CHATTL 'TIME OF DAY TEMPERATURE WAS RECORDED'
[27] 'Y' CHATTL 'DEGREES CENTIGRADE'
[28] CHAPLOT MAT

[29] AFINALLY BANG OUT A NOTE USING X/Y CO-ORDINATES .....
[30] 18 14 CHANOTE '27/A Typical;Chart Note'
[31] CHATERM

[32] AWRITE ALPHA FIELDS, AND HOLD FOR USER RESPONSE ...
[33] F$MAFILL FMTPLT32
[34] PF←F$MAFFHOLD
[35] +(Copy,Exit)IF PF=1
[36] Copy:(VALIN 'HOW MANY COPIES ? :- ')GSACOPY 'LU40226'
[37] '*** HARD COPIES TAKEN ***'
[38] Exit:R←'READY'
      V
```

Fig. 2



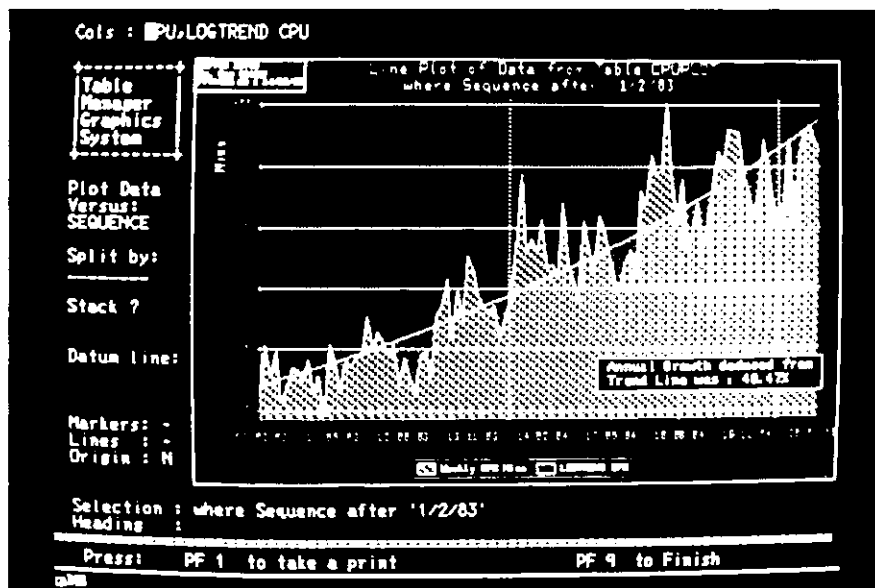
The Future

There remains one unexplained box on Figure 1, labelled 'PC/G'. So far IBM graphics has meant the 3279, green flashes and all. Because GDDM assembles the whole picture in the centre, then ships the finished product up to the VDU, it can take an eternity to transmit a complex chart to a remote screen. As a stop-gap the 3270-PC looks quite tempting, but suffers from being only a 24-line device. If you can make it work (we can't, so far) you can scroll the 32-line image locally, but the pictures remain hopelessly distorted. Alternatively you can re-draw all your screens to fit on 24 lines, and then build in some code to switch formats as required (messy!).

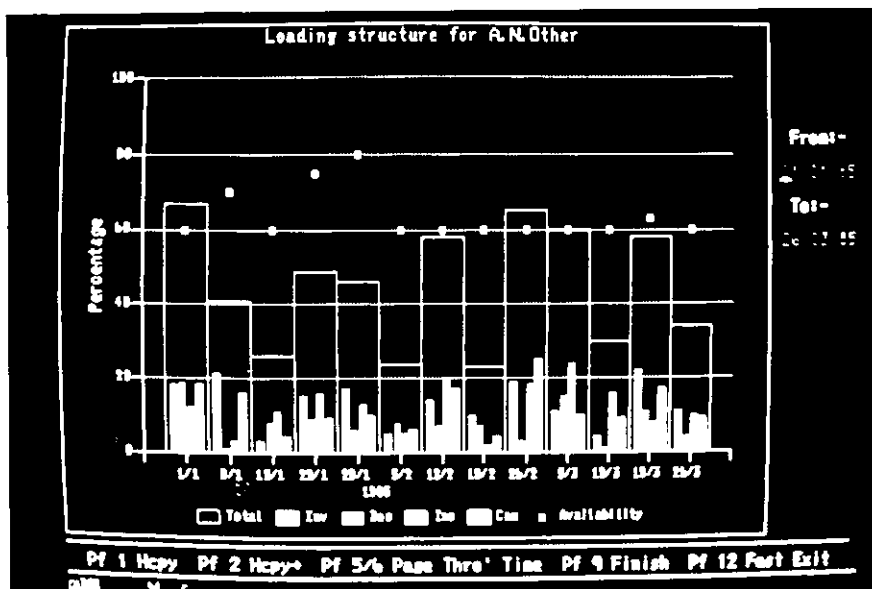
A much more promising option (though expensive) is the 3270-PC/G. For the first time this builds the picture locally; only a 'graphics definition file' is sent down the line. This looks remarkably like a series of GKS commands (you can simply pop out into DOS and have a look at it!), and is usually at least 10 times smaller than the actual picture it will build. The effect on both mainframe CPU time and remote response time is obvious. As an added bonus you can get decent quality output on real HP plotters (at last including acetates) which is an enormous improvement over dot-matrix printing.

Postscript

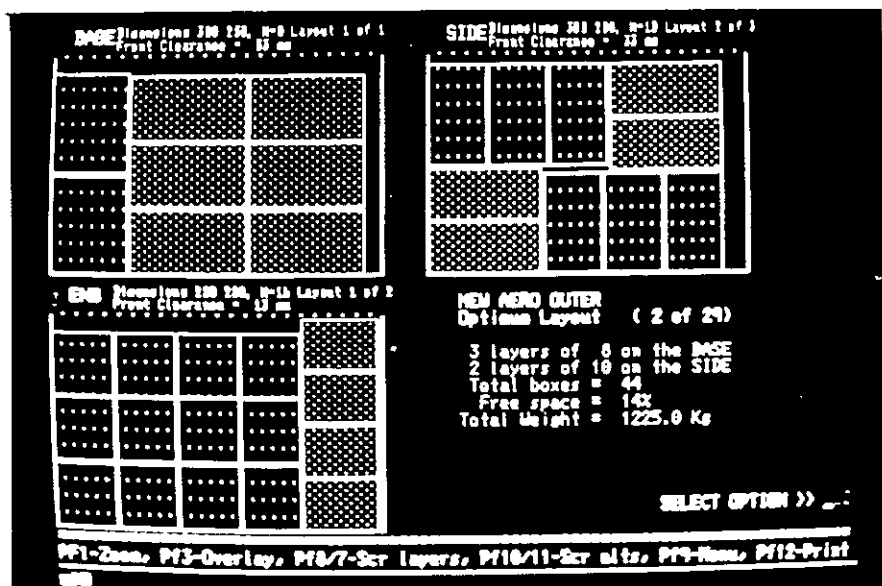
All the FSM functions (and the GS line-drawing routines) are easily rebuilt for APL*PLUS/PC using the built-in quad-functions. This fits in well with Richard Nabavi's notion of providing compatibility at the level of user-defined functions, rather than through APs. It is certainly much faster than either the AP205 on IBM's PC/APL, or even than MicroAPL's AP124 on the Sage. What chance a fast assembler-coded PGF look-alike for the PC/AT? It could answer a lot of prayers.



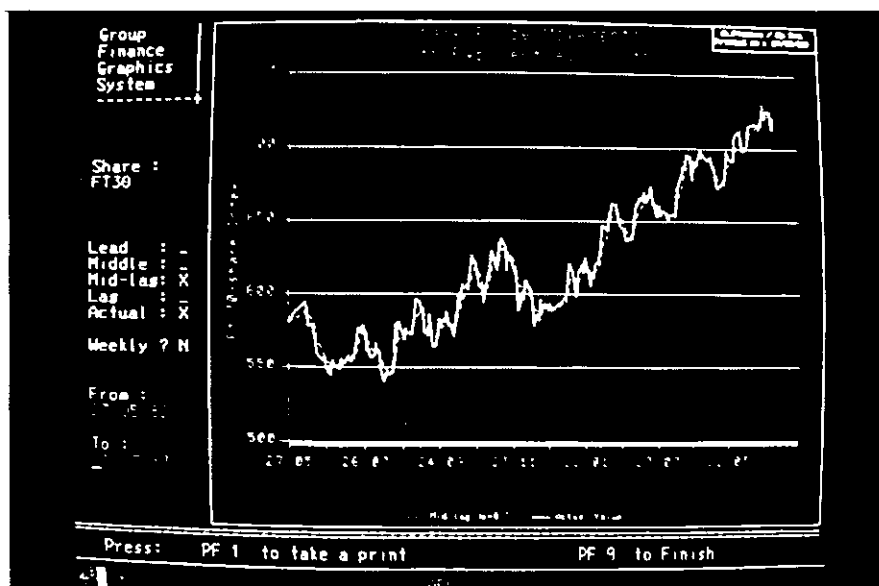
More complex use of graphics: PGF with several 'chart types' overlaid.



Simple line-drawing (at the GDDM level)



Note the mix of alpha fields and graphics.



GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL nor may be interested in learning it. However, we hope that you are curious about why, under the right conditions, such impressive results can emerge so quickly from APL programmers.

ANNOUNCING

dyalog APL

VERSION 3.0

The HIGH PERFORMANCE APL for UNIX* systems

Dyalog APL is FAST

The new version of Dyalog APL is at least twice as fast as before. The component file system and the performance of many primitive functions has been substantially enhanced, so that for some applications even greater improvement will be experienced. Our own benchmarks based on real applications indicate that there is now no faster APL available for supermicros.

Dyalog APL is FUNCTIONAL

Features include nested arrays, full-screen editor, full-screen data manager, component file system, error trapping, external variables, large dynamic workspace.

Dyalog APL is POWERFUL

Dyalog APL's interface to the UNIX* Operating System is unrivalled. Not only can you run any UNIX* program from within the workspace, but you can capture its results in an APL variable for subsequent processing. Furthermore, you can write functions in a compiled language such as C or Fortran.

Dyalog APL is RELIABLE and WELL ESTABLISHED

Dyalog APL is now implemented on over 20 different UNIX* systems and has already been installed in more than 50 companies in eight countries.

For further information contact

Sales Department
Dyadic Systems Limited
30 Camp Road Farnborough Hampshire GU14 6EW
Telephone Farnborough (0252) 547222 Telex 858811

*UNIX is a trademark of
A T & T Bell Laboratories

REINVENTING MAN: GOODBYE TO VON NEUMANN

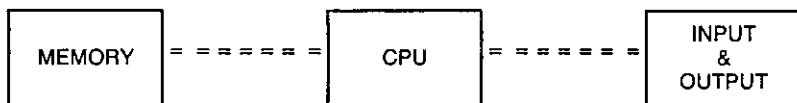
by Robert Bittlestone

The first half of my title is borrowed from a recent and very remarkable book (1), written by Igor Aleksander and Piers Burnett. The reference to John von Neumann depicts our continuing and almost total dependence for almost all current computer architectures on the model which he laid down nearly 40 years ago. Ever since then we have been waiting for a radical departure from this architecture. It is now evident that Igor Aleksander has made this breakthrough, although curiously this fact is almost buried in his new book.

Such a claim demands some justification and for this I need your indulgence for a few paragraphs of background, without which the importance of Aleksander's work cannot be conveyed. My apologies to those who are familiar with the following outline.

Why are computers made the way that they are? How are they made anyway? For those of you who have never opened the bonnet, this is the basic structure of almost all current machines:

Figure 1: Clocked Sequential Digital Computer Architecture



Of course there are exceptions, such as direct memory access by high data rate peripherals, but these don't tend to change the fundamental truth, which is that at any one instant, only one thing is happening.

Admittedly the instants are very short. The most recent 32-bit microprocessor designs are expected to clock at 20 Mhz, ie. twenty million times per second. This implies one clock cycle every 50 nanoseconds, and a nanosecond is a thousand-millionth of a second. It sounds impressive, and it is. That's the kind of approach which works well for WORDSTAR, for example, as I type at this screen. But WORDSTAR doesn't understand anything at all about what I'm writing, and despite spelling checkers and style correctors, neither does any current assemblage of CRAY supercomputers. Nor, irrespective of the current volume of work in expert systems development, is the prospect even remotely on the horizon. Why not?

Computers are made the way they are at present primarily as a result of John von Neumann taking up and elaborating the concept of a digital clocked architecture, which was first proposed by Turing in his famous exploration of Godel's incompleteness theorem. Godel's theorem states, roughly, that every formal system must contain some proposition that cannot be determined within that system: in other words, a formal system cannot be closed with respect to its set of axioms and consequent proofs. Turing chose to investigate Godel's theorem with the aid of a logical machine: an infinite strip of paper tape passing bidirectionally under a tape reader. But here was the crucial feature: the direction of the tape was determined by the pattern of the data itself. This reflects a tangled hierarchy in which objects at one level seem to reach out and interact with objects at a higher, or meta, level; objects which we do not expect them to be able to access.

At the time this was an absolutely extraordinary notion: that the same set of binary numbers could represent instructions as well as data. John von Neumann was instrumental in converting this idea into a viable specification, although those who knew Turing himself believed that had he lived longer he would have been the first to carry the concept through to reality. Unfortunately Turing committed suicide instead, almost certainly as a reaction to persecution because of his homosexuality. I suppose it is a measure of social progress that this particular fate is now becoming improbable, but in case any readers are contemplating the option sympathetically let me cheer you up instead with an anecdote about von Neumann, the leading mathematician of that era.

A female journalist at a cocktail party asks von Neumann the well-known brainteaser about the two cars and the fly. A car leaves London for Brighton travelling at a constant 80 mph. Simultaneously a car leaves Brighton for London at a steady 40 mph, and a fly, travelling at 90 mph (killer flies were a real problem in those days) leaves in the same direction. The fly encounters the Brighton-bound car somewhere on the road and immediately reverses its flight. On meeting the London-bound car again it reverses once more, and so on in an ever decreasing zig-zag until the unfortunate insect is finally crushed between the radiators of the two equally unfortunate vehicles.

"So, Dr. von Neumann, the question is, supposing the distance from London to Brighton to be 60 miles: how far does the fly fly?"

"45 miles".

"You're right of course, but how unusual. Most people quickly see that the cars collide after half-an-hour, and that therefore the fly must fly for the same time irrespective of his route. But mathematicians usually miss this short cut and use the much harder techniques of integral calculus to arrive at the same result."

"But that was precisely my approach".

Evidently there were no flies on our John. But his energy and emphasis were crucial; they defined the direction of computing from then on. Despite the work of cybernetician of the time such as Norbert Weiner, Ross Ashby and Warren McCulloch, and the subsequent explorations of their present-day counterparts such as Gordon Pask and Stafford Beer; despite even the convincing findings from neurophysiology, the issue of computer architecture was to all intents settled. The new machines worked well, and they worked even better when the invention of the transistor and subsequently the integrated circuit allowed the clumsy ferrite cores of computer memory to be replaced by bistable logic circuits. The momentum was unstoppable.

The most fervent adherents of digital architectures are now busy working on parallel processors, transputers, data flow logic and other layouts in an effort to accelerate processing yet further. So WORDSTAR will end up right-justifying a whole book in a microsecond. So what? This is not at all how our brains work. Can a machine recognise a human face? Can it penetrate a disguise? Can it detect emotion in an expression? Astoundingly, since 1981 the answer to all these questions has been yes, but the machine concerned is not a computer and it probably never will be. It is a neural net designed specifically for pattern recognition, but its components are memory chips not neurons. It is called WISARD and it is the architecture we have been waiting for. It is going to make the computer obsolete.

"Reinventing Man" is subtitled "The Robot becomes Reality" and much of the book is taken up in describing approaches to robotics, including limb movement and artificial vision. It is a fascinating mixture of down-to-earth engineering detail and an almost apologetic bouleversement

of the entire computing establishment. The core is contained in Chapter 10, "The Silicon Neuron", which I shall try to summarise.

Suppose we want a robot to distinguish between shapes on a production line. The conveyor belt contains two kinds of object: car wheels and car doors. How can we train the robot to distinguish between the two? Ignoring the question of scale and orientation, the shapes appear thus:

Figure 2: Images of a Wheel and a Door

XXX	XXXX
XXXXX	X X
XXXXXXXX	X X
XXX XXX	XXXXXXXX
XXXXXXXX	XXXXXXXX
XXXXX	XXXXXXXX
XXX	XXXXXXXX

This assumes 8x8 digitisation in a camera eye; practical systems are of course far larger. Now, how is the robot going to decide which is which? Suppose we sit down to write a computer program to do the job. We shall probably start scanning the image from left to right and from top to bottom, although we are not of course obliged to. In APL we may treat the pixels in most unusual ways, although let's remember that this is a convenience only: the processor which does the work is still clocking sequentially through single instructions at a time. It's not entirely obvious how best the program should be written, except that one may make the observation that provided the shapes are always identical and each element is either on or off, the problem reduces to the trivial one of matching against a pre-specified 64-bit binary word.

But the shapes will not remain. They will move around in the field of view: they will compress and expand as the camera zooms: they will be obscured as von Neumann's fly crawls over the lens; they will, in short, exhibit the fabric of reality. Since the two shapes are topologically identical (a surface with a single hole in it), how shall the program draw the line? What is the essential closure of a door: what is the rotundity of a wheel? Suddenly we are searching for Platonic gestalts and we really aren't about to find them in our algorithms.

And if we succeed? We have written a program, at great cost of time and effort, for one paltry distinction. We have to start all over again for separating apples and pears.

Now consider the amount of processing of this nature required by an activity that human beings perform almost effortlessly: driving a car. I can only guess at the data rate consistent with survival; let us suppose that in critical manoeuvres such as overtaking, an update of all information at least twenty-five times a second is required. This should be about right since it corresponds to the rate of persistence of vision - i.e. the cinema effect at which individual frames become continuous. Let us also suppose that only a few thousand shapes need to be discriminated each instant.

I believe I hear some of you protest at this mention of those thousands. Surely it's just the road, the other car, a traffic light or two? But wait a minute: what's so special about the traffic light that isn't contained in one of the leaves of the tree you're passing? In that split second in which motion has been frozen on the robot's digitised eye, who is going to distinguish the boundary

of the car from the road? If we perform a first differential on the pixels of the scene - i.e. detect all contours at which a string of 0's or 1's in any angular direction changes, then we shall certainly find thousands of such boundaries on even a minimally resolute picture. As humans we discard the unimportant ones - and that's the problem in a nutshell.

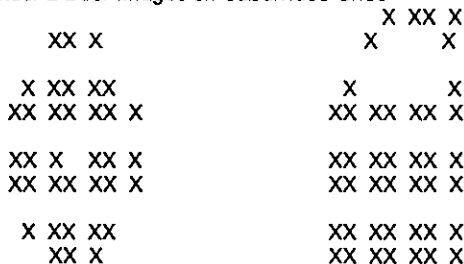
Supposing our computer program is surveying a three-dimensional scene of resolution 1024 points on each axis, which is typical of current medium-to-high resolution computer graphic applications, although very coarse-grained indeed by human optical standards. Supposing also that 256 levels of colour are distinguishable per picture element (pixel). The eye can actually detect far more subtle nuances of colour, but we'll let this pass. These 256 colour levels may be represented by the permutations of 8 bits, so in computer terms the scene therefore contains $1024 \times 1024 \times 1024 \times 8$ bits, which is conveniently 8 Gigabits or 1 Gigabyte, a gigabyte being approximately one thousand million 8-bit characters.

All this is of course changing 25 times a second, which means a continuous data rate of 25 Gigabytes per second. Current computers cannot even read data in at speeds much above one-thousandth of this rate, and even that is for short bursts only rather than over a sustained period. Computers are of course getting faster, but even if this data rate could be matched they would be building up storage at a rate of 90,000 Gigabytes per hour, so they would soon run out of places to put it. After such considerations, to imagine that they could possibly run algorithms in real time to distinguish several thousand elemental objects is simply ludicrous. And don't you also talk to your passengers while you're watching the road?

The problem is not just with the computers. It is evident that the whole way of thinking about this problem in conventional digital terms is completely unhelpful. Nobody really believes that the brain is handling data like this. Digitising everything into a single serial data path and applying a single program to it is an absurdity, a classic example of the perils of reductionist thinking. We have to be sufficiently bold and brave to abandon the obsession for writing computer programs for everything. We have to be prepared to go back to school and to learn about the properties of neural nets and automata theory. We must expect that it will take some time before the benefits make the effort worth while. Meanwhile, listen to Igor Aleksander as you would have listened to von Neumann thirty years ago, at his 1956 lectures in which he squashed the options by pointing out the unarguable implications of the hardware economics of the time. If you only have valves, you had better forget about neural nets: your circuits will be too expensive.

But the pursuit of digital architectures has given us high-density RAM chips, and it looks as if this will be judged by our descendants as the serendipitous sensation of the century. Aleksander uses RAM chips, but in an utterly different way from their deployment in your friendly microcomputer. Now that we have the silicon neuron, let's look at the kind of architecture that von Neumann might have pointed to had this been the hardware of the 1950's.

Figure 3: Wheel & Door Images on Subdivided Grids



Here are the same two shapes - the wheel and the door - but each is now broken up into 16 4x4 grids. If a grid sees the pattern which it has been taught, it will fire: if not, it will fail to fire. For convenience let's assume equal weighting per grid, so that if all 16 grids fire we score 16, and if none fires we score 0.

If we show the door discriminator a view of the wheel, a total of six grids will fire. If we show the wheel discriminator a view of the door, the answer is of course the same: an overlap is symmetric. So the wheel discriminator says that a door is six-sixteenths or 37.5% like a wheel, and vice-versa. If the same image is offered to both discriminators simultaneously and another network (not so far described) interprets the result, we are getting somewhere interesting. We have a real-time pattern analysis system that uses simultaneous majority voting among parallel elements to detect shapes that it has been taught. We also have particular chunks of neural circuitry that recognise particular objects.

What about a shape that is genuinely ambiguous?

Figure 4: A wheel, a door and a ???

	XXXX	
XXX	X X	XXX
XXXXX	X X	XX XX
XXXXXXXX	XXXXXXXX	XX XX
XXX XXX	XXXXXXXX	XX XX
XXXXXXXX	XXXXXXXX	XXXXXXXX
XXXXX	XXXXXXXX	XXXXX
XXX	XXXXXX	XXXXX

Before you read on, try deciding for yourself which the image might be. There's the angular slope of the door, but the cut-out at the bottom resembles that of the wheel. Well, how would Aleksander's discriminators behave? If you count the 4x4 sub-grids you find in fact that they both score 4 out of 16, ie. 25% recognition. Interestingly I drew the picture first: the coincidence of an identical score was unplanned, although I tried to maximise the ambiguity in the drawing.

There is more, far more of course, in Aleksander's book (I must not forget to acknowledge Piers Burnett's achievement as a freelance technical writer in helping to bring this work to the general public). The notions of synaptic weighting and of multiple levels of neural nets cause one to realise that this simple example is just the tip of the iceberg. For those who wonder how it all started, there is a fascinating account about the progression from small beginnings at the University of Kent - the SLAMs (stored logic adaptive modules) one read about many years ago, to the present day sophistication of WISARD. Some of you may have seen WISARD briefly in action on a Horizon television programme last year, recognising Aleksander's face despite the grimaces.

Please resist the temptation to reflect on how you could program all this in FORTRAN, PASCAL or even APL. No doubt you could. Take time out to read the book instead. It will change your life, as it will eventually change all of ours.

A final observation. In their reflection at the end of the book in the chapter entitled "The Millenial Machine", the authors make use of personal construct theory and in particular Kelly's work in this area (2). However attractive the theory, this seems a major betrayal of the cybernetic profundity of the rest of their opus. It seems to me that there is a wonderful opportunity here to marry their work on pattern recognition with the extraordinary advances made by Powers on the nature of perception and the multi-level character of human thought (3).

Like Aleksander, Powers takes us through designs that are empirically testable in his search for a model of the human process. Where Powers is weak - on the detailed circuitry of each stage - Aleksander is outstandingly strong. By contrast, Powers shines in the area of the grand design, the multiple levels of hierarchy required to integrate the perception of Cartesian space and to progress from low-level perception and neuromotor control through the ability to reason right up to the instinct for survival. This is a region in which Aleksander has not yet trodden, or if he has he is keeping the results to himself. But Powers' own model of speech recognition fits Aleksander's nets so beautifully that one almost suspects collusion. I have referred to Powers' work in more detail elsewhere (4) and it would be magnificent if the marriage of these two relatively little-known innovators' extraordinary accomplishments provided the ultimate synthesis.

References

- (1) Reinventing Man: The Robot becomes Reality, by Igor Aleksander and Piers Burnett. Kogan Page 1983.
- (2) The Psychology of Personal Constructs, by G.A.Kelly. Norton, New York, 1955.
- (3) Behaviour: The Control of Perception, by W.T.Powers. Wildwood House, 1973.
- (4) XPL: A Framework for Expert Systems In APL, by R. Bittlestone. VECTOR Vol. 1 No. 2, pp.65-72. Oct. 1984 (BCS).

QL/APL - A NEW APPROACH TO POPULARIZING APL

by David Eastwood

Background

APL has been available as a working computer programming language for nearly two decades. During that time, it has attracted a very enthusiastic band of users, and, to those who know APL, the advantages of the language over conventional programming languages are evident and very considerable. These advantages include:

- Faster development time;
- Elegance and self-consistency;
- Modularity;
- Excellent environment (the workspace) for building up a system in manageable parts;
- Unsurpassed debugging facilities;
- Freedom from annoying restrictions imposed by computer hardware;
- Good speed of execution compared with other interpreted languages;
- Fairly good portability of code.

The list is not exhaustive, but the overall message is clear: many APL users, including the author, regard APL as so superior to languages such as BASIC that they would never even consider using other languages if they could possibly help it.

Many commentators have considered this issue in the past, and it is indeed a vital question: If APL is so wonderful, why is it that APL accounts for such a small proportion (probably well under 1%) of computer language use? And in particular, why is it that a language of such obvious shortcomings as BASIC should be orders of magnitude more popular than APL?

The factors impeding the spread of APL

A full discussion of the factors which have led to the (in world terms) unpopularity of APL is beyond the scope of this paper (see John Myrna's paper 'Increasing the Use of APL' in the APL83 proceedings). There are a number of explanations, of which two seem to be paramount:

- APL has tended to be available only on machines which are more expensive than comparable systems running BASIC;
- The APL character set is a disadvantage because it is at first sight off-putting to potential users, and because it involves use of non-standard peripherals.

This paper describes work which addresses both of these points. An APL interpreter has been implemented on a very low-cost, mass market machine, the Sinclair QL, in a version which uses a keyword syntax which is directly aimed at converting users from BASIC to APL. (For existing APLers, a version of the interpreter using APL symbols is also available for the QL).

Keyword versions of APL

In the past, there have been a number of proposals for versions of APL which did not require the use of the special APL character set. Of these, the STSC keyword system for APL *PLUS/PC is probably now the best known, and our initial idea was to retain compatibility with this

system in our own interpreter. However, previous keyword versions of APL have, in the view of the author, been started from the wrong premise. The reasoning, if for example we take the STSC product as an example, seems to have been as follows:

- We have produced an APL interpreter for the IBM PC, which uses normal APL characters via a special character generator PROM.
- However, not all machines which might use the interpreter will have the special PROM fitted.
- Therefore we will provide a keyword option which will allow APL programming on standard machines.

The result of this way of looking at the problem is a keyword scheme which is fine for users who already know APL, but is not helpful in popularizing APL for people who have never seen an APL keyboard. There are a number of problems with the STSC approach:

- A single APL symbol can be expressed in a number of different ways. For example, the rho symbol can be entered as $\#$ shape (Monadic form), $\#$ reshape (dyadic form), or $\#$ rho (ambivalent form). $\#$ is used as a delimiter to distinguish keywords from user-defined names, but this can be switched off. Thus, in the STSC keyword system, the APL symbol rho can appear in six different forms. This we felt to be over-complicated for our purposes.
- As will be seen from the above example, the STSC choice of keywords is not always meaningful to users not familiar with an APL keyboard. Further examples are $\#$ theta and $\#$ domino.
- In common with other proposed keyword systems, the STSC approach is to use ASCII symbols if possible to correspond with APL symbols. For example, $\sim$ is used for high minus, giving problems when negative numbers are displayed:

5-11

~6

This approach also gives rise to a scheme which is not regular in its approach to which functions have keyword forms. If we are to have a keyword system, then we should use keywords for all primitive functions, except for the most commonly used (plus, times, multiply, divide). It makes little sense to have the rarely-used random-number generator as a single symbol ρ , if multiply is to be a keyword.

The STSC keywords are, we felt, often too long. The encode symbol, for example, becomes 'represent'.

A different approach

Having studied various schemes for keyword APL interpreters, we eventually (and reluctantly) decided that, if a keyword APL was to be a tool to popularize the language, then a completely fresh approach was mandatory. The following criteria were established:

1. The keyword system should be viable in its own right as a programming language, on the assumption that the user will never see an APL keyboard.
2. It should as far as possible build on users' experiences with other languages, particularly BASIC.
3. It should be as far as possible consistent with APL, and certainly should not violate APL's unique elegance and internal regularity.

4. It should use the standard ASCII character set, but not make use of those ASCII symbols which differ in other national languages. For example, the ASCII square brackets [and] become special letters in Swedish or Danish, and it would make the language almost unusable for those countries if we employed such symbols.
5. It should be as concise as possible without compromising the readability of lines of code.
6. As far as possible, applications written in standard APL should be transportable to the keyword version.
7. The number of keywords should be limited.
8. To ease entering of functions, input should be case-insensitive, although we have chosen that on output keywords and user-defined names should be distinguished.
9. For practical reasons, we wanted to make the specification simple to implement starting from an existing APL interpreter.

The final result

It is not possible to find a perfect solution to the problem of making a keyword version of APL, but after considerable work and experimentation, we believe we have come up with a version which is much better than merely acceptable; indeed, APL programmers with very considerable experience have found it very easy to take to. It is difficult to appreciate the flavour of the new version of APL without actually trying it at some length, but the following details should make clear the approach we have taken.

Case handling

The handling of upper/lower case is as follows:

- Comments and text strings can of course be entered using any valid ASCII characters, and are left unchanged.
- Lines of APL can be entered in either case, and case is ignored on syntax analysis (including in checking for keywords and Quad names).
- Keywords are displayed in LOWER CASE.
- Quad names are displayed in UPPER CASE, with # replacing Quad.
- All user-defined functions and variables are displayed in UPPER CASE.

This method of handling case implies that there are reserved words in this language. Alternatives, such as preceding keywords by a delimiter or making a distinction between upper and lower case input, were felt to be ergonomically unacceptable. However, there are only 47 reserved names in the specification, most of which are unlikely names for user-defined functions or variables, and in practice this has not caused any difficulties. By contrast, Microsoft's BASIC for the IBM-PC has no fewer than 159 reserved words.

The distinction between standard (upper-case) letters and underlined (here lower-case) letters for APL names has been dropped. This partly because it was felt to be extremely confusing and tedious to have to use the shift key when entering lines of code, and partly because it would not ease readability if, for example, XX, xx and Xx were distinct APL variables.

Choice of keywords

The choice of particular keywords is of course largely a matter of taste, and there is no ideal solution. The following criteria were used in selecting keywords:

- They should be short (preferably less than four letters).
- They should be English words, rather than mnemonics or abbreviations.
- They should ease readability of code.

Some compromise was necessary, and the final choice is shown in the table.

One additional primitive was added to make the language easier for novice users. This is the IF operation, which is the same as the standard APL compress function, but with the arguments reversed. This makes it possible to enter function lines such as:

```
goto LOOP if COUNT eq MAXIMUM
```

Choice of symbol equivalence

The ASCII character set is badly thought out as regards availability of arithmetic symbols. The absence of multiply and divide signs is particularly unfortunate. However, the ASCII character set is one thing which cannot be changed, and so we have to adapt to it. Clearly, use of keywords for multiply and divide would be unacceptable because of the frequency with which they are used, and the fact that users will expect symbols for these functions. We therefore applied our overriding criterion - the convenience of users upgrading from BASIC - and settled on * and / for multiply and divide. The same criterion was used in selecting the symbol for comment.

The choice of symbols for the APL square brackets [and] was a tricky one. Although these symbols do exist in the ASCII character set, they vary in the standard ISO national character sets, and so they were ruled out. The same is true of { and } . We therefore adopted angle brackets < and >, which are available and do not vary. Although it might appear at first sight that these symbols should be reserved for their APL use (greater than and less than), we wanted regularity in our implementation. Since there are no ASCII characters for 'greater than or equal', 'less than or equal', and 'not equal', we had to choose keywords for these, and we felt that we should therefore choose keywords for all the comparison functions. Thus, we selected GE, GT, EQ, LT, LE and NE for these functions (to be consistent with Fortran if not with BASIC), leaving angle brackets free for indexing. This also left free the = sign, which we considered using for assignment. However, in this case we could not swallow our natural distaste for this choice, and so the = sign was left unused, with the keyword 'is' selected for assignment.

The choice of a symbol for Quad was also tricky. Although we could have chosen different equivalences for Quad on its own, and Quad as part of a system name, we were not keen to do this. The use of the ⍷ is fairly natural as an equivalent to Quad, and we eventually made this choice. This was the only case where we relaxed our requirement for invariance of the symbols used across the different ISO national character sets, for ⍷ sometimes prints as a U.K. pound symbol on U.K. and French terminals.

A full list of the valid symbols in keyword APL is shown in the table at the end of this paper.

Minus sign handling

When this project was begun, we feared that the problem of the minus sign would be almost intractable. We certainly wanted to use the ASCII minus sign for subtraction, but it was also essential in our view that negative numbers should be entered and displayed using the same symbol. To have reports which produced negative numbers using any other symbol would rule out the use of the language for serious applications. Thus we were faced with the problem of putting back the confusion which APL had neatly avoided in distinguishing subtraction from negative numbers.

In practice, the problem turned out to be easily and very naturally solvable. We simply applied the rule that the minus sign immediately preceding a digit would be interpreted as the APL high minus, provided it was immediately preceded itself by a space. All other instances of the minus sign are treated as subtraction. This has proved to be completely natural in use, and unambiguous, and we are completely happy in this choice. The following examples should indicate how it works in practice:

```

6-4
2
  X is 6 -4 5 -3
  X
6 -4 5 -3
  6- -4
10
  X - 4
2 -8 1 -7
```

Implementation

The keyword system described here has been implemented in versions of APL.68000 for Mirage-based systems, and for the very low-cost Sinclair QL. The implementation was done by modifying the syntax analysis portion of APL.68000 to look for keywords, as well as making a number of other minor changes in the interpreter and supervisor to deal with certain special cases. Keyword checking is done for the execute primitive, and Quad-FX, as well as for user-entered lines of code.

The internal representation of APL functions written using this method is identical to that of the conventional APL version. This means that a workspace, saved using standard APL.68000, can be loaded and run using the keyword version. There are cases where this will not work, but these cases are relatively rare, and the changes necessary to allow interchangeability between the two versions of APL are not very significant. The interpreter uses the same rules for printing out lines of code with keywords as it does for standard APL names, so that at the ends of lines, keywords will not be split into two.

User reaction

Extensive work has been done in giving users the opportunity to comment on the keyword implementation. We have canvassed both experienced APL programmers, and people who have never used APL. The reactions have been almost unanimously very favourable indeed. We have been particularly pleased by the response of the experienced APLers, who might have been expected to have been negative about the various compromises which had to be made, but the following excerpt from one of our beta-test reports indicates the sort of response we received (this was written by a user who has around 7 years' practical experience in the commercial use of APL, mainly on IBM mainframes):



H.M.W. PROGRAMMING CONSULTANTS LTD

Why not discover more about

- ★ **Consultancy/Support Service**
- ★ **APL on the IBM PC**
- ★ **VM/CMS Packages**

Ring Ken Jackson at

H.M.W. PROGRAMMING CONSULTANTS LTD

142 FELTHAM HILL ROAD,
ASHFORD, MIDDLESEX TW15 1HN
Telephone: Ashford 41232

"For comparison, I selected an existing sales statistics analysis system written in VSAPL. Translation was troublefree, being simply a matter of replacing symbols... In use, there was no discernible difference between the mainframe and QL versions... As for the general flavour of keyword QL/APL, I liked it immensely - much to my amazement... Since starting to use keyword QL/APL, I have become convinced that the conventional APL character set was just an elitist trick to restrict the spread of an immensely powerful language."

The Future

We believe that keyword QL/APL represents a major breakthrough in the effort to bring APL out of obscurity. As far as we know, this is the first time that a keyword version of APL suitable for people unfamiliar with the language has been released.

As far as the particular implementation of the language on the QL is concerned, here again we believe that a significant breakthrough has been made. This is due not only to the price/performance ratio - QL/APL costs under 100 pounds including VAT, making it possible to purchase a working APL system for under 500 pounds - but also to the excellent facilities which we have been able to include in the interpreter. Foremost amongst these are very good built-in colour graphics, and support for multiple windows on the screen.

Clearly, the important thing is to get large numbers of people familiar with the language, and we certainly hope that this version of keyword APL will become a de facto standard, simply by virtue of the large anticipated volume of sales in Europe and the US. It will then be up to the APL community as a whole to build on that base.

Appendix 1: The symbols in traditional APL and the QL/APL keyword equivalents

FHOLD	⊞	ON	/
FDROP	⊟	OVER	\
FGET	⊠	PICFMT	α
FPUT	⊡	DECODE	ι
NOR	∨	MIN	⌊
NAND	∧	IN	ε
DOWN	⋄	DEFN	∇
UP	⋆	INDEX	ι
ROTATE	⊛	OUTER	∘
TRANS	⊙	ABS	
LOG	⊙	ENCODE	τ
LOCK	⊞	TRIG	∘
EXEC	⊞	EXP	*
FMT	⊞	RAND	?
PROMPT	⊞	SIZE	ρ
FACT	!	MAX	⌈
MATDIV	⊞	NOT	~
LT	<	DROP	†
LE	≤	IF	ω
EQ	=	TAKE	†
GT	>	IS	+
OR	∨	GOTO	→
AND	∧	GE	≥
NE	≠		

Appendix 2: Example function written in keyword APL

To show how keyword APL looks in a real example, we have translated Professor Hawkes' Numerical Integration function from page 122 of Vector, Vol 1. No. 2. The function is also reproduced below. It is interesting to compare the two listings side by side. The style (for example in branching method and the use of underlined letters and delta) has been changed very slightly to illustrate stylistic features of keyword QL/APL, although this is not necessary.

```

defn R is Y NUMINT X;N;RANGE;RT;DX;GRID;FO;FE;M;S
<1>  ! Numerical Integration. Y is a string of form 'A FNAME'
<2>  ! The global INTERR specifies accuracy
<3>  ! Result is integral over N subintervals of X<1>, X<2>
<4>  XCHECK: goto XOPTIONS if (1 ge size size X) and 3 ge size ,X
<5>  XMESS: 'X should be vector of shape 3. X<3> is number of intervals'
<6>    'into which (X<1>, X<2>) divided.'
<7>    'Default X<3> is 1 if not specified.'
<8>    'If only scalar X given then it is taken as X = 0,X,1' & goto 0
<9>  XOPTIONS: goto TWO if 1 le size ,X
<10>    X is 0,X
<11>  TWO: goto XCHECK2 if 3 eq size X
<12>    X is X,1
<13>  XCHECK2: goto XMESS if X<3> ne 1 max min X<3>
<14>    M is 1
<15>    R is (N is X<3>) size 0
<16>    RANGE is X<2>-X<1>
<17>  LOOP: RT is R
<18>    M is M*2
<19>    DX is RANGE/S is M*N
<20>    GRID is X<1> + DX*0,index S
<21>    FO is exec Y, ' GRID'
<22>    GRID is X<1> + DX*(index S)-0.5
<23>    FE is exec Y, ' GRID'
<24>    R is (DX/6)* + on(N,M)size(4*FE) + (1 drop FO) + -1 drop FO
<25>    goto LOOP if INTERR lt max on abs R-RT
defn

```

```

V R←Y NUMINT X;N;RANGE;RT;DX;GRID;FO;FE;M;S
[1] A NUMERICAL INTEGRATION. <Y> IS A STRING OF FORM 'A FNNAME'
[2] A THE GLOBAL <ΔINTERR> SPECIFIES ACCURACY
[3] A RESULT IS INTEGRAL OVER <N> SUBINTERVALS OF X[1],X[2]
[4] xCHECK:→((1>ρX)^(3>ρ,X))/xOPTIONS
[5] xMESS:'X SHOULD BE VECTOR OF SHAPE 3. X[3] IS NUMBER OF '
[6] 'INTERVALS INTO WHICH (X[1],X[2]) DIVIDED'
[7] 'DEFAULT X[3] IS 1 IF NOT SPECIFIED'
[8] →0,0ρ←'IF ONLY SCALAR X GIVEN THEN IT IS TAKEN AS X←0,X,1'
[9] xOPTIONS:→(1<ρ,X)/tWO
[10] X←0,X
[11] tWO:→(3=ρX)/xCHECK2
[12] X←X,1
[13] xCHECK2:→(X[3]≠1/[X[3]])/xMESS
[14] M←1
[15] R←(N←X[3])ρ0
[16] RANGE←X[2]-X[1]
[17] LOOP:RT←R
[18] M←M×2
[19] DX←RANGE÷S←M×N
[20] GRID←X[1]+DX×0,1S
[21] FO←±Y,' GRID'
[22] GRID←X[1]+DX×(1S)-0.5
[23] FE←±Y,' GRID'
[24] R←(DX÷6)×÷/(N,M)ρ(4×FE)+(1+FO)+~1+FO
[25] →(ΔINTERR<f/|R-RT)/LOOP
V

```

Appendix 3: Valid symbols in keyword APL

Apart from alphabetic and digits, the only ASCII symbols which have a meaning in keyword APL are as follows:

ASCII	APL equivalent
!	⋈
)	)
>	]
/	÷
*	×
+	+
.	.
(	[
<	[
;	;
*	×
:	:
Hash	⊞
-	- or - according to context
&	⋈
-	Δ (NB Valid in variable names - eg JUNE_DATA)

Editor: A review of QL/APL appears in the Technical Section of this issue of VECTOR.

THE INFORMATION CENTRE AS A STRATEGY FOR THE FUTURE

by Fred J. Perkins

The challenge facing the DP Department today is that of meeting demands for flexible and efficient access to, and control of, the information resources at the heart of any successful company's operation. The Information Centre potentially does meet the challenge. There are however many pitfalls, and as the issues are very much at the centre of an organisation's total strategy for computing, communications, and office automation, it is important to understand the history of the development of Information Centre concepts. There are perhaps more accounts of failure than of success, but this should not deter an organisation from seeking to follow what is fundamentally a very sound strategy.

The Information Centre concept came about in the early 1970's. Traditionally, DP departments had been predominantly batch or transaction processing in their orientation. Unhappily, interactive users were generally unwelcome on such systems, as their demands were spasmodic, unpredictable, and all too often liable to cause system crashes. Controllable, regular jobs run as card decks were much more acceptable to the DP manager with his scant resources.

The gulf that arose between the DP department and the end users is one familiar to most large organisations. It benefited only the external service companies, who were able to charge premium prices for access to systems which end users felt were much more responsive to their real needs.

However, as computer hardware costs came down, and minicomputers (and later, microcomputers) came almost within the budget capabilities of user departments, serious problems of anarchy arose, as users set about solving their problems entirely outside the influence of the DP department. At the same time, the growth of such systems, isolated by their incompatibilities, began to cause real difficulties in the efficient control and integration of the information resources of the organisation.

The concept of the Information Centre

The Information Centre was a concept, originally developed by IBM, which attempted to bring order to the impending chaos. Its fundamental aim was to shift the DP focus back on to the end user, but under the control and direction of the Data Processing department. This aim would be achieved by providing tools and basic support from the DP group, together with sufficient education to allow the user to solve his own problems, largely in a "Do It Yourself" mode.

The Information Centre was a management philosophy, implemented in software. It offered a small number of "strategic" software packages and building blocks aimed at end users, typically in the M.I.S. department. Basic database mechanisms were provided to allow the user access to corporate data.

Hence, the increasing levels of end user expertise in the use of computer systems was channelled into a relatively small number of basic application systems, and the DP department role moved from one of total responsibility for the development of systems to one of encouraging user self sufficiency within a controllable environment. Backlogs in system implementation timescales would largely disappear, and in any event they were now on the shoulders of the user himself.

The concept was excellent. It was welcomed from all sides of the debate, and quite a few organisations established Information Centres which even today still serve their companies well. A truce was established between the users and the DP department.

Defects in the original concept

Computing however is probably one of the most rapidly changing environments of the last decade, and the flaws in the Information Centres soon became apparent.

It has been said of the Information Centre that "never in the field of human endeavour have so many waited so expensively, for so little." This may indeed be true. It would be naive to believe that a major new concept propounded by a hardware manufacturer was done out of purely philanthropic motives. The underlying motive was undoubtedly to promote the sale of large computer systems, and associated software packages that would perhaps encourage rapid consumption of hardware resources.

Perhaps the single most obvious flaw in the original offerings was the lack of integration between the elements that made up the Information Centre. Indeed, the most cohesive product on offer was probably the marketing brochure itself!

Users were quick to discover the "missing links" of I.C. implementation. These fell into 3 main areas:

- * End user terminal hardware
- * Communications
- * Implementation software

Hardware:

Professional DP users were quite happy to utilise bulky complex terminals, generally from a single supplier, and heavily oriented towards use by computer programmers. End users however, were demanding flexibility and portability of user terminals, and wanted the ability to take advantage of all manner of user-friendly and application-specific devices, like flatbed plotters, viewdata terminals, and communicating word processors and micros. The Information Centre did little to promote the attachment of anything other than a small range of "standard" devices.

Communications:

The early Information Centre tended to assume that all users would be in relatively close proximity to the central mainframe. But organisations were becoming more international in their outlook. Information pertinent to the running of today's companies is typically highly geographically distributed. Communications facilities available to the Information Centre implementors were woefully inadequate. The network was more often a barrier to communication than an interface between remote sites.

Implementation software:

Although "integration" was a watchword, the basic offerings of the Information Centre gave little hope of achieving this. The applications packages, being a collection from different historic origins, were implemented in the whole range of computer languages, and to entirely different standards of design and user interface. So, while the user might be content if his requirements were met entirely by one package, attempts to interface between them, or extend and customise them, were liable to meet with frustration. The implementation backlogs, which had begun to decrease with the arrival of the Information Centre, increased again along with the growth of demands for the ability to properly exploit, or extend the range of, the basic facilities.

The "missing links" manifested themselves in several problem areas:

User Isolation: I.C. users were victims of the ultimate in "Personal Computing", but unable to communicate with each other, or with each other's data.

Lack of Portability: Software applications, often with significant development effort, were rarely made available to other parts of the organisation, and "reinventing the wheel" became the rule rather than the exception.

Growth and Migration Problems: Ideas and solutions were typically incapable of growing beyond quite tight constraints, dictated by the implementation mechanisms.

Integration Difficulties: With few standards or guidelines, the "Do It Yourself" approach made any attempts at integration extremely difficult.

Insufficient Tools/Data: The Personal Computing philosophy, by encouraging users to think small, denied them access to the main strategic systems which often contained the most valuable data.

Add to these problems the arrival of the cheap, but very powerful, micro, with highly "user friendly" software packages, and it can be seen that the DP department is once again under attack. These micros can be bought at prices often well within an individual's unqualified signing approval, and yet they often provide power and sophistication above that achievable on the company's largest computers.

Towards a better strategy

We are very nearly at a stage where it will be possible, if we want, to place on every executive's desk a user workstation which can provide virtually infinite computing power, together with essentially unlimited memory capacity. But what do we want to do with it? All too often, this fundamental question is left unanswered.

For the end user, the cost of the hardware technology will no longer be any obstacle to the accessibility of computer systems. As computerisation spreads within organisations, there will be increasing demands for provision of new data and information to help the user in making decisions. End user systems will spread and overlap with each other.

The micro itself is **NOT** the answer to these new problems and opportunities. In a short time, the anarchy which has typified the growth of standalone microcomputer systems will be replaced by a need to re-integrate scattered, and often incompatible, systems. Equally, users will come to realise that their Lotus 1-2-3 or Multiplan systems are not an end in themselves, but should be only front ends to larger corporate databases and systems, since their inputs and outputs belong in, and relate to, other parts of the organisation. The end user does not function in isolation.

The Global Information Centre

The Global Information Centre (GIC) takes the basic framework of the Information Centre, addresses its problems, and adds the missing links in the earlier concept. There are several fundamental components which are critical in achieving true acceptability by both end users and DP managers — though each may regard the relative priorities as different.

The GIC assumes that in the typical situation, users may well be scattered around the world; that they will want to use a variety of terminal technology to access corporate systems, including intelligent terminals and micros; that there will be a requirement at a later stage to integrate applications systems, software, and data, even though the eventual needs may not be apparent at the outset; and that the DP department will want to offer a growth and development strategy with new products attractive to end users, while at the same time maintaining central coordinated control over corporate DP development.

The GIC strategy therefore does not prohibit end user acquisition of micros, though it insists that they must be capable of interfacing, at hardware and software levels, into the other elements of the GIC. Far from being seen as a restriction, many users will gain a feeling of comfort from the knowledge that their chosen micro system forms part of the long term strategy in their organisation.

Account should also be taken of Word Processing, automated PABX's, Local Area Networks, and the full range of office automation technology. They all can, and should, be integrated into the GIC strategy. This does not imply a commitment to a single-vendor policy, but simply a requirement that all selected vendors comply with INTERFACING standards. Careful attention should be paid to a vendor's commitment to Open System Interconnection (OSI).

The principal missing links which then, should be added to the original definition of the Information Centre are:

- * Worldwide Communications
- * Corporate AND external public data
- * Technology Independence
- * INTEGRATION
- * CO-ORDINATION

The best examples of Global Information Centre implementation are today probably in the service companies, whose historical development has been along Information Centre lines even before the phrase was coined.

Case Study: I P Sharp Associates

I P Sharp has in effect been running a Global Information Centre for some 15 years. It has developed almost entirely driven by client demands, serving users in a variety of organisations and industries who looked outside their own organisations for solutions to their problems.

The key elements of the Sharp GIC are as follows:

WORLDWIDE COMMUNICATIONS NETWORK: A private packet-switched network, developed internally in the absence of suitable available technology. A principal design goal was that it should be *invisible* to the end user, yet flexible in terms of the technology supported. Also interfaced to public networks, it provides local call access from some 600 cities worldwide.

SINGLE IMPLEMENTATION LANGUAGE: Utilising only SHARP APL, a proprietary enhancement to APL, all applications are implemented in the same language. Hence maintenance, integration, and interfacing of packages becomes relatively straightforward. The APL language (James Martin has said it is the only serious contender for the title of Fifth Generation Language) provides an extremely powerful user-friendly environment, with great attractions for systems programmers and end users alike. Applications, simple or complex, can be written quickly and elegantly — and can be easily modified when required.

INTEGRATED APPLICATIONS PACKAGES: Although a vast suite of packages and building blocks is available, a kernel of applications are seen as essential for the GIC:

- Financial Consolidation
- Financial Modelling
- Generalised Record Management
- Time Series Database Maintenance and Analysis
- Business Graphics
- Electronic Mail

— and these applications all readily interface to each other.

DATABASES: While the basis of the corporate Information Centre is very much an organisation's own private data, an increasingly common demand is arising for external data. The I P Sharp GIC offers some 50 million time series in areas like Finance, Economics, Energy, Aviation, Demography, and others. These databases can all be accessed using the same tools as are used for private data, and data from several sources, private and public, can be combined.

IBM PC INTEGRATION: An identical implementation of the Sharp APL environment on the IBM PC, together with software to interconnect mainframes to PC's allows applications and data to migrate from one to the other, and also permits interfacing to other PC applications.

SUPPORT and EDUCATION: Undoubtedly, the key to any successful DP strategy lies in the level of support and education available to the end user. The GIC user is encouraged to utilise the services of some 60 local branch offices around the world, where he can be assured, by virtue of the GIC structure, that the staff are acquainted with the user's applications (and can help him in his native language!).

The DP department too, requires support, but at a system level. This is provided 24 hours a day by staff specialising in, and using, precisely the same software environment as on the client machine.

A Strategy for the future

Whether following the I P Sharp philosophy, or that of another vendor, the fundamental issues remain the same. End users are increasingly becoming unconcerned with the technical implementation issues, whilst their demands continue to increase. On the one hand, they will say they want more "simplicity" in systems they use, but at the same time they expect much more sophistication.

The need is therefore very clearly for an implementation infrastructure that meets the needs of both the DP department and of the end user, even though their goals often conflict.

The Information Centre concept recognised that the end user was becoming more important in the computing environment. There can be no doubt that this trend is continuing, and so it is evident that the major issue of the next decade will be that of software development, with the end user as the main driving force.

The Global Information Centre, and the principles which underly it, do, it is suggested, offer a viable strategy for the future.

STEPS TOWARDS A BETTER BASIC — PART 3

by Anthony Camacho

This is the third extract from the series in Datalink in which Anthony Camacho attempts to explain some APL concepts in terms familiar to BASIC programmers. Here he takes BASIC string handling and re-casts it using the APL ideas of indexing and character vectors.

The article is reproduced by kind permission of Datalink magazine.

When there are many items in a single variable, there has to be a way of determining how many items there are and of picking out one or several of the items for separate processing. In BASIC only string variables can contain many items, and LEN, MID\$, LEFT\$ and RIGHT\$ are the functions used. They are fairly clumsy in use as the expression needed to exchange the third and fourth letters in a ten letter variable makes clear.

```
T$ = "ABCDEFGH I"
```

```
T$ = LEFT$(T$,2) + MID$(T$,4, 1) +  
      MID$(T$,3, 1) + RIGHT$(T$,6)
```

```
PRINT T$  
ABCDEFGH I
```

In Sinclair BASIC the expression is more elegant:

```
T$ = T$( TO 2) + T$(4) + T$(3) + T$(5 TO )
```

Some versions of BASIC allow:

```
MID$(T$,3,2) = MID$(T$,4,1) + MID$(T$,3,1)
```

and the Sinclair version is:

```
T$(3 TO 4) = T$(4) + T$(3) — even better.
```

In APL a variable can hold many items of any one kind: many integers, floating point numbers or Boolean values as well as letters; these arrays can have several dimensions so that tables and sets of tables are practical. LEN in BASIC only reports size in one dimension; APL needs 'SHAPE' and uses the Greek letter 'R' (rho) for the function. 'SHAPE' reports the shape of a variable by giving the length of each dimension in turn, so there are as many items in the result as there are dimensions in the variable. For example the shape of a variable which has two tables each three rows of four columns would be 2 3 4. The shape of 2 3 4 is 3 because it contains three separate integers, so you can find the number of dimensions of a variable by asking for the shape of the shape of it.

You can change the shape of a variable by giving a left argument which is the required shape (making the function 'RESHAPE'). If the number of items in the reshaped variable is not the same as in its original shape then items are lost or repeated as necessary. See Fig. 1 for some examples. APL uses the left arrow for assignment instead of the equals sign. Notice how convenient it is that APL displays any value you enter which is not explicitly assigned to a variable.

Fig.1

```

      A←1 2 3 4 5 6 7 8 9 0
      ρA
10    A ← THIS IS THE APL SYMBOL FOR
      COMMENT
      A NOW SOME RESHAPING OF A
      2 5ρA
      1 2 3 4 5
      6 7 8 9 0
      7ρA
      1 2 3 4 5 6 7
      3 9ρA
      1 2 3 4 5 6 7 8 9
      0 1 2 3 4 5 6 7 8
      9 0 1 2 3 4 5 6 7
      A NOW RESHAPE A INTO TWO
      TABLES OF 3 ROWS BY 4 COLUMNS
      2 3 4ρA
      1 2 3 4
      5 6 7 8
      9 0 1 2

      3 4 5 6
      7 8 9 0
      1 2 3 4
      A OF COURSE A IS UNCHANGED BY
      ALL THIS
      A
      1 2 3 4 5 6 7 8 9 0

```

The comma is used to join arrays together where BASIC joins strings with ' + '. The APL equivalent of LEFT\$ is the upwards pointing arrow (called 'take') which takes items from the left (or right if the argument is negative), and the downwards pointing arrow (called 'drop') similarly drops items from the left or right. APL uses square brackets to contain an index (and if there are several dimensions, separates the indices with semi-colons), so in the last example Fig. 1 A[1;2;3] would be 7. With these and other functions which can operate along the rows or down the columns of tables, APL offers very versatile ways of handling variables containing many values. Fig. 2 shows five APL equivalents of the BASIC above, incidentally showing in the fifth example that APL can assign values to part of a variable.

Fig.2

```

      A←(2+A),A[4],A[3],4+A
      A←(8+A),A[4 3],6+A
      A←A[1 2],A[4 3],A[5 6 7 8 9 10]
      A←A[1 2 4 3 5 6 7 8 9 10]
      A[3 4]←A[4 3]
      A THE VALUE OF A AFTER ANY OF
      THESE IS:
      1 2 4 3 5 6 7 8 9 0

```

There are far more operations that work on numbers than there are that work on strings. The handling of numeric arrays gives great scope. See Fig.3.

Fig.3

```

      B←2 5ρA
      B
1 2 4 3 5
6 7 8 9 0
      A NOW ADD 5 TO ALL ITEMS
      B+5
6 7 9 8 10
11 12 13 14 5
      A OR ADD VAT
      B×1.15
1.15 2.3 4.6 3.45 5.75
6.9 8.05 9.2 10.35 0

```

One frequent need is to carry out the same operation repeatedly on all members of an array. The function 'over' (APL uses the oblique stroke) inserts the function on the left between each item of the variable to the right. See Fig.4.

Fig.4

```

      +/A
45      A THE / REFERS TO THE LAST
      DIMENSION SO +/B ADDS THE
      COLUMNS
      +/B
15 30      A BUT ≠ REFERS TO THE FIRST
      DIMENSION SO +≠B ADDS THE ROWS
      +≠B
7 9 12 12 5
      A AND +/+/B ADDS ALL ITEMS
      +/+/B
45

```

CASE STUDY

A PRACTICAL INTRODUCTION TO EXPERT SYSTEMS

By Christopher Harvey

Part 2

Ed: The first part of Christopher's paper can be found in VECTOR Vol.1 No.3.

PROBLEM AREAS HIGHLIGHTED

Knowledge Engineering

One of the most interesting things to arise was the concept of Knowledge Engineering.

It is fair to say that most experts are reluctant to admit to the amount of knowledge they have. A second party is generally required to try to extract all the knowledge from the expert. This second party is the Knowledge Engineer.

We found this to be a highly rewarding task, although it was not particularly easy. The problem is that the expert does not know exactly what you are after. Some background knowledge of the expert's domain is undoubtedly of great use. As is the ability to "ferret out" information from the (willing) expert.

As well as acquiring the knowledge the Engineer must also organise the knowledge into a form that can be used by the system. This must always be in the Engineer's mind as he 'interrogates' the expert. The Engineer must therefore have the ability to convert the 'rough' data he obtains from the expert to, possibly, rules that the system can understand and use.

On the user side of the system, it is the Knowledge Engineer's responsibility to see that the system appears 'comfortable' to the human user and also to make the system's inference processes understandable to him.

Knowledge Acquisition and Refining

An expert generally becomes more of an expert. The system should really be able to do the same. There must also be a means of changing and amending the knowledge held within the Knowledge Base since the knowledge held will rarely be perfect; there will be inconsistencies and it must be possible to correct these.

Ideally these functions should be able to be performed by the user rather than having to call on the services of the Knowledge Engineer. We were therefore aware that this facility had to be made as easy as possible to operate.

However, a new problem arose: the 'power' of an Expert System derives from the knowledge within the Knowledge Base, not from how the inferencing mechanism operates on that knowledge. There is a point of view here that says that one does not let the 'naive' user enter knowledge into the Knowledge Base. The reason: when we say that the power lies with the Knowledge Base, we mean with its *structure*. In the case of a rule-based system, it is the interaction of rules

which is important. This implies that the user need have some knowledge of the structure of the Knowledge Base if he is to be able to alter the data held there to achieve anything near optimum performance from the system. This problem can be helped, however, by the inclusion of "knowledge about the knowledge", or "meta-knowledge", in the system. This is mentioned elsewhere herein.

A more far reaching, and powerful, feature of the Knowledge Refining process is that of the system being able to maintain and update its own Knowledge Base automatically, i.e., without the intervention of humans at all. Due to the continual updating of the knowledge held by the system, the Knowledge Base will become more complex. Various problems, such as duplication and even conflicts of knowledge will arise. Therefore, it is necessary that the system is able to detect, and correct, these problems. It should also be able to update itself when, during normal operation, it discovers some piece of information that it decides it should keep a note of permanently. But, how does it know when to add something to its knowledge and even then, how does it formulate the rule around which to store the information?

These problems will become more important as systems increase in size. Present systems are of the order of hundreds of rules but when Knowledge Bases hold 20,000 rules?

System and Implementation Language

Although, perhaps, it was not relevant to the operation of the system, the language that the system was written in provided problems which were most unwelcome.

We cannot understand why languages such as C and Pascal are popular for writing systems such as these. Obviously, it can be said that (most) languages have particular areas of application in which they are 'better' than other languages but what slowed this work up more than was appreciated was that so much time was spent worrying about implementation details of the language; "You can't do that in C". Languages (and their associated environments) exist which are far superior for a considerable number of problem areas, especially areas such as Expert Systems where "brain power" ought to be going into design problems not language implementation problems. Examples of such (fairly) widely available languages are Prolog, Lisp and APL. System writers who use one (or more) of these rarely move back to a 'conventional' language with any degree of enthusiasm!

The Operating System used was UNIX. There is a considerable amount of excitement presently about this system. A very complex file directory system had to be written for our Expert System. Compare this with, say APL's component file system! The point is made that the Operating System should do the work *not* the application system designer.

Another area worthy of mention was the (comparatively) poor run-time debugging available for the design of this system. Again, a considerable amount of time was wasted taking care of details that could have been handled by good programming aids.

Languages are now being developed with Knowledge-Based Systems in mind. These will be most welcome.

Whilst on the question of language, and for the record, PATTIE consists of 6000 lines of C, that being 120K bytes of source code which translated into some 20K of 68000 object code.

Pattern Matching

One of the great restrictions of our system was that of the pattern matching of rule clauses. Again, due to the time constraint involved, in the time available it was only possible to include a simple matching mechanism. This means that the system would not be able to recognise (for the purposes of processing) that the following clauses were identical:

- i) IF the pipe site is buried
IF the pipe site is buried underground
- ii) IF the pipe environment is near to a road
IF the pipe environment is close to a road

When one considers this problem it is by no means simple. Our system merely compares the text strings "buried" and "buried underground" with each other and then says that they are not the same. As the second example shows it is not just a case of matching parts of strings.

One way of solving this problem that we are currently working on is that of using "meta-knowledge". In the second example, it is the *concept* of "being in the vicinity of" that is important, not the actual wording used. A secondary level of "knowledge about knowledge" could be of considerable use here.

Another example of where this could be of some use is, for example, in the case of:

- IF the pipe site is buried
- IF the pipe site is not buried

At present, the system treats these two clauses as entirely distinct from each other. But, it should know that one is the 'opposite' of the other.

A further example, and another constraint, was the fact that our system has no 'relational' comparison ability. E.g. given the clause:

IF the pipe diameter is greater than 1 m
the system took "greater than 1 m" as a text string, as in previous examples. Thus if a comparison was made with the clause:

IF the pipe diameter is 4 m
the required result would not be obtained because the system would see no connection between the two clauses. This clearly depends on the type of information the system is handling but in our case it turned out to be a very important point. Not least because the system would ask the user what appeared to be daft questions, e.g.

- Q) Is the pipe diameter greater than 1 m?
- A) YES
- Q) Is the pipe diameter less than 1 m?
- A)

At this point, the system does not appear to be an 'Intelligent' Knowledge-Based System at all!

Calculations

Whilst on the point of the system holding numeric information; the system could, at certain stages of the processing, perform calculations based upon information gained up to that stage. The results of these calculations (on, say, pressures in pipes) could then be used to answer further system queries and/or be returned to the user. There would appear to be great scope here for the system to ask for certain information and then perform quite complicated mathematical calculations.

User Friendliness

The help facility was vital. It is all too common for systems to refer the user to a manual. We found that heavy use was made of this facility.

Also important is the general view the user has of the whole system. Items of relevance in this area are now mentioned.

It was considered that the system operating in terms of 'I' and 'You' (as mentioned previously) was an interesting approach that proved popular.

A strong criticism made was that the user is asked a lot of questions without actually knowing why they were asked. The inclusion of "I'm now going to ask you a few questions about" would be helpful. This may aid the user to accept what may appear to be 'odd' questions. Although the user always had full use of the Explanation Facility, it was considered better for the system to take the initiative and tell the user something rather than the user having to ask.

Another criticism was that all results from the system were presented "all in one go" at the end of the session with the system. This is a debatable point and depends upon the domain in question, the preferences of the user and the structure of the information returned.

The concept of 'noise' words was a quite successful idea which made the text output to the user much more readable. This point emphasises how important we feel the user's view of the system is.

Miscellaneous Areas

It was found that the operation of the system was fairly slow at times due to the vast amount of rule comparison (and its associated file accessing). Our system had about thirty rules; had it had of the order of hundreds, it would have been unusable. Not until the system is re-written and further tests carried out will we be able to say whether this is a major problem. Up to a point, we consider that slow speed of operation is not all that bad. There is something un-nerving about a computer that comes up with an answer too quickly. Especially in the case of Expert Systems it is felt that the system taking a few seconds over a question would instil more credibility in the belief attached to the answer.

Our system allowed the user to reply DON'T KNOW to a question but then did not actually do anything with the parts of the processing that were thus avoided. DON'T KNOW had to be taken as an answer of NO since it would not have been safe to proceed as if the reply had been YES. In this sense it was merely an additional aid to the user interface. It would be a good idea for this to affect the processing in some less drastic manner than to stop it altogether in the affected areas, i.e. to introduce some degree of fuzziness.

Our initial realisation that the Explanation Facility was important was re-emphasised. It is *vital*. Again, the question of credibility is raised. The system will have none unless the user can see exactly what the system did in reaching the conclusions it did. The extra work required in this area was well worth the effort.

it has been shown that the knowledge that an expert has is largely heuristic, i.e., good guesses and good practice rather than fact and rigour. This usually finds its way into how the system processes knowledge and this leads to the system needing to handle uncertain knowledge or information with some degree of confidence. We simplified the design of our system by choosing rules from a domain that did not require this kind of thought. This idea is however an area of research and debate and almost certainly (sic) needs to be taken into account in most system implementations.

Due to the way our system was designed it was found that the user would have to know a certain amount about the knowledge that the system had before he could use it. Careful thought has to be given to just where the dividing line is between the system being all-knowledgeable (in its own area) and the user having, to an extent, 'previous' knowledge. The domain expertise in question will be a deciding factor. In our case however, it did prove to be a problem.

Our system is specific to the domain of pipes and, indeed, was designed around rules about pipes. We did this to gain experience in building an Expert System. Our next system will be designed with the concept of a 'shell' in mind. That is, a system not specific to any one particular domain. This is important if systems are not to be re-written to some extent before being applied to a different subject area.

CONCLUSIONS

The aims of the project were more than satisfied. Developing our own system has been an invaluable experience and has highlighted the problem in many fascinating areas in which research is being pursued.

We can see a very exciting future for Intelligent Knowledge-Based Systems of one form or another. The number of these systems in operation can only multiply many fold over the coming years.

Specifically on our system, we have learnt in the best way; encountering problems as we went along. The final system is by no means an effective solution to the original design objectives. We view the system as a prototype. It requires redesign, bearing in mind what we have now learnt, and rewriting. We intend to do this in two languages, Prolog and APL.

In summary, we can say of specific areas:

Knowledge Engineering — This is one of the most interesting, fascinating and important aspects of the whole exercise. If this stage is not performed correctly then the system is almost certainly doomed to fail.

Knowledge Acquisition and Refining — Very important if the system is not going to remain 'static'. An Expert System must be able to grow and to learn. For the system to be able to do this automatically ...

Explanation Facility — Absolutely vital. Without this, it will have no credibility whatsoever in the eyes of the user.

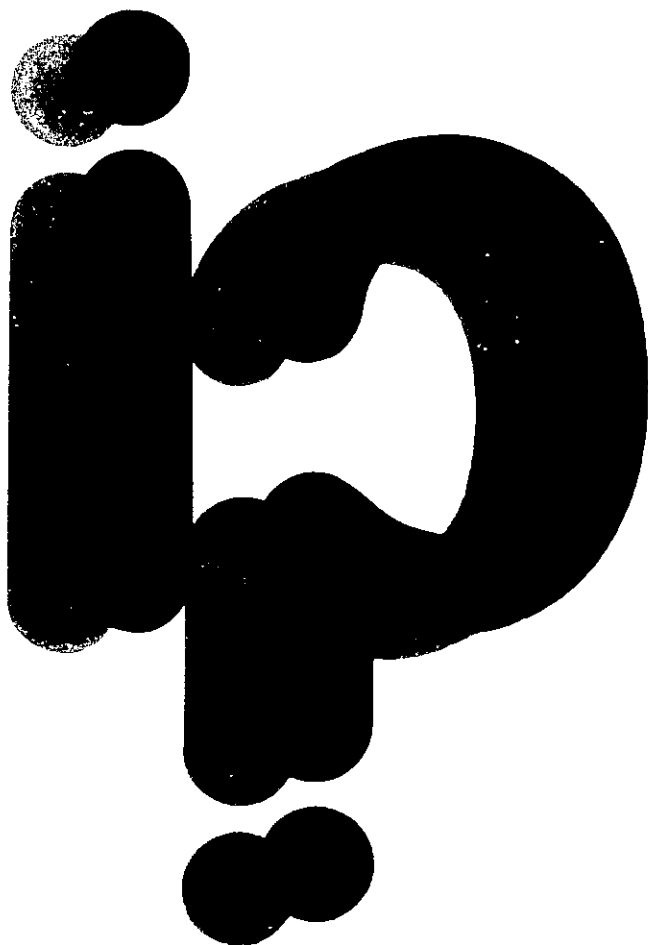
Knowledge Representation — There are many ways of storing knowledge and many associated inferencing mechanisms. Considerable research is being done in these areas.

Natural Language Interface — At one end of the scale is a simple Question/Answer system, at the other is free-format speech input. This part of the system must not be considered an ancillary add-on. Whether the system will be actually used or not could depend upon this interface. The most important part of the system is the user.

On the whole, a thoroughly worthwhile experience and one worth recommending.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL, and will contain items to interest people with differing degrees of fluency in APL.



**THE SOLUTION TO
YOUR APL QUESTIONS**
on IBM PC or Mainframe.
INNER PRODUCT

For further information, contact JAMES MANNING, Marketing Manager, Inner Product, Eagle House, 73 Clapham Common Southside, London SW4 9DG, or telephone 01-673-3354.

TECHNICAL EDITORIAL: IS APL ALWAYS APPROPRIATE?

by Jonathan Barman and David Ziemann

When should APL be used? As this journal is read mainly by those who have been converted to APL, perhaps the question should be put another way; are there any programming tasks where another language would be more cost effective? Those of us who are very committed to APL should stand back every now and again to take stock of their reasons for using APL, and look to see if it would be worth investing the time to obtain equivalent skills in another language. The APL community is rightly accused of religious fervour in favour of APL, to the exclusion of all other languages.

APL interpreters are large, and need a reasonable amount of memory to work satisfactorily. If there are less than 32K bytes of memory available, then one must consider using another language. BASIC is small, universally available, and universally hated by APLers, but does the job when it has to. FORTH is even smaller, and is pleasant to use when one gets ones head screwed on the right way!

APL can be slow, and if speed is important, then a compiled language or assembler may be more suitable. Most APL interpreters allow machine code to be mixed in with APL, so speeding up heavily used pieces of code is reasonably easy. Although it is possible to run machine code programs from within VS APL this technique does not seem to have been widely used, and it has been largely left to APL interpreters on micros to provide this facility. APL is not however relegated to those tasks where speed is not an issue. In fact it is sometimes used effectively in time-urgent environments; Myriade APL for example has most of its operating system environment written in APL. How about writing an APL interpreter in APL? Ridiculous? Not so; parts of VSAPL and the APL2 interpreters are written in APL (you may have noticed a strange message appearing when a workspace load is interrupted, which is because quadLX is written in one of the secret APL functions). The 10%—90% rule is often said to apply in such cases, where 10% of the code in an interpreter account for 90% of the running time. In this situation, the essential 10% is written in assembler, while parts of the less critical 90% can be implemented using APL itself.

The ACM Communications has a regular feature 'Programming Pearls' written by Jon Bentley, and in the February issue he explores these 10%—90% 'rules', which apply to many other areas beside speed of execution - recommended reading if you have access to a copy.

As well as running machine code, we also want APL to talk to packages and systems written in other languages. Sometimes the interface can be made directly by APL itself, and other times a low-level interface program written in C or assembler might be used. Two contributed technical articles in this issue report on the successes using these kinds of technique.

Finally, and perhaps most importantly, there is the question of whether an application is, by its nature, more appropriately programmed in a language other than APL. This is a very difficult question to answer, as it requires extensive knowledge of at least two programming languages, which is a fairly rare skill. With nested arrays, APL is coming of age and appears difficult to beat. The major contenders for special applications seem to be Prolog and its derivatives, and Nial.

We are particularly keen to receive letters and articles on APL interfacing and also on the use of languages and systems other than APL. Please keep your contributions coming!

VECTOR Publication Standards for APL Code

In order to make the production of VECTOR easier, the following rules should be adopted when preparing letters, articles and other contributions for publication.

Please do not include APL symbols within the text. If an APL symbol needs to be referred to, use its name; most APL textbooks have a list of symbols and their standard names.

Where variable or function names are referred to within the text they will be set in ordinary capitals. This means that sentences have to be carefully worded so that a function or variable name can be easily distinguished from any other words that need to appear in capitals.

APL statements and function listings will be reproduced from the submitted document whenever possible, and will be pasted onto a separate line between the text. This means that the listings should be of high quality; a 'daisy-wheel' printer with a carbon ribbon should be used. If you do not have a quality printer, we are prepared to re-set the APL ourselves if it is not too long, but we would much rather not do this as it is all too easy to make mistakes. Ideally we need two documents; the "readable" copy which shows what the final appearance is likely to be, and the "printer's" copy which we actually send to the printers. The "printer's" copy needs to have all the APL completely separated from the text. The easiest method is to leave three blank lines in the text where the APL is to appear, and to have a separate sheet containing all the APL; each section also separated by three lines. A simple numbering scheme can then be used to show where the APL block is to appear in the text. These APL characters will be photo-reduced and inserted into the text body.

Drawings, figures and black and white photographs are welcome, as they help to break up the text and make articles more visually interesting. Please prepare each figure on a separate sheet of paper, and give it a number and caption. The caption will be typeset and will appear at the top of each figure, which will be presented in a box as near as possible to the reference in the text. Do not worry about the size of the drawing, as the printers will photo-reduce it to fit the size of the page or the space available. Your text should refer to the drawings strictly by sequential figure numbers.

Footnotes and references will be grouped together at the end of the article, so please avoid writing text that requires a footnote to be on a particular page.

Introduction to Contributed Articles

This issue we are presenting three contributed articles. In addition we have a review of QL/APL and also a "Pot-Pourri of Improveable Code" from Dick Bowman, employee of the CEGB and the Association's Activities Officer. Dick's contribution started as a letter in response to our plea for improveable code, but as postscripts were added one by one it became apparent that it was too long for the letters page. Therefore the result of Dick's efforts can be found in the "Surely There Must be a Better Way" feature, and we welcome any responses or further input to the column.

The contributed articles follow the theme of "Is APL Always Appropriate?". In "Moving Data from 1-2-3 to APL" Michael Carmichael of Occidental International Oil Inc. recognises that many people use Lotus 1-2-3 to ease the strain of developing data entry front-ends in APL. He describes four techniques that he has used to import 1-2-3 worksheets into APL.

A different approach is taken by Romilly Cocking, a Director of Cocking and Drury Ltd., who uses a Symphony utility in conjunction with a program written in the C language to import Symphony worksheets into APL via DIF files. His article is entitled "A DIF File Interface".

Finally, "When Domino is not Sufficient" explains how APL code can be used to avoid the intrinsic problems caused by the application of the matrix inverse function to singular and near-singular matrices in MIPS APL. This is presented by A Sykes and J Ansell of the Department of Management Science and Statistics, University College of Swansea.

Technical Correspondence

From Peter Donnelly

13 February 1985

Sir: With reference to Mr Horten's letter in the October issue of VECTOR, please note that Dyalog APL provides a system variable to control the result of division by zero.

```

      DIV←0      A the default
      2÷0
DOMAIN ERROR
      2÷0
      A
      DIV←1
      2÷0
0

```

Yours sincerely,

Peter Donnelly
Dyadic Systems Limited,
30 Camp Road,
Farnborough,
Hampshire GU14 6EW

From: Dan Wimbock MBAA

12 February 1985

Objets Trouvés on the Circle Line

I found the attached document underneath a discarded copy of the Financial Times on a westbound Circle Line train at Baker Street after the What's New for 1985 show; it appears to contain details of a few new features of a hitherto-unknown APL implementation.

Perhaps the author would recognise the document if you were able to find some room to publish these proposed language extensions, indeed perhaps it would be of interest to many members of the Association.

Might we even hope for some comment from members of the APL Standards group?

Best Wishes,
Dan Wimbock.

Ed: The document Dan Wimbock refers to is reproduced below in the hope that it will be identified by the author. Unfortunately Dan did not provide us with his address - but doesn't that name seem somehow familiar?

Proposed New APL Features

quadDX - The Dormant Expression

We are all familiar with quadLX, which is activated whenever a workspace is loaded, and its value for conditioning the environment. Of equal importance in many applications is the need to decondition the environment. Take as an example a graphics application, where we might have a personal need to produce text upside down in large pink letters. If we leave the terminal in this state the next user may not get overly enthusiastic about us (a subtler ruse to ensure personal monopoly of the graphics terminal is to define all colours as invisible - this unfortunately leads to large bills from the engineer's calls).

While it's easy enough to write functions to perform this deconditioning it's not so simple to remember to use them (if the timesharing service just falls over you don't get the chance anyway). Which is where quadDX comes in; in the same way as execute quadLX is the first thing to happen when a workspace is loaded, so execute quadDX will be the last task carried out when you signal disinterest in your current workspace (e.g. by)OFF or loading a new workspace).

quadZILCH - Prototypical assignment

Easy enough to wipe out a variable with quadEX, or to assign it a completely new value; but rather tedious to keep its name in use, retain its type and give it an innocuous value.

In the new regime quadZILCH namelist is going to take all variables named in the namelist (same construction as quadNC), and set them to have prototypical values. That is, rank unchanged but all elements of shape set to zero and the variables flagged as containing characters or numbers in their last incarnation.

)RECOVER - Workspace undropping

The)DROP system command is commendable in its ability to make more library space available but suffers from a degree of finality; how many times have you realised that there was something useful in a workspace you no longer have? And how often did you realise this two seconds after you dropped it?

)RECOVER saves all the heartache by restoring dropped workspaces to your library.

quadREVERT - Inverse Assignment

How useful a facility, to be able to gracefully unravel the results of illadvised assignment. Examine the following sequence of events:

```

WALLY+2 3ρ '• '
WALLY+99
WALLY+'A CHAR VECTOR'
WALLY+3 4ρ 'A'
WALLY+9
DOMAIN ERROR
WALLY+9
      ^
WALLY □REVERT 2
WALLY+9
11

```

The left argument names a variable, while the right argument specifies how many assignments to ignore (naturally quadREVERT is origin-sensitive).

)WSFNS and)WSVARS - Contents of other workspaces

A frequent occurrence during the development process is the need for a specific function or variable; you have a vague idea where it might be but not an exact one. The tedious solution in oldAPL is to save the workspace you have and successively load likely candidates. Not only is this tedious, but if shared variables are in use it may not even be possible. Two mechanisms for avoiding this problem are proposed:

- An extension to the present)FNS and)VARS whereby a workspace name may be specified:
e.g.)WSFNS 12345 WALLY
will display the names of all functions in workspace 12345 WALLY
- A more active searching method in which the variable itself is specified to the system:
e.g.)WSFNS RTJUST
displays the names of all workspaces on the system which contain a function called RTJUST

The latter may be preferable from the viewpoint of the seeking user but might need further consideration from a security aspect.

Ed: Undoubtedly the above proposals are long overdue for standardisation, save for quadZILCH perhaps, which can be coded via the user-defined function given here:

```

V ZILCH W;1;J
[1] A COLLAPSE EACH VARIABLE NAMED IN <W> TO ITS PROTOTYPICAL VALUE
[2] J←(I+1)⊕W+MIM,' ',W
[3] LP←(I>J)/0
[4] W[I;],'+(0LP',W[I;],')p',W[I;]
[5] I←I+1
[6] →LP
V

```

where the function MIM converts a blank-delimited vector into the corresponding matrix. The variable names used above should of course be changed to reduce the chance of name clash with those in the function argument.

Further to Dan's proposals, we would additionally like to offer the following suggestions:

Interpreter version control

A surprising omission from the above (as yet unclaimed) document is the extremely useful system variable quadVER which returns the version number of the interpreter being used. Naturally it allows the user to determine at run time whether the interpreter level is recent enough to support some desired APL feature. Sometimes it is also helpful to be able to rely on a feature belonging only to a previous release of the interpreter, such as some error behaviour or a particular quirk. Assignment of the required version number to quadVER achieves this result. Application developers will now be able to protect themselves from the untoward effects of interpreter improvements, and can guarantee that their code will run even after such modifications. The careful localisation of quadVER can allow this effect to be isolated to certain sensitive areas of the system.

This powerful tool is open to abuse however, and great care must be taken to avoid version specifications that refer to interpreter levels that were written prior to the implementation of quadVER. If this should be done, the user will never be able to return to the current release.

Another use of this extremely flexible tool is of course in writing APL code that employs interpreter features that have not yet been developed. This is achieved by merely assigning the version number of the required release. The interpretation of vector assignments to quadVER is currently a topic of hot debate in certain parts of the APL community.

Completeness of quadNL.

As you probably already know, APL's system function quadNC returns the name class of each of the names specified in its argument according to the following table:

- 3 - Function
- 2 - Variable
- 1 - Label
- 0 - Identifier available for use

Similarly, the system function quadNL does the reverse trick, returning all the names of specified class numbers. For example executing quadNL 3 will return a matrix of all the defined function names in the active workspace.

Now, regarding APL's usual consistency and completeness concerning such affairs, it is a matter of no small surprise to discover that this feature is in fact incomplete. QuadNL always yields the correct result when applied to any of the integers 3, 2 or 1, but does not work for 0! Clearly this is an oversight made in the early days of APL\360 design which needs to be rectified as soon as possible. The result from quadNL 0 should of course be a table of all the valid APL identifiers that are currently available for use in the active workspace.

Apparently there seems to be resistance from some of the hardware manufacturers to fix this (what can only be described as a bug), although IBM have recently expressed interest in including the feature in APL2.

SURELY THERE MUST BE A BETTER WAY

A Pot-Pourri of Improveable Code

by Dick Bowman

How To Survive In NOTXXAPL

People jump to conclusions about what XXAPL really is; there's no significance whatever to the two X's save that it makes for a convenient cadence when you say it. We can regard it as a generic term for any APL which doesn't have the bits you want and may well manifest itself as XXXAPL, XXXXXAPL or even XxxxxxxxAPL for the historically minded.

Having been using one particular XXAPL for some years I've noticed myself spending a certain amount of effort emulating features in other APLs; ironic now to find myself in possession of an 'other-brand' interpreter and a need to emulate things I'd been taking for granted.

A specific instance is the quadEA system function, a sort of dyadic execute (why it wasn't called that I'd be pleased to find out) which for the benefit of the unendowed has the syntax:

<L quadEA R> which is translated as:

execute R, but if it fails
execute L, and if this fails as well thanks for spending the CPU cycles

Not all APLs have this feature, and faced with an application using it you need to emulate it. So here I present a first (low-tech) approach using the APL *PLUS/PC error handling facilities.

```

      V L ΔEA R;⊞ELX
[1]  ⊞ELX←'→LEFT' ⋄ ⍉R ⋄ →0
[2]  LEFT:⊞ELX←'→RIGHT' ⋄ ⍉L ⋄ →0
[3]  RIGHT:⊞ERROR((⊞DM1⊞TCNL)-⊞IO)+⊞DM
      V

```

Which seems to work reasonably well; the only case to consistently foul it up is having a branch as one of your arguments (I prefer to avoid this situation). It also gave me a painless insight into using this style of error-handling. An analagous example in SharpAPL is contained on page 143 of the Proceedings of the 1980 Sharp APL Users meeting.

I leave it as an easy exercise for non-Committee members to emulate error-trapping via quadEA.

Am I In Belgium?

One of the indicators of whether you're looking at useable graphics hardware is support of polygon filling; what you want to be able to do is to issue a command to draw a shape and have it filled with colour without doing too many mental gymnastics yourself (after all, these boxes cost money). Generalising, you can divide the hardware into three categories:

- the appropriate command is intrinsic (this covers the case of a certain well-known specialist)
- the appropriate command is intrinsic and doesn't work (I draw a veil over who this might be)
- there's a command to draw the outline and another which lets you start filling from a specified seed point (a well-known vendor of APL interpreters offers this one)

If your equipment came from the first category you're home and dry, except that I wager it probably hasn't got an APL keyboard. If from the second you have an interesting problem on your hands. For the third category I offer some assistance.

As one coming very much from the school of why work it out for yourself when somebody else already did the hard work I acknowledge quite freely that this is a lift from the pages of APL Quote Quad. All I've done is to cut out the bits I really don't need and correct the typos (as an aside, Quote Quad typos are an excellent way of learning APL).

The source from which all flows is the March 1984 issue in which Bastian and Cervini presented a set of functions for hatching polygons; once you get it working you can draw lines across irregular shapes which are visible inside and invisible outside. With the problem of a polygon in one hand and a need to find an internal seed point in the other we can now reduce it to finding the mid-point of any one of the visible line segments which Bastian/Cervini will so prolifically provide. Even the committee can work this one out, especially when they realise that only a single line is needed.

For clarification, I am perverse enough to have persisted in using a polygon definition which is the transpose of everyone else's (first row is move (0) or draw (1), second is x-coordinates, third is y-coordinates); these functions are restricted to the simple case of a single closed polygon with no invisibility in its border.

```

V H←SEED POLY;ΠIO
[1]   ΠIO←1
[2]   H←(RAY@POLY)[1 2 ; 2 3]
[3]   H←(I/H)+0.5*(I/H)-I/H
V
V H←RAY PL;A;B;J
[1]   B←PL[;3]
[2]   B←(I/B)+((I/B)-I/B)+2
[3]   H←B HACHEL PL[; 2 3]
V
V H←D HACHEL P;I;J;K;F
[1]   F←P[;2]-D
[2]   F←F,[1.5]I@F
[3]   J←v/I+0>×/[2]F+1E-8×0=F
[4]   F←(F,[1.5]I@F),J/F
[5]   K←I/F
[6]   K[;2;]←-/[2]K
[7]   K←K[;1; 1 2]-K[;2; 1 2]*K[;1; 3 3]+K[;2; 3 3]
[8]   H←((1+ρK),1)ρ0 1),K[4K[;1;]]
V

```

I repeat my obligation to Messrs Bastian and Cervini for doing the hard work; I claim only to have simplified their code to achieve a specific objective. The result returned from <SEED> is a two-element vector of x and y coordinate for a seed point; it has been working in practice for a little while but I would be grateful to anyone providing perverse cases. An example:

```
U+3 9p0.(8p1).10 60 60 50 50 20 20 10 10 10 10 60 60 20 20 60 60 10
SEED U
15 35
```

Postscript 1 — Yankee Imperialism

Perverse cases do indeed exist which can result in a seed point which is considered to be outside the polygon by the graphics hardware; such case being those where the calculated point is very near or even on the border of the polygon. What seems to happen is that the discrete nature of the hardware takes over; finds the point to be on the border and arbitrarily decides that inside is outside.

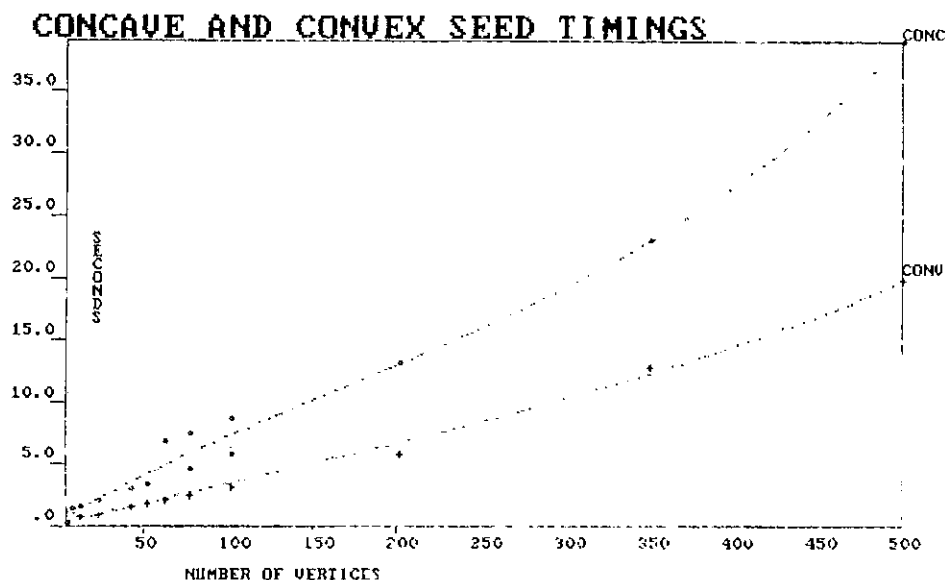
Postscript 2 — Its nicer in Wyoming

Some polygons are simpler than others; how much of your time is taken up in drawing triangles, rectangles, minor segments of circles, and other convex objects? The nice thing about these objects being that they have a centre of gravity which is strictly within their boundary. The modified function below determines whether or not the polygon is convex (triangles always are, for others we take a tour of the boundary noting whether the first turning is to left or right - so long as we keep turning the same way we're going round the outside of a convex polygon). If it's convex then we can get away with using the centre of gravity as the fill seed.

```
V H←SEED POLY
[1] I←1
[2] →(42~1+pPOLY)+L1
[3] →(CONVEX POLY)+L1
[4] H←(RAY@POLY)[1 2 ; 2 3]
[5] H←(L/H)+ 0.5 0.5 ×((L/H)-L/H)
[6] →0
[7] L1:H←(+ / 1 ~1 +POLY)+~1+~1+pPOLY
V
V Z←CONVEX POLY;M;TH;S;XY
[1] XY← 0 ~1 +~POLY[2 3 ; ]-1@POLY[2 3 ; ]
[2] TH←~1@XY[2 ; ]+(+XY*2)*0.5
[3] S←XY[1 ; ]<0
[4] M←((~S)\(~S)/TH)+S\S/(01)-TH
[5] M←(1+(M=1/M))11@M
[6] Z←(1+M)^.2~1+M
V
```

Timed using APL*PLUS/PC 3.0 on a PC with Hercules graphics board the saving is around 40% if you're drawing convex shapes (see figure 1); purists may rightfully point out that it's only necessary to calculate a seed using three vertices of the original polygon but my excuse is that using all points might lessen the chances of getting too near to the border.

Fig. 1



Postscript 3 — I guess it all depends on what you know

Those in possession of an APL which supports user-defined ambivalent functions might consider a modification allowing the input of a seed point if the application already knows what it is. Thus

```
SEED←{SEED} SOLIDPOLY POLY
```

So that if you know what the seed ought to be you can bypass all of the difficult sums, thereby making things even more economic for circles and so forth.

FORTRAN - A Popular Language

Some of you may have heard of Fortran; it has been around a while and amongst its other attributes it demonstrates that APL users aren't the only people who have the arrogance to expect their users to absorb the conventions of the programmer.

In this case the offender is a construct resembling

2,3,4*5,6,7*8

which means one repetition of each of the numbers 2 and 3, followed by four number 5s, a six and seven 8s. Users of many versions of XXAPL will immediately recognise a mutated form of their own replicate function; if you haven't followed my gripping prose, the above is Fortran for

1 1 4 1 7/2 3 5 6 8

Not only can Fortran programmers understand this, they also expect their users to incorporate it into input data; which is just great when it gets read by Fortran programs. If you have to read this gibberish into an APL function you might like to consider <DECODE> below:

```

      V Z←DECODE R;L;M;N;S
[1]  L←', ' SHAPE R
[2]  L←(0 2 +pL)÷L
[3]  L←(pL)÷(2×v/Lε'*)φ(((1+pL),2)p'1*'),L
[4]  S←,Λ\Lε'*'
[5]  M←zS\S/,L
[6]  S←,φΛ\φLε'*'
[7]  N←zS\S/,L
[8]  Z←(,M+.≥1Γ/M)/.q((Γ/M),pN)ρN
      V

```

It's not very elegant, but it worked for long enough to convince my user that he didn't like the Fortran program. Incidentally <SHAPE> is just a vector-to-matrix converter, the one below is filched from IBM's APL Handbook of Techniques.

```

      V ΔA+ΔB SHAPE ΔC;ΔD
[1]  ΔA+((=∼ΔC='''')^ΔCεΔB)/1ρΔC+ΔC,1+ΔB
[2]  ΔD←Γ/ΔA+ΔA-□IO,1+1+ΔA
[3]  ΔA+(-0=ΔA)÷ 0 -1 +((ρΔA),1+ΔD)ρ(,ΔA+.≥(∼□IO)+1ΔD),1)\ΔC
      V

```

The variable names within <SHAPE> are the result of an encounter with a nameless local variable renamer at an earlier stage in my life when I needed to rationalise local names to a user-inaccessible form.

And for the benefit of anyone who still hasn't worked out how Fortran came to be so popular with programmers and users alike:

```

      DECODE '2,3,4*5,6,7*8'
      2 3 5 5 5 5 6 8 8 8 8 8 8 8

```

Yes, I thought * was Fortran for multiply as well.

PRIZE COMPETITION: RESHAPING ROWS OF A MATRIX

by Jonathan Barman

This problem arose from a user requirement that initially seemed irksome, but turned out to have an easily stated solution. It involved the creation of a set of tree structured codes from a list of basic codes. The solution came down to taking a matrix of codes of varying lengths and reshaping each to fill a specified width. The function <ROWRESHAPE> does the trick; notice that trailing blanks are not reshaped.

If the variable <A> contains the codes to be reshaped, then:

```

      A
CODES
TO BE
RESHAPED
ON EACH LINE
      ρA
4 12
      20 ROWRESHAPE A
CODESCODESCODESCODES
TO BETO BETO BETO BE
RESHAPEDRESHAPEDRESH
ON EACH LINEON EACH

```

The problem is to write the function <ROWRESHAPE>. As usual the program must conform to the draft ISO APL standard (assume VS APL). A first prize of £30 and two others of £10 each will be awarded to the winners. The closing date for entries is 31st July 1985.

Prize Competition Result: Numeric Matrix Validation

by David Ziemann

When I discovered that we hadn't received a single entry for this prize competition my immediate reaction was intense disappointment - especially considering the importance and relevance of the subject. However, this emotion was soon converted to relief when I imagined one of the possible alternative outcomes - that of receiving ONE very bad entry!

Anyway, having learned my lesson, I shall endeavour to ensure that future competitions don't actually involve too much hard programming.

One possible solution to the problem is now described.

```

V R+A VALIDNUMSMAT W;V;L;W1;B;E;M;V;C;D;EI;IO
[1] A VALIDATE EACH ROW OF CHARACTER MATRIX <W> AS ONE VALID NUMBER
[2] A |<A> IS SCALAR VALIDATION TYPE: 1/2/3/4=BOOL/INTEGER/REAL/E FMT
[3] A <R>[;1]=VALID/INVALID(0/1) <R>[;2]=THE NUMBER IF VALID
[4] A BLANK ROWS OF <M> ARE VALID ONLY IF A<0, WHEN 0 IS RETURNED
[5] A TOO LARGE E FORMAT NUMBERS ARE INVALID AND RETURN A 1 IN <R>[;2]
[6] A  $\rho R \leftarrow (1 + \rho W1), 2$ 
[7] IO+1
[8]  $V \leftarrow W1 + (\sim 2 + 1 \ 1 \ \rho W) \rho W$ 
[9]  $V[(V = '-' ) / 1 \rho V] \leftarrow '-'$ 
[10]  $L \leftarrow / \wedge \phi' \ ' = W1 + (\rho W1) \rho V$ 
[11]  $W1 \leftarrow (-L) \phi W1$ 
[12]  $\rightarrow (a, b, c, 0) [1 \ 2 \ 3 \ 4 \ 1 | A]$ 
[13] A BOOLEAN
[14] a: R+2 VALIDNUMSMAT W1
[15]  $R \leftarrow 0 \vee R \times B, [1.5] B \leftarrow R[;2] \in 0 \ 1$ 
[16]  $\rightarrow W$ 
[17] A INTEGER OR REAL
[18] b: M+ '01111111111239' [' 0123456789' . ' ' ' , W1]
[19]  $V \leftarrow 1 + \pm '0' \ , ((V \in '23') \vee V \neq '1' + ' ' , V) / V \leftarrow , ' ' , M$ 
[20]  $R \leftarrow V \in (6 \times 2 = |A|) + 13 \ 31 \ 131 \ 213 \ 231 \ 2131 \ 1 \ 21$ 
[21]  $R \leftarrow R, [1.5] R \setminus 1 + \pm '0' , , ' ' , R / W1$ 
[22]  $\rightarrow W$ 
[23] A E FORMAT
[24] c: E+ + / v \ W1 = 'E'
[25]  $M \leftarrow (-E) \phi (1 \ 2 \ \times \rho W1) + W1$ 
[26]  $C \leftarrow 3 \ \text{VALIDNUMSMAT}(\rho W1) + M$ 
[27]  $D \leftarrow \sim 2 \ \text{VALIDNUMSMAT}(0 \ 1 \ -\rho W1) + M$ 
[28]  $E1 \leftarrow (A / 0 \times C[; , 2], E) / 1 \rho E$ 
[29]  $V \leftarrow D[E1; 2] + 10 \times |C[E1; 2]|$ 
[30]  $B \leftarrow V < 10 \times | / 10$ 
[31]  $C[E1; 2] \leftarrow (\sim B) + (\times C[E1; 2]) \times D[E1; 1] \times B \setminus 10 \times B / V$ 
[32]  $R \leftarrow (C[; 1] \wedge D[; 1]), C[; , 2]$ 
[33]  $R[E1; 1] \leftarrow R[E1; 1] \wedge B$ 
[34]  $w: R[; 1] \leftarrow R[; 1] \vee (A < 0) \wedge L = \sim 1 + \rho W1$ 
V

```

14 9 ρ TEST

TEST

3

-7.1

3..1

2.2 3.4

1E3

-1.71E27

131.7E-1

13E1.6

6E789

1.23E-307

1.23E-308

1E2E3

3.E2

```

      TEST,V-4 VALIDNUMSMAT TEST
3      1      3
-7.1      1      -7.1
3..1      0      0
2.2 3.4      0      0
1E3      1      1000
-1.71E27      1      -1.71E27
131.7E-1      1      13.17
13E1.6      0      0
6E789      0      1
      1      0
1.23E-307      1      1.23E-307
1.23E-308      1      0
1E2E3      0      0
3.E2      1      300

```

The function <VALIDNUMSMAT> above expects as a right argument a character matrix where each row is to be validated as a single number. The scalar left argument indicates the type of validation required; boolean, whole number, decimal number or E format (scientific notation), each successive validation type being a 'superset' of the last. The result is a two column matrix where the first column is a 1 for a valid single number in the corresponding row of the argument, or a 0 otherwise. The second column is the actual number, if valid. Blank rows of the argument are considered invalid unless the left argument is negative, in which case 0 is returned.

The first few lines prepare the argument by ensuring it's a matrix, mapping any minus signs into high-minus symbols and right-justifying it. Line 12 is a branch according to the required validation type. Lines 18-21 are an expression of that by now well known technique for numeric validation, as demonstrated in VECTOR Vol.1 No.1 page 133. The parenthesised part of line 20 determines whether to validate for decimal numbers as well as whole numbers.

The boolean validation segment in lines 14-15 first validate the argument as a set of integers (type 2) and then check that the results are 0 or 1 only. Note that in this case the explicit result is forced to be boolean on line 15.

A similar approach is used in lines 24-33 for E format input, but with the text to left of each 'E' being validated independently from that to the right. Line 24 determines which rows are potential E format numbers. Lines 25-27 validate the left portion (mantissa) as a decimal number, and the part to the right of the 'E' as a whole number (exponent).

The next four lines create the actual number from the two validated portions. Because an entered number may be too large for the machine (e.g. 1E791) the result cannot be formed directly. Instead the logarithm of the number is compared with the logarithm of the largest machine number to determine validity, on line 30. Line 31 then forms the actual results without fear of a DOMAIN (or LIMIT) ERROR. For numbers which are too large, line 31 causes a 1 to be returned in the second column of the result, so that the application can tell why the E format number was bounced. Taking logs may be slow, and so the technique is used only for those rows that have 'E's in them. The advantage of this method is that it will work regardless of the size of the implemented number limit.

The last line permits blank rows to be accepted with a result of 0 provided that the left function argument is negative.

Competition Rules

- Entries must be in legible English or APL as appropriate and should preferably be machine produced.
- Entrants must declare the type of computer and the version and release level of the APL interpreter on which their functions were written.
- The date and your full name and address should appear on each sheet of your entry.
- Entries should be physically separate from other contributions such as letters, and should be clearly marked 'Competition Entry'.
- All submissions should be sent to the editor.
- Those on the committee, activities group or journal group of the British APL Association are ineligible.
- DOS format diskettes containing APL*PLUS, IBM or Sharp APL workspaces are acceptable. Diskettes will be returned.

KEYWORD QL/APL - AN (OCCASIONALLY) TECHNICAL REVIEW

by David Ziemann

Well, it's here at last - a fully featured APL interpreter that runs on a home micro. It was not long after the first rumours that MicroAPL were planning an APL implementation for the Sinclair QL that I finally stopped procrastinating about buying a home computer. So with a telephone in one hand and a plastic card in the other I ordered my QL and expectantly awaited its arrival. Several years later (or so it seemed) it surfaced. I looked at the four Psion packages that come with the machine (Easel is good) and - dare I admit it - toyed gingerly with the Superbasic language (an excellent Basic, but appears to be slowish). Then I put it away. A while later QL/APL arrived, and the machine seemed strangely more attractive. QL/APL vied with the likes of Dallas and Doctor Who for the use of the television, and often won. This is the story of my initial excursion into QL/APL.

What you get

The QL/APL package that I received comprised a QL/APL manual, a handy reference "card", a ROM with carrier and a microdrive cartridge that contained the APL interpreter and a demonstration workspace. In fact about 30K of the interpreter resides on the ROM, which must be plugged into the ROM slot at the back of the QL if you want to use APL. Presumably this frees up some space on the supplied cartridge as well as providing a good degree of copy protection. Having previously had a few nasty experiences with microdrive cartridges (poor maligned beasts) I swiftly took a copy of the master to use as the working copy.

The manual seems comprehensive and introduces QL/APL to three types of user: the user with no computer skills, the user with experience of a computer language and the traditional APL programmer.

Starting off

The first thing the QL does when you power it on or press RESET is to ask you if you're using a TV or monitor. QL/APL makes its presence felt even at this early stage by additionally displaying the following reassuring message:

QL/APL, Keyword Version 1.03

This tells you that the APL ROM is in place and all is well.

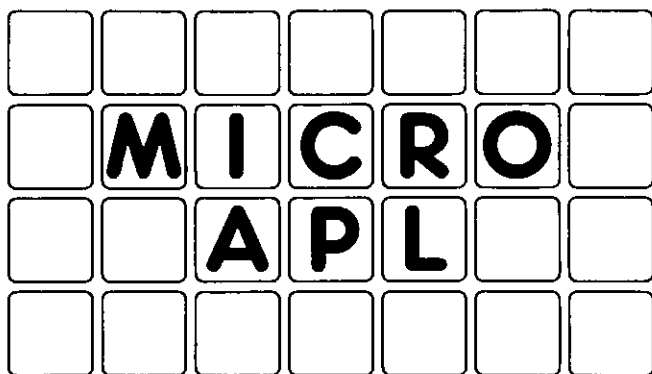
Initial problems

Because I was using an (old and failing) colour TV, I need to press function key F2 (there are 5 function keys in all). This tells the QL that a TV is being used, and then searches for a boot file on the cartridge in microdrive 1 (the left one of the two). With the QL/APL cartridge in place this causes APL to be loaded, and a large red banner is displayed to confirm this. The total load time comes out to a variable 15-20 seconds (not bad compared with a PC), and after this period I was greeted with:

QL/APL

Copyright (c) 1984 MicroAPL Ltd.

Keyword Version 1.03 Ws size = 28K



where APL means business

You know what's good about micros. Fast response; no monthly bills; software designed for users, not programmers; access whenever you like. In a word, independence.

You know what's bad about some micros. No networking; no access to your mainframe data; fewer features than you're used to; no technical and service backup. In a word, isolation.

Wouldn't you like to see a full range of APL micros, from the IBM PC to large multi-user machines? With local area networking, and IBM and ICL terminal emulation? With full lookalikes of features like IBM's AP124 and AP126? All from a company with the commercial and technical muscle to support everything it sells?

MicroAPL Limited
Unit 1F, Nine Elms Industrial Estate
87 Kirtling Street, London SW8 5BP
Telephone: 01-622 0395 Telex: 896885 IOTA

followed by the message

LEAR WS

Experience with other APL systems suggested that I had probably not been dumped into some Shakespearian application, but rather that I was just losing the first character of the machine's responses. Experimentation showed this to be the case, and it was clear that the TV was giving the first column the chop. Although QL/APL was certainly not to blame for this behaviour, I decided to see if QL/APL could be used to provide a solution. It was apparent from the thin white border around three sides of the TV image that QL/APL had defined itself a window to work within, and that if the size of the window could be reduced a solution might be found. APL.68000, on which the QL interpreter is based, uses a console control system function to handle console operations, and it seemed likely therefore that QL/APL should do things similarly. After finding quadCC in the manual (or hashCC as it should probably now be pronounced) a solution was obvious. The statement

```
#CC -8 15 2
```

did the trick. The first element indicates the type of operation to be performed; -8 means "set border width and colour". (In this case, the minus sign is a high-minus and so the number is negative 8. More about minus-sign handling later). The other two elements here specify a border thickness of 15 with a border colour of 2. The effect was to thicken the screen window border sufficiently to bring its subsequent contents clearly into view.

QL/APL displays what you type in red and all its responses in green - very useful unless your TV is shot. It was simple enough to set the output colour to blue so that it could be seen more clearly, by typing

```
#CC -2 5
```

QL/APL had passed its first test (with flying colours). The ease with which the two problems were fixed was impressive, and result from a good integration of the QDOS operating system's calls with APL. By the way, if you can afford a monitor then problems with recalcitrant TVs are avoided and you'll get a readable 80 columns rather than 40. (You can use a TV in 80 column mode, but it's harder to read the screen). The picture quality is of course much better, and you can also exploit the QL's high resolution graphics mode.

QL/APL features

QL/APL supports all the functions, operators, system variables, system functions, system commands and function editing mechanisms required by the draft ISO APL standard i.e. the ones you'd expect to see in VS APL. In addition it also supports the following features:

- an APL statement separator (&)
- ambivalent user-defined function calling
- a simple single-user component-filing system
- a variable and function overlay facility
- an error-trapping facility
- "if" and picture format as primitive functions
- silent versions of)COPY,)DROP,)PCOPY,)SAVE and)WSID
- application of the execute function to system commands
- a shared-variable processor (although no auxiliary processors are supplied).
- many extra system variables and functions, including:

#A	the alphabet from A-Z
#BOX	vector-matrix interconversion
#CC	console control
#CONF	APL configuration function
#D	the digits 0-9
#DBR	delimited blank removal
#DR	data representation report and conversion
#ERS, #ERM,	
#ERX and #LER	error handling
#HC	hard copy function
#M	English month names
#MOUNT	assign devices to logical units
#SI	SI stack as a character vector
#SS	string search and replacement
#TT	terminal type
#W	English weekday names

QL/APL does not currently support the following facilities:

- the quadFMT formatting system function
- a full-screen object editor
- a Superbasic command interface
- groups (as in)GROUP etc.)
- an alternate alphabet

The filing system

The whole of the component filing system is controlled via three primitive functions: fput, fget and fdrop. As far as APL is concerned files have numbers, not names. To write an array X to component 3 of file 27 you would use

```
X fput 27 3
```

Component numbers can be sparse and if the file doesn't exist when you do an fput, then it's created. Retrieve the data by coding

```
X is fget 27 3
```

and drop the component by

```
fdrop 27 3
```

```
1
```

When the last component of the file is dropped, the file no longer exists. You can discover the numbers of the extant components on a file by entering

```
5 fget 27
```

```
1 2 4 5 6 7 11
```

and obtain a vector of file numbers by

```
5 fget 0
27 99
```

A logical unit number may also be specified with each of these three primitives, in order to gain access to the second microdrive or other storage media such as floppy or hard disks.

Sound and vision

All QL/APL's control over the console is achieved via the $\#CC$ system function, which provides about seventy distinct operations. These can be broadly grouped into:

- arbitrary I/O
- terminal control; cursor positioning, character translation, character attributes.
- channel management; open, attach and close channels.
- sound control
- graphics (the largest group)

In many cases the intrinsic power of the QL is made available to APL by direct QDOS calls from $\#CC$. For example, the QL's graphics facilities can be invoked by use of $\#CC$. Points, lines arcs and ellipses (generalised circles!) can all be directly plotted in both relative and absolute coordinate systems. Flood mode can be turned on to cause subsequently drawn graphic objects to be automatically filled with the desired colour and filled rectangles may also be drawn. In high resolution mode 8 distinct solid colours are available, with blended colours, or stipples as they are called, making the total up to 255. As an example, an ellipse can be drawn by

```
 $\#CC$  -204 50 60 1500 15 100
```

The -204 tells $\#CC$ that an ellipse is to be drawn using the absolute graphics coordinate system. The x and y coordinates are 50 and 60, with the eccentricity of the ellipse 1500 in this case. (An eccentricity of 1000 will produce a circle). The radius is 15 graphics scale units and the rotation of the ellipse is 100 milliradians.

The syntax chosen for CC does not allow for a matrix to be passed in, and so a whole set of ellipses (say) cannot be drawn at once in this way. However the right argument is allowed to be a vector of CC calls catenated together:

```
 $\#CC$  -2 3 -24 70 80 -222 90 100
```

Here the -2 sets the output colour to 3, the -24 sets the graphics cursor to graphics coordinates 70 80 and the -222 draws a line from the graphics cursor to the absolute point at 90 100.

Unfortunately the right argument to $\#CC$ is restricted to 50 elements, and therefore no more than 8 ellipses, for instance, can be drawn at once. It is fairly easy however, to write a cover function to $\#CC$ that supplies its own argument to $\#CC$ in no more than 50 element chunks. A relaxation of this restriction would apparently result in a smaller clear workspace size, and at 28K for a standard QL this is probably not a good idea. For users with a memory expansion board however, the ability to specify their own $\#CC$ size limit at APL load time might be a bonus.

Keywords and symbols

If there are any controversial aspects of QL/APL, then the choice of keywords and the dual use of the minus sign must be the two main ones. The reasons behind these decisions are documented in David Eastwood's paper "QL/APL - A new approach to popularizing APL" which appears later in the technical section of this issue of VECTOR. Because QL/APL aims to convert Basic users rather than those who already know APL, the choice for a particular keyword is not based on the APL name for the corresponding symbol. Thus the keyword "size" is used rather than "rho", which would have little meaning to a non-APLer. Additionally, no name distinction is made between a keyword that represents a monadic or a dyadic function call. So "size" is used both to discover the shape of an array as well as to reshape an array. This is entirely consistent with user-defined functions, where the function name is the same whether we invoke the function monadically or dyadically. This scheme leads to only 47 reserved names, but can be confusing. It is difficult for example, to explain to a non-APL user why it is necessary to type

V is rotate W

when we actually want to *reverse* the order of elements in W. (Actually this is also a mistake made in speech by even the most experience APLers - it is a subject which can lead to much philosophical speculation about whether symbols or functions are being named). Anyway, as an APLer I found the scheme very easy to use, and rarely had to refer to the manual to look up a keyword, the obvious choice usually being correct.

As far as the choice of symbols goes, most of them are obvious, with +, -, * and / retaining their meanings as in Basic. The symbol = has no meaning in QL/APL because the keyword "eq" is used instead, to be consistent with the use of keywords for all the other relational functions. The angled brackets <> replace the square brackets [] throughout, i.e. in indexing, axis or drive specification and in function editing requests. The function editor is different in other ways too, with the comma replacing the quad character in all display requests. Thus a request to open definition for a function FOO, display it and close the definition becomes

```
defn FOO<,>defn
```

A line can be deleted from a function by issuing, for example

```
<-17>
```

As there is only one type of minus sign in QL/APL, a modification to the syntax analyser was made in order to determine the appropriate meaning according to context. So the sequence space/minus-sign/digit as in

X is 3 -2 9

indicates the use of minus as a high-minus, or negative marker, whereas its use in subtraction can be seen in

```
-3      5-8
```

In the expression

```
X-1
```

however, the minus is taken as a negative marker and a syntax error is produced, which is worrying. A minus sign can be forced to mean subtraction by always following it by a space, as in

X- 1

which works fine. This convention will surely lead to problems when code is transferred between QL/APL and other APLs.

Performance

This is always a tricky one, because it depends on what you measure. Casually, QL/APL "feels" fast, but how does it compare with other APLs? Well, there are no definitive benchmarks tests that I know of, and those that have been published are usually woefully incomplete - either because they ignore important areas, overemphasise less relevant ones or just fail to test the sort of demands placed on an interpreter in a real application. Anyhow, time seemingly never permits the design of a good set of benches and so we'll have to make do with what we've already got. In this case what we've got is a set of benchmarks comparing IBM's APL and APL * PLUS on the IBM PC, published in BYTE magazine in March 1984. These tests were additionally timed using APL.68000 on a MicroAPL Spectrum and on the QL. The results are summarised in Table 1, with BYTE's original PC timings left in for comparison. Considering only these benchmarks, QL/APL comes out at about 60% as fast as APL.68000 on a Spectrum and 35% as fast as APL on the PC, although these figures should be taken with a pinch of salt as the tests are by no means exhaustive.

When trying to assess the speed of component filling onto microdrive cartridges, things start to get a little nasty, with the appearance of apparently non-deterministic behaviour. After creating a one thousand element vector of random integers by

A is rand 100 size 100

the following expression was timed:

A fput 1 1

After ten trials, the shortest time was 30 seconds and the longest 55 seconds (the first fput), with an average of 36 seconds. Presumably the random position of the read/write head over the tape loop is causing this variation. The time to write the vector to ten successive components on file was 8 minutes when I tried it, with a reduction to 40 seconds when empty vectors were written instead. The time to read the 1000 element vector timed in at a significantly lower 8 seconds over twenty trials. These tests were performed using a newly formatted cartridge, and I have no indications of timings with nearly full cartridges.

Table 1

	IBM PC	IBM PC	SPECTRUM	QL
	-----	-----	-----	---
CHIP :	8088	8088	68000	68008
APL :	IBM APL	APL*PLUS	APL.68000	QL/APL(K)
VERSION:	1.00	2.6	4.10/B	1.03
BENCHMARKS				
1. Z++/VI	90	102	26	54
2. Z+v/VL	0.4	3	6	16
3. Z+[/[1]MI	40	25	20	42
4. Z+VI*.1	390	282	4312	6824
5. Z+ VR	80	79	87	169
6. Z+VR[VI[120]]	20	14	23	51
7. Z+VI[VVI]	600	112	190	325
8. Z+2 1+MR	9	24	12	27
9. Z+VIV	150	146	217	202
10. Z+2 1+MC	450	60	188	355
11. Z+VC+.=VC	360	141	158	363
12. Z+(150)*.+.150	2530	439	241	535
13. Z+VR[. +VR	210	341	192	340
14. Z+MR[10+VR	70	1488	340	559
15. Z+FIB	2200	3827	1654	3250
16. Z+VR*3.14	100	136	229	396
17. Z+VR+3.14	110	142	389	541
18. Z+VR	150	143	2428	3807
19. Z+10VR*.1	411	438	3462	4559

Note 1: Each benchmark was run 100 times in succession. The times are in milliseconds and are adjusted to represent one execution of the given expression (net of looping overhead).

Note 2: The figures for the PC benchmarks were taken directly from the March 1984 issue of BYTE, page 248. Since that time two major updates to the APL*plus/PC product have been announced, and version 4.1 shows significant speed-ups over 2.6 in many areas.

Note 3: The variables used in the benchmarking were created by
 MI+10 10 pVI+(500p 0 1 0 0 1)/1500
 VL+1 0 1 1 0 0 0 1
 MR+10 10 pVR+VI+0.1
 MC+26 26 pVC+'ABCDEFGHJKLMNOQRSTUVWXYZ'

Note 4: The IBM PC was running with the 8087 maths coprocessor installed and enabled. The MicroAPL Spectrum was running in single-user mode.

Table 1: 19 benchmarks for IBM PC APL, APL*plus/PC, APL.68000/Spectrum and keyword QL/APL

Behaviour and "feel"

An important part of how popular a new interpreter becomes is its general overall behaviour. QL/APL feels robust and responsive, and would appear to behave itself in most cases. Many of the problems I experienced were a direct result of my own APL knowledge - e.g. it was extremely difficult to remember to use the keyword "on" in place of / for doing compressions. This is particularly unfortunate because the resulting division is often not detected until some time later! Incorrect use of the minus-sign has also plagued me somewhat.

The names of user-defined functions must be chosen to be different from those of the 47 primitives or a defn error will result - a problem if you need to transfer code to a QL from another APL interpreter.

A very frustrating drawback of the version I used was the inability to run Superbasic commands from within APL. In particular, the need to format a microdrive cartridge from APL. This would be wonderfully useful when you've just created a workspace only to discover you haven't enough space left on a formatted cartridge.

An interesting quirk of the system command)OFF is that it clears the active workspace, but leaves you in APL, forcing you to press the RESET button to return to Superbasic ("Surely you don't really want to leave APL do you?").

Some philosophy

The use of keywords (and the space that usually precedes and succeeds them) results in function lines that suddenly become very long, especially if you're working in 40 columns rather than 80. Although I made the shift to keywords quite easily, I found that program development and debugging became much harder, my productivity certainly being reduced. The code would always be written in symbolic APL and then mentally converted to keyword form on input. (Using a keyboard that feels like you're typing in cold porridge doesn't help either). Spotting idioms in keyword APL becomes more difficult; how long would it take you to recognise the following idiom?

N is((V index V)eq index size V)on V

It may be possible that such idioms take longer to arise in the mind of a keyworder than in that of a symbol person. Alternatively it may be that QL/APL will attract users who are more "wordy" than symbolic and who have none of the problems we might experience as traditional APLers. If QL/APL can be used in a classroom environment, we might even expect to see a higher proportion of "right-brain" people becoming interested, e.g. those with a greater capacity for synthesis and word-skills rather than analysis and symbol-skills. Although many will buy a QL to run APL on, the important step as far as APL is concerned is in convincing people to buy APL for a QL they've already got; let's hope the 100 pound price tag won't scare them off.

One final word of warning to those who might think that QL/APL will permanently silence APL's detractors: a friend of mine with no computer knowledge came into the room where I was using keyword QL/APL and remarked, after taking a look at the screen "Well, its all Greek to me".

MOVING DATA FROM 1-2-3 to APL

by Michael Carmichael

Summary

This article describes methods of transferring data from a LOTUS 1-2-3 worksheet into an APL*PLUS/PC workspace. The merits of reading directly versus using an intermediate DOS file are examined.

Introduction

Why move data from 1-2-3 to APL? Are there signs of mass defection from LOTUS 1-2-3 to APL*PLUS/PC? Certainly not, but there is a need for APL and 1-2-3 programs to talk to each other. Both are successful PC products in their own right with solid reputations for speedy program development and flexibility. There is clearly a lot of overlapping capability, but a view of the areas in which one or other product excels will clearly depend on a person's background. Typically a new PC user will be satisfied with the scope of 1-2-3 whereas a serious programmer may demand the extra power of a product such as APL*PLUS/PC.

Would a "hybrid" - an application written in both APL and 1-2-3 offer the best of both worlds? Consider the hypothetical example of a company budgeting system. The data flow might be:

1-2-3 → APL → 1-2-3 → APL → 1-2-3

- Skeleton input forms are designed and saved in 1-2-3
- APL reads the forms and customises them according to differing departmental requirements.
- Users enter the figures onto the prepared 1-2-3 worksheets. "Local" calculations and comments can be appended.
- APL is used to validate, consolidate, analyse and report.
- The 1-2-3 graphics facility is used to prepare bar-graphs of key totals on a pen-plotter.

APL is the "glue" which binds together the 1-2-3 parts of the application and adds advanced analysis and reporting. Now what are the advantages?

- Flexibility - development can evolve according to the available mix of user/programmer time and expertise.
- Shared data - greater accuracy and standardisation, less misplaced effort.
- Maintenance - The 1-2-3 User's Manual will be adequate documentation for many parts of the system! Minor changes and new features can be added to the application without the direct involvement of the APL programmer.

And where will the line be drawn? Why insist that a simple - tedious even - part of the application be written in APL? Summing rows and columns with a few deft keystrokes is no longer a feature exclusive to APL, and most 1-2-3 users will be quite capable of writing programs of the "Add/Change/Delete/Print" variety themselves.

So assuming there is role for both PC products how easily can data be passed between them?

Foreign Correspondence

The heading of this section is taken from the chapter in the 1-2-3 User's Manual which describes how 1-2-3 and other programs may pass data to and fro. For example 1-2-3 has a TRANSLATE facility enabling a worksheet (file extension .WKS) to be converted to a DIF standard file and vice-versa.

But for passing data to and from APL an obvious route is via an intermediate DOS file.

APL to 1-2-3:

```
workspace -----> DOS file -----> worksheet
      NWRITE                      FILE IMPORT
```

1-2-3 to APL:

```
worksheet -----> DOS file -----> workspace
      /PRINT FILE                      /NREAD
```

To move data from APL to 1-2-3 write an "unformatted" report to a DOS file with the APL *Plus/PC system function NWRITE ("native file write"). The data should be in distinct columns with double quotes around text fields. This ensures that from 1-2-3 a /FILE/IMPORT/NUMBERS command will put each item into a separate cell.

From 1-2-3 the /PRINT/FILE command prints a specified range to a DOS file (with file extension .PRN) instead of to a printer. The system function NREAD can then bring the data into a workspace as character data.

There is a second approach to moving data from 1-2-3 to APL: one can attempt to read the data directly. Obviously this requires a basic knowledge of how a worksheet is stored.

Let it be stated that our objective is limited solely to moving "displayed" 1-2-3 data. A column of "values" is to become a vector of numbers; a column of "labels" is to become a character matrix. Just as a displayed number in APL may conceal its type and rank so a value in 1-2-3 may conceal a wealth of underlying information in the cell. But here no attempt will be made to carry over that information stored "beneath" the worksheet - i.e formulae, range-names etc.

In what follows the two chosen approaches

- reading via a DOS file
- reading the worksheet directly

will be explored in some detail.

Reading via a DOS file

In common with other PC packages 1-2-3 has the option of directing a "print" to disc rather than to a printer. The .PRN file so produced is a straightforward ASCII text file; there is a CR,LF pair at the end of each line and a right-arrow character (DEC 26) marking the end-of-file, followed by a number of trailing nulls.

To send a portion of worksheet data to a printer just set the range and GO! Similarly to send data to a file the sequence of commands is

/PRINT/FILE/Enter Filename/Set RANGE/GO/QUIT

Because the goal is to move data, it is a good idea to first of all select the print option "unformatted" and to set the page margins to zero. By default 1-2-3 prints have a left margin of four spaces.

A "macro" in 1-2-3 is a sequence of keystrokes that the user can store on the worksheet as an executable procedure.

The following macro will write the data from the worksheet NUMS.WKS shown in Fig 1 to a DOS file named NUMS.PRN. (Note that $\leftarrow$ represents the ENTER key.)

```
/p f NUMS ← A2..E5 ← g q
```

Who says cryptic one-liners are exclusive to APL!

Fig. 1

	A	B	C	D	E	F	G	H
1								
2	1234	5678	9012	3456	7890			
3	-2	-1	0	1	2			
4	35	46	57	68	79			
5	80	91	102	113	124			
6								
7								
8								
9								

EXAMPLE 1 demonstrates reading this file NUMS.PRN into the APL workspace. In this particular case rebuilding the data is very easy because there is a fixed number of characters per line.

Please note that the examples are presented as an APL "session" in Appendix 1. The APL functions that are called are listed alphabetically in Appendix 2. For instance EXAMPLE 1 uses the function <nREAD>. This is a cover function for the NREAD system function and is also used in other examples.

EXAMPLE 2 is another demonstration of reading a .PRN file, this time one containing the data from the worksheet CURR.WKS shown in Fig 2. A little sophistication has been added to the new 1-2-3 macro. Fig 3 lists this "file-save" macro which is stored along with the data on the CURR.WKS worksheet. When the user presses the alt-F key both .WKS and .PRN files will be updated. Use of the keys {end} {down} {end} {right} ensures the range is extended to include any newly entered data. (Note that the macro will fail the first time it is run because the DOS file CURR.PRN will not exist!)

Fig. 2

	A	B	C	D	E	F
1	REF	CODE	NAME	LIMIT		
2	803	FREF	French Franc	25		
3	804	GERM	German Mark	45		
4	806	UKPD	United Kingdom Pound	55		
5	808	HOLG	Dutch Guilder	15		
6	811	ITAL	Italian Lire	15		
7						
8						
9						

Fig. 3

	AA	AB	AC	AD
	MACRO	ACTION	COMMENT	
1	-----	-----	-----	
2				
3				
4	\F	/fsCURR~r{esc}	/* overwrite WKS file	
5		/pfCURR~r	/* overwrite PRN file	
6		r{bs}	/* set print range	
7		{home} . {end} {down} {end} {right} ~	/* from A1 to end of data	
8		gq	/* go, quit	
9				

The function <prnREAD> calls <nREAD> and the make-into-matrix utility <MIM> which, using the LF character as a delimiter, returns the contents of the .PRN file as a character matrix. The looping function <prnSPLIT> "splits" the data vertically into APL variables <field1>, <field2>, etc. The single-word column headings in the first row are a device to supply the function with "field widths". Of course when a DOS file is used as an intermediate file the specific row, column and datatype information is lost.

How robust is the reading-via-a-DOS-file approach? New rows and columns can be added with confidence. Altering cell widths will not confound it provided there is no overlapping of the data. However inserting a blank row would cause a problem with the example given.

One advantage of a .PRN file is that being an ASCII text file it can be inspected and manipulated in DOS. It is "space efficient": a file such as CURR.PRN is substantially smaller than its progenitor CURR.WKS.

Reading the worksheet directly

Why mess about with macros and intermediate files if it is possible to grab the data directly off the worksheet?

Indeed it is possible, and in a special case such as all integers the solution is simple and elegant. The challenge is to find a speedy and reliable method that will read any worksheet.

The four methods that will be looked at are:

- a) BFI method
- b) Fixed Length Records
- c) String Searching
- d) Worksheet Analysis

a) BFI method

This is the celebrated "brute force and ignorance" method.

If in ignorance the PC-DOS command TYPE B:ALLTXT.WKS is executed it is possible to see bits of the text data from the worksheet. But the accompanying "beeps" and "flashes" due to the control characters are very distracting! If in APL the control characters are replaced before display, the whole worksheet can then be studied as an informative "wallpaper pattern".

A .WKS file always starts with a header sequence DEC 0 0 2 0 4 4 (ASCII DECimal codes, ie the zero-origin quadAV index). The cell data, usually only small proportion of the total worksheet, is seen at the end of the file in cell-by-cell "ravel order". Once again the file ends with a right-arrow (DEC 26) and a variable number of nulls.

EXAMPLE 3 shows what the last part of the worksheet ALLTXT.WKS of Fig 4 looks like. Each item of text ends with a null and starts with an apostrophe (the "label prefix" of 1-2-3). The nine preceding control bytes start with the "circle-star" character.

Fig. 4

	A	B	C	D	E	F	G	H
1								
2		NORK	Norwegian Kroner					
3		SPAP	Spanish Pesata					
4		KUWD	Kuwaiti Dinar					
5								
6								
7								
8								
9								

What happens if instead of replacing offending characters we just remove them? In EXAMPLE 4 the worksheet ALLTXT.WKS is read, any members of the first 32 characters of quadAV suppressed, and a matrix is then made up of what remains using the apostrophe as a delimiter.

The result of this brutality is surprisingly effective. But it will only work satisfactorily for small worksheets (<32 rows and columns) with text strings less than 32 characters in length. And beware! The apostrophe is the "label alignment prefix" of 1-2-3 which the user is at liberty to change (eg to " or \).

Predictably the BFI method is less successful reading numbers. EXAMPLE 5 shows a bold attempt to read the worksheet NUMS.WKS of Fig 1. The crude APL expression only manages to salvage numbers in the range 31 to 255!

At this point I was lucky to come across the article (REF 1) by William Sharp explaining the basics of 1-2-3 record structure.

A worksheet is a collection of variable length records with the general record structure that shown in Fig 5. The first pair of bytes in the record is the "record type". There are many record types in 1-2-3 but our interest is confined to the four types shown in Fig 6: integer, character, floating point and formula. For example the "circle-star" seen in EXAMPLE 3 indicates a character record type.

Fig. 5

General Structure:

record	-type	record	length	format	column	number	row	number	data	etc
--------	-------	--------	--------	--------	--------	--------	-----	--------	------	-----

Example: the text "APL" in the cell at row 4, column 3

15	0	10	0	255	2	0	3	0	39	65	80	76	0
----	---	----	---	-----	---	---	---	---	----	----	----	----	---

Fig. 6

TYPE		LENGTH	
13	integer	11	16 bit two's complement
14	floating point	17	64 bit IEEE double precision
15	character	variable	label-prefix, text, null
16	formula	variable	64 bit , formula description

The second two bytes of a record give the record length and point to the start of the following record. The record length is fixed for numeric data but of course variable for text.

The fifth byte is the "format byte" which dictates how the cell contents are displayed on the worksheet (and also the cell's "protection" status). The default format is DEC 255 which actually explains the appearance of this number in EXAMPLE 5 above. Bytes 6 7 8 9 are the column and row numbers.

The remaining three methods of this section are the fruits of this new knowledge.

b) Fixed Length Records

In the last example brute force failed to decypher the numbers in the file NUMS.WKS. A more scientific approach is now adopted.

Numbers that are directly entered into the cells of a worksheet are stored as fixed length records, and fortunately 1-2-3 employs the same internal representation used by APL*Plus/PC.

An integer record is eleven bytes long, the last two bytes being a two's complement representation of the value. Thus the procedure for reading say twenty integers is:

- take the last 20 data records from the file
- reshape them into a 20 by 11 character matrix
- do a "163 quadDR" on the last two columns

(In APL*PLUS/PC quadDR is the "datatype conversion" system function.)

The function <wksNUMREAD> is a version of this procedure but generalised so it can handle other sorts of 1-2-3 fixed length records too. EXAMPLE 6 is the second attempt to move the integers from NUMS.WKS into the APL workspace. This time when suitable parameters are entered as a left argument to <wksNUMREAD> the function returns the correct results.

EXAMPLE 7 demonstrates that with a different left argument the same function can handle floating point numbers. A floating point number in 1-2-3 is seventeen bytes long (eight bytes for the value), and the APL*plus/PC datatype is 645. Note that the numbers are moved from 1-2-3 to APL without any loss of precision.

Fig. 7

	A	B	C	D	E	F
1						
2		Nose :	41,234,567.1249341			
3		Wing :	14,857,263.6857719			
4		Fin :	51,845,736.5877143			
5		Tail :	34,968,756.2537234			
6						
7						
8						
9						

One can get away with inserting labels on such a worksheet provided the length of the text string is chosen to exactly match the record length of the numbers. String lengths of 11, 22, 33 etc for integer and 5, 22, 39 etc for floating point are acceptable. <wksNUMREAD> is designed to ignore those records types not specified as part of the left argument.

The worksheet RATES.WKS of Fig 8 is a more realistic example featuring floating point numbers. It also illustrates a problem. In contrast to the other rates on the worksheet the entry in the last row is 2065 - a whole number - which 1-2-3 in its wisdom stores as an integer. This is very annoying. If only the entry were say 2065.5 then each row of the worksheet would conveniently take up the same number of bytes.

Fig. 8

	A	B	C	D	E	F	G	H	I	J
1										
2	803	FREF	10.198							
3	804	GERM	3.337							
4	806	UKPD	1.09							
5	808	HOLG	3.78							
6	811	ITAL	2065							
7										
8				+C2	+C3	+C4	+C5	+C6		
9										

The solution is to employ 1-2-3's formula datatype. In this type of record even integers are stored in the floating point format. A formula record consists of the current value followed by a variable length string representing the formula itself. A formula such as "+ C2" takes up a total of 25 bytes.

A string of such formulae at the bottom of the worksheet means that values entered in the rates column will automatically be placed at the end of the .WKS file as well and in a pattern of fixed length records. These items can then be retrieved by applying <wksNUMREAD> with a suitable left argument. EXAMPLE 8 demonstrates the effectiveness of this technique. Note that the formulae +C2 + C3 + C4 etc should be keyed in directly; if the /COPY command is used here the fixed record length requirement will probably not be met.

By the way, LOTUS SYMPHONY features "string arithmetic" which could extend this avenue. For instance one is able to write a formula which will format a number, join it to a piece of text and then trim the result to a fixed length.

c) String Searching

A very different approach to unscrambling a .WKS file is to specifically hunt for character sequences using quadSS. This APL *Plus/PC string search function might not be pure APL but it is fast!

The first application of quadSS is in the worksheet reading function <wksREAD> used to return only the "displayed data" section of the .WKS file. The end of the worksheet is always signalled by the sequence DEC 1 0 0 0 26, and so far as I know (not guaranteed!), the displayed data section always starts with DEC 113 113 1 (look for the pair of lowercase q's in EXAMPLE 3). If by a very unlucky coincidence one or other pattern coincides with part of the representation of a floating point number on the worksheet the function will of course fail!

An underhand but effective technique is to put "markers" into the worksheet. For example an important data entry cell could be preceded by the sequence colon, space, dash. When the file is read into the workspace quadSS searches for this particular sequence. The user's entry is then to be found nine bytes along (and terminated with a null if text).

Intrinsically (that means without cheating by using markers!) the worksheet has some interesting sequences. Look again at the example of a character record in Fig 5 and consider the problem of flagging the occurrence of text in a worksheet. Searching for DEC 15 0 will indeed find the start of the character records but most probably a lot of junk as well (such as row and column numbers). It would be more reliable to search for DEC 0 39 (the apostrophe) or better still DEC 0 255 (the format byte)

In EXAMPLE 9 the function <wksSPLIT> is used to process the result of <wksREAD> reading the worksheet CURR.wks of Fig 2. The integer and character data is separated into the two variables <zint> and <zchr>. The function <wksSPLIT> first extracts all the integers by searching for the string DEC 13 0 7 0 255.

Provided a worksheet contains only text and integers this function will be reliable although there are some unusual circumstances which will make it fail. Entering the value 256 in row 14, column 18 is one of them!

d) Worksheet Analysis

The above examples have all placed restrictions on one or more of the following: record type, label prefix, format byte, size of worksheet. Such assumptions might be acceptable when say an APL programmer designs a worksheet for the entry of data tables. But in general the 1-2-3 worksheet will have a complex layout containing many record types. So far we do not have a method which will help answer questions such as:

- Does row 35 contain only numbers?
- Does the word "TOTAL" appear anywhere in column 5?

The general case requires a "looping" solution, a record by record analysis of the worksheet. The final example EXAMPLE 10 will demonstrate reading all the items in the worksheet RATES.WKS of Fig 8. The APL functions featured are now described.

The function <wksOPEN>, which takes the name of the worksheet as its right argument, creates two variables <wksdata> and <wksindx>. (The function <wksclose> expunges them!). <wksdata> is the raw contents of the file returned by the function <wksREAD> used in the previous example. <wksindx> is the "worksheet index" a numeric matrix, each row of which summarises a record on the worksheet. <wksindx> is returned by the function <wksINDEX> which starting at the top of the data loops around record by record. Every record contains a pointer to the start of the following one, and as the function proceeds it builds up a six column matrix:

- Col 1 - index of the start of the record
- Col 2 - length of the record (actually less 4 bytes)
- Col 3 - 1-2-3 record type (eg 13 14 15 16 etc)
- Col 4 - row number (in zero origin)
- Col 5 - reserved for row number - second byte
- Col 6 - column number (in zero origin)

Building the "worksheet index" can be quite slow but once complete the hard work is over. Queries about the worksheet data such as:

- show all the text in column 2
- show all formula data in row 8

can now be satisfied without further looping. Functions can now easily be written which use the index; they can rapidly access <wksdata> according to criteria involving record type and row and column numbers.

In EXAMPLE 10 the function <WKSGET>, which takes a record type left argument and a row or column number right argument, is shown in action. A fragment of the worksheet index is also given.

Worksheet analysis opens up further possibilities. As suggested in the introduction, the ambitious can use APL to make changes and additions to a 1-2-3 worksheet. We will not dwell on this particular aspect!

Performance

How quickly can data be moved from 1-2-3 to APL? How long does it take to read the file from the disc and convert the data into APL variables?

Fig. 9

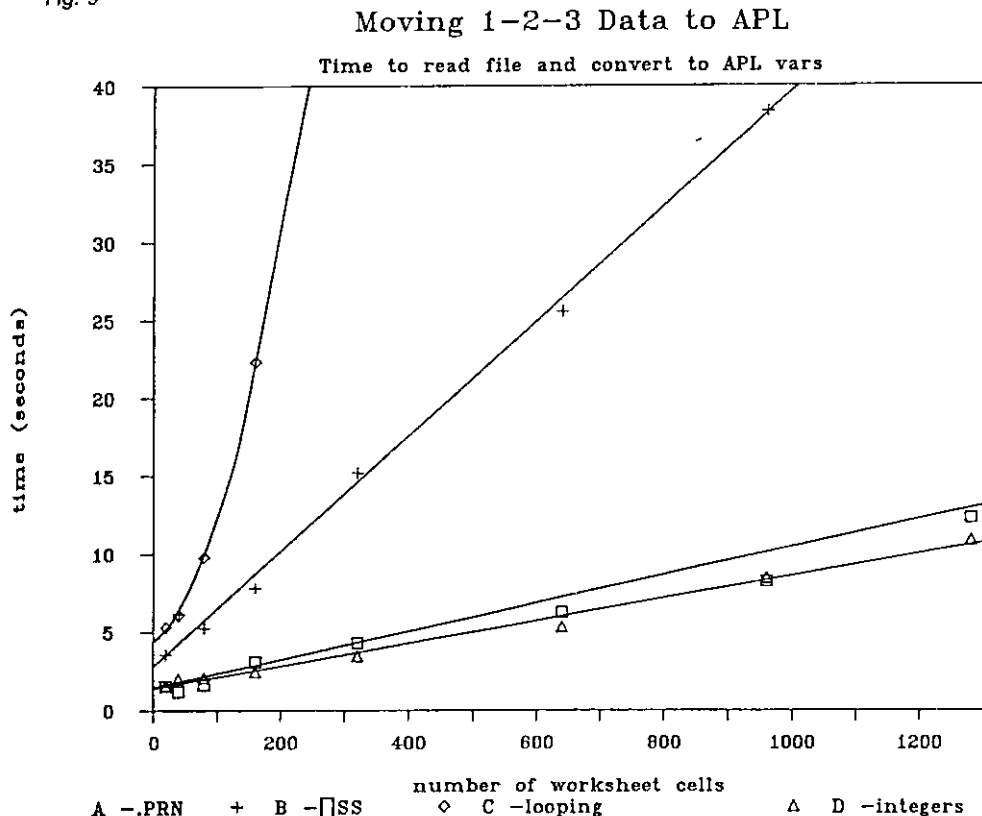


Fig 9 shows the results of experiments using the worksheet CURR.WKS of Fig 2 as a standard. Versions of this worksheet with 20,40,80,160,320,640,940,1280 cells were constructed using the 1-2-3 /COPY command. For each reading the minimum times for five runs was recorded.

- A reading via an intermediate DOS file using the functions <prnSPLIT> and <prnREAD>
- B string searching using the functions <wksSPLIT> and <wksREAD>
- C worksheet analysis - the total time to run <wksOPEN> plus <wksGET> operating appropriately on the four data columns.
- D integers - for reference this graph records the performance of <wksNUMREAD> moving data from versions of the worksheet NUMS.WKS of Fig 1.

The graphs are only meant to serve as a rough guide. It is misleading to give precise figures because the following items are arbitrary or not defined:

- the worksheet environment
- the file size, layout and disc drive performance
- the state of the APL workspace
- the choice of test data (eg length of text strings)
- "processor stall"

The tests were run on an IBM PC with standard dual floppy disc drives and fitted with 512k memory, an APL ROM and 8087 FP co-processor.

The worksheets were written with LOTUS 1-2-3 Release 3.1 ; the worksheets were read with APL *Plus/PC Version 3.1.

Conclusion

It has been shown that data can reliably be moved directly from a 1-2-3 worksheet into an APL workspace. The process is effective but rather inefficient. Two reasons for this are that only a small proportion of the data read into the workspace is of interest, and the worksheet analysis method is necessarily a looping solution and therefore not very fast.

Which method is best?

- For extracting a few key values from a large and complex worksheet the "formula" technique (placing copies on the bottom of the .WKS file) is attractive. It is discreet because it does not require a macro or impose restrictions on the 1-2-3 user.
- For reading large amounts of data the intermediate DOS file is the obvious choice for reasons of speed. It can be argued that some of the time saved when reading in the data is at the expense of the extra time (because of the additional file) to save in 1-2-3.
- For small amounts of data reading the worksheet directly is recommended. If at all possible the fixed length record or string searching method should be applied, but backed up by worksheet analysis, if only to provide helpful error messages when a problem is detected.

Deciding on the right method, indeed putting up a case for implementing a joint APL/1-2-3 application depends on other factors besides the size and complexity of the worksheet. How many programs will share data and how frequently will they be run? How often will the data be updated?

To conclude here is a message for all APL*PLUS/PC users who are also Lotus 1-2-3 enthusiasts. If the two products are co-habiting on your PC let them talk to each other like good friends.

Reference

William Sharpe, *Learning Lotus File Forms*, PC Magazine (UK Edition), Vol 1 Issue 7, November 1984, published by VNU BV, London.

APL CONSULTANTS London & Reading

Do you have experience in

- * Decision Support Applications**
- * Technical Services**
- * Marketing Support**

We are looking for experienced APL Consultants to join our Applications Development team and our newly formed Technical and Marketing Support groups.

You will be working with IBM PC's, AT's, and other multi-user systems, using VS APL, APL2, Unix, and APL*PLUS.

Of course, as Europe's leading APL consultancy, we offer first class benefits - excellent salaries, profit sharing, free health insurance, and a non-contributory pension.

If you have at least a year's APL experience, send us your CV or ring Brian Drury on 01-493 6172.



COCKING & DRURY LTD.
THE APL PROFESSIONALS

16 Berkeley Street, London W1X 5AE

Appendix 1 — Sample APL session

AA Example 1

AA -----

A reading a .PRN file ; constant of characters per line

pX←0 nREAD'K:NUMS.PRN'

256

+W+(5×9)+2-1 A CR,LF less one trailing blank on last column

46

0 -2+(4,W)pX

1234	5678	9012	3456	7890
-2	-1	0	1	2
35	46	57	68	79
80	91	102	113	124

AA Example 2

AA -----

A reading a .PRN file; a more general case

pX← prnREAD'K:CURRP.PRN'

6 38

prnSPLIT X

4

Q5 2p±.field1.field4

803	804	806	808	811
25	45	55	15	15

field2,'|',field3

FREF		French Franc
GERM		German Mark
UKPD		United Kingdom Pound
HOLG		Dutch Guilder
ITAL		Italian Lire

AA Example_5
AA

A the BFI method on integer data - almost a total failure!

X+0 nREAD'K:NUMS.WKS'

4 5 p⁻²⁰+1+AV₁(X₃₂+AV)/X

255	35	255	46	255
57	255	68	255	79
255	80	255	91	255
102	255	113	255	124

AA Example_6
AA

A reading integers directly

4 5 p 20 13 11 2 163 wksNUMREAD'K:NUMS.WKS'

1234	5678	9012	3456	7890
-2	-1	0	1	2
35	46	57	68	79
80	91	102	113	124

A thats's better!

AA Example_7
AA

A reading floating point numbers directly

'F20.7' FMT 10 14 17 8 645 wksNUMREAD 'K:FLOAT.WKS'

41234567.1249341
14857263.6857719
51845736.5877143
34968756.2537234

AA Example 8

AA

A reading 'formula' data

5 16 25 8 645 wksNUMREAD'K:RATES.WKS'

10.198 3.337 1.09 3.78 2065

A from the cells containing +C2 +C3 +C4 +C5 +C6

AA Example 9

AA

A reading a mixture of text and integers

pX+0 wksREAD'K:CURR.WKS' ◇ wksSPLIT X

368

zint A here are the numbers

803 25 804 45 806 55 808 15 811 15

50+zchr A here is the text - some of it!

'REF'CODE'NAME'LIMIT'FREF'French Franc'GERM'German

X+(4 0+MIM zchr)[0 ^1+.+2×15;] ◇ X[1;;].'|',X[2;;]

French Franc	FREF
German Mark	GERM
United Kingdom Pound	UKPD
Dutch Guilder	HOLG
Italian Lire	ITAL

```
## Example 10
```

```
##
```

```
# worksheet analysis method
```

```
wksOPEN'K:RATES.WKS'
```

```
(pwksdata),pwksindx
```

```
334 20 6
```

```
wksindx[(16),20:] # part of the 'worksheet index'
```

1	7	13	1	0	0
12	11	15	1	0	1
27	13	14	1	0	2
44	7	13	2	0	0
55	11	15	2	0	1
70	13	14	2	0	2
310	21	16	7	0	7

```
# start/count/record type/row /-/column
```

```
.13 wksGET 1 # integers in col 1
803 804 806 808 811
```

```
.15 wksGET 2 # text in col 2
'FREF'GERM'UKPD'HOLG'ITAL
```

```
.14 wksGET 3 # floating point in col 3
10.198 3.337 1.09 3.78
```

```
.13 wksGET 3 # integers in col 3
2065
```

```
.16 wksGET 4 # formula data in col 4
10.198
```

```
.16 wksGET ~8 # formula data in row 8
10.198 3.337 1.09 3.78 2065
```

```
wksCLOSE # over and out!
```

Appendix 2 — APL function listing

```

V R←INDXGEN M;N;M1;M2;M3
[1] A index generator utility fn
[2] A M[;1] ↔ start ; M[;2] ↔ count
[3] A eg: M ↔ 3 2p 2 4, 8 3, 11 2 <=> R ↔ 2 3 4 5, 8 9 10, 11 12
[4] N←0[1/M[;2]
[5] M1←Q(N,1+pM)pM[;1]
[6] M2←Q(N,1+pM)pM[;2]
[7] M3←((1+pM),N)p-1+1N
[8] R←(,M2>M3)/,M1+M3
[9] A 8503081313 78
V

```

```

V R←MIM CV
[1] A convert delimited character vector to matrix
[2] A (calls APL*PLUS/PC fn <CSSMATML> from supplied ws <UTILITY>)
[3] R←CSSMATML CV
[4] A 8503072147 78
V

```

```

V R←A nREAD B;N
[1] A read DOS file <B>
[2] A A ↔ convert/start/count see APL*PLUS/PC doc P 3-209
[3] A if 0=pA then read whole file as char data
[4] B □NUNTIE -1 □ N+-3+A,82.(□NSIZE -1),0
[5] R←□NREAD -1,N □ □NUNTIE -1
[6] A 8503072140 78
V

```

```

V R←prnREAD A
[1] A read special print file <A> and return as a matrix
[2] R←0 nREAD A
[3] R←(“2+R1’+”)R A drop nulls after EOF marker
[4] R←MIM □TCNL.(“2+R1’+”)R A make into matrix
[5] R←0 1 R A remove □TCLF char
[6] A 8503121918 78
V

```

```

V wksCLOSE
[1] A expunge worksheet global vars
[2] 0 0 pEX 'wksdata wksindx'
[3] A 8501241139 78
V

V R←A wksGET B;BV;M
[1] A get record type <A> data from <B>
[2] A <B> is column number or record number if negative
[3] A globals: <wksdata> <wksindx>
[4] A eg: 13 wksGET 3 returns all integers in col 3
[5] BV←(wksindx[;3]=A)Awksindx[;6-2×B<0]=~1+|B
[6] +(a,b,c,b,c)[13 14 15 16 1A] A int/FP/char/formula
[7] a:R←163 [DR wksdata[(BV/wksindx[;1])+. 9 10] ◇ +0
[8] b:R←645 [DR wksdata[(BV/wksindx[;1])+. +8+18] ◇ +0
[9] c:M←BV/wksindx[; 1 2] ◇ R←wksdata[INDXGEN M+(pM)p 9 ~6] ◇ +0
[10] A 8503252018 78
V

V R←wksINDEX A;INDX;VEC6
[1] A compose index to worksheet data <A>
[2] A loop through record by record
[3] A R[;16] ↔ startposn,lengthofstring,recordtype,row,row[byte2],col
[4] A this version assumes above 'attributes' are all less than 256
[5] A <VEC6> contains 'attributes' for record starting at index <INDX>
[6] A
[7] R←' ' ◇ INDX+1 ◇ MAX+~7+pA
[8] a:
[9] +(INDX>MAX)/b
[10] R←R.VEC6+INDX.A[INDX+ 2 0 7 8 5]
[11] INDX←4+INDX+VEC6[2] ◇ +a
[12] b:
[13] R←(((pR)+6),6)pR
[14] A 8503252021 78
V

V R←A wksNUMREAD B
[1] A read numbers from WKS file <B>
[2] A A ↔ /noofrecs/123rectype/reclength/noofbytes/[DRdatarep
[3] A eg:
[4] A A ↔ 5 13 11 2 163 A integer
[5] A A ↔ 5 14 17 8 645 A FP numbers
[6] A A ↔ 5 16 25 8 645 A formula eg: 25 is length of +A1 type
[7] A [DR is described in APL*PLUS/PC sys doc P 3-54
[8] A
[9] B [NTIE ~1 ◇ R←[NREAD ~1 82 .[NSIZE ~1 ◇ [NUNTIE ~1 A read file
[10] A shape up last part of file
[11] R←[R ◇ R←(x/A[1 3])+(R[1]AV[2])R ◇ R←A[1 3]p[R
[12] R←(R[;1]=AV[1+A[2]])/R A compress out all but specified record type
[13] R←R[;9+1A[4]] ◇ R←A[5][DR,R A convert to numeric
[14] A 8503252022 78
V

```

```

V wksOPEN A
[1] A read worksheet <A> putting info into vars <wksdata> and <wksindx>
[2] wksdata←0 wksREAD A
[3] wksindx←wksINDEX 1+AV[wksdata]
[4] A 8503081339 78
V

```

```

V R←A wksREAD B;BEG;END;BV
[1] A read 1-2-3 worksheet (displayed) data
[2] A drop the worksheet status preamble
[3] B UNTIE 1 0 R←NREAD 1 82 ,NNSIZE 1 0 NUNUNTIE 1
[4] A if (0=pA) assume normal start of data mark qqU (no guarantee!)
[5] A otherwise <A> is text 'marker' placed in worksheet by user
[6] BEG←A.(0=pA)/AV[1+ 113 113 1]
[7] END←AV[1+ 1 0 0 26] A end of wks record
[8] BV←(R [SS BEG]v(R [SS END])
[9] A drop data outside the marked limits
[10] R←(1+pBEG)+(≠BV)/R
[11] A 8503252024 78
V

```

```

V wksSPLIT A;INDX;BV;R
[1] A non-looping analysis of wks <A>; splits data into integer and char vars
[2] A eg: wksSPLIT 0 wksREAD'B:CURR.WKS'
[3] A relies on AV[256] being the 'format' byte
[4] A fails if any formulae or floating-point present
[5] A char data into <zchr> ; and num data into <zint>
[6] R←A
[7] BV←R [SS AV[1+ 13 0 7 0 255] A mark integer records
[8] INDX←.(BV/1pR)←. 9 10 A extract 'value' indices
[9] zint←163 [BR R[INDX] A convert
[10] A
[11] R[INDX]←TCNUL A set integer values to null
[12] A set leader bits of char and int records to null
[13] INDX←.((R←AV[256])/1pR)←. 4 2 0 1 3 0 R[INDX]←TCNUL
[14] R←(R←TCNUL)/R A compress out nulls
[15] zchr←R
[16] A 8503252023 78
V

```

```

V R←prnsSPLIT M;S;N;T
[1] A columnwise split of 'table' <M> to variables <field1> etc
[2] A assume 1st row has one word col headings indicating field widths
[3] A return number of variables created
[4] N←1 0 S←' 'M[1;] 0 S←+S'140,S
[5] A S is now a series of ints indicating which col belongs to which field
[6] a:
[7] →b IF N>1+S
[8] T←1 0 +M[;(S=N)/1pS]
[9] S←'field',(N),'+T'
[10] N←N+1 0 →a
[11] b:
[12] R←N-1
[13] A 8503121920 78
V

```

A DIF FILE INTERFACE

by Romilly J Cocking

Abstract: Spreadsheet packages offer facilities that complement those available in APL. This article describes software that allows easy data transfer between the two environments. The conversion software is written in C, but can be invoked from the APL workspace in a manner transparent to the user.

Introduction

Most companies operate budgeting systems. In Cocking & Drury we have a 12-monthly operating plan, which we currently maintain using Symphony. The plan contains anticipated headcount, expenditure and revenue, divided into about forty sub-categories. We enter 'actual' data as it becomes available, and we produce variance reports, year-to-date reports, and consolidations that combine figures by department, location and subsidiary company. The data is not only used to monitor our performance against plan; we also use it to improve the accuracy and flexibility of the model from which our plan is derived.

While Symphony or Lotus 1-2-3 are ideal for data entry and for the preparation of simple reports (e.g. P & L for a single unit), APL is much better for the sort of data manipulation required for variance reports, consolidations and extrapolation.

The ideal solution would be to use each tool for the task it is best at. We needed an easy way of moving data from the spreadsheet environment into an APL workspace and back again; to our surprise, we found nothing available that met our requirements.

The structure of spreadsheet data

We wanted a method of moving data between a spreadsheet and the APL workspace. Although we found no suitable software, there is a standard file structure developed for this sort of requirement. Called the DIF file format, it was developed by Software Arts Inc (authors of VisiCalc). Most spreadsheets available today can export and import data in DIF format.

The file format was designed with BASIC and PASCAL programmers in mind, and DIF files are not very tractable in APL. It would be hard to write APL functions to handle DIF files efficiently, and the resulting code would be difficult to maintain and prone to WS-full.

A spreadsheet is just a table of data, though, and ought to be easy to handle in APL. The only problem lies in the fact that each spreadsheet cell may contain either a single (scalar) number or a text (character vector) label. Let us look at a simple example

	Jan 85	Feb 85	Mar 85	Total
Sales	89000	93000	117000	299000
Costs	73450	81830	95460	250740
Profit	15550	11170	21540	48260

Figure 1 - sample spreadsheet

We can divide this up into the cells which were set up in SYMPHONY. The result is shown in Figure 2.

	Jan 85	Feb 85	Mar 85	Total
Sales	89,000	93,000	117,000	299,000
Costs	73,450	81,830	95,460	250,740
Profit	15,550	11,170	21,540	48,260

Figure 2 - spreadsheet cells

If we were using a nested array system, such as Sharp APL, APL2 or APL *PLUS/1000, we could represent this structure as a matrix in which each element was either a number or an enclosed character vector. We wanted to use APL *PLUS/PC, so we needed to find a simple representation in 'conventional' APL.

We found a solution that has proved easy to understand, and straightforward to implement. The spreadsheet is represented by four APL arrays - a numeric matrix, a text matrix, a mask matrix and a vector of text column widths.

The numeric matrix has the same number of rows and columns as the spreadsheet. Numeric spreadsheet cells match numeric values in the matrix; text cells match numeric matrix entries of zero. For the example above, the numeric matrix looks like the one in Figure 3.

0	0	0	0	0
0	89000	93000	117000	299000
0	73450	81830	95460	250740
0	0	0	0	0
0	15550	11170	21540	48260
0	0	0	0	0

Figure 3 - 'nums' matrix

The text matrix has as many rows as the spreadsheet. Each column of the spreadsheet corresponds to a set of consecutive characters in the text matrix. The widths vector tells us how many characters wide each column of the spreadsheet is. Going back to our example, the text matrix (called chars) and widths vector (called widths) look like Figure 4.

chars	Jan 85	Feb 85	Mar 85	Total
Sales				
Costs				
Profit				

```

widths
15 9 9 9 9

```

Figure 4 - 'chars' and 'widths'

In most cases it will be clear from looking at 'nums' and 'chars' which cells of the original spreadsheet contained numbers and which ones contained characters. However, this information is explicitly held in the mask matrix, which has one character per spreadsheet cell - a 't' for text cells and a 'v' for numeric-values cells. In our example, the mask matrix is shown in Figure 5.

```

mask
ttttt
tvvvv
tvvvv
ttttt
tvvvv
ttttt

```

Figure 5 - 'mask' matrix

The transfer technique

The processing of our budget data is done by several people, not all of whom are expert in APL. We needed to be able to package up the transfer of spreadsheet data into an APL function that could be integrated into a reporting system. In principle this transfer consists of three distinct stages:

- (i) conversion of the spreadsheet into DIF file format
- (ii) conversion of the DIF file to APL format
- (iii) fetching the data into the active APL workspace

The first stage can be done by one of the programs on the SYMPHONY utilities diskette. The program (WRKDIF) expects a worksheet pathname and a DIF-file pathname to follow the program name on the line where it is invoked; it can be called from within APL by means of the quad-CMD system command, provided that it has been copied into the appropriate DOS sub-directory.

The second stage could be done by an APL function, but the DIF file format means that this would be very inefficient. We decided that we would develop a DIF file conversion program in the C programming language. C has many advantages for this sort of application; these include portability, ease of maintenance, efficiency and simplicity.

The program that we developed expects a DIF filename and a DOS filename to follow it on the line where it is invoked. It reads the DIF file, creating the numeric and mask matrices and storing each text string and column width in appropriate data structures. It then creates a DOS file, and writes out a series of entries in the same format used internally by APL. The file contains

- (i) the shape of the spreadsheet (number of rows, number of columns, both held as two-byte integers)
- (ii) the ravelled numeric matrix (an array of floating-point numbers in IEEE format, as used by APL *PLUS/PC).

- (iii) the ravelled mask matrix (a series of bytes representing ASCII characters)
- (iv) the width vector (a series of two-byte integers)
- (v) the ravelled character matrix (in the same format as the mask matrix.)

The user is blissfully unaware of this, as the APL program reads all the relevant data into APL variables using the APL*PLUS 'native file read' quad-function. A listing of the relevant APL function is included in appendix 2, as is a sample session using the interface.

Software development in C

We approached the development of the DIF file conversion program with some apprehension. I was keen to do the development myself, as we expect the combination of C system software and APL applications software to play an increasingly important part in our business. I found many points of similarity between C and APL, but there are also fundamental differences. Like APL, C is terse and powerful. This can lead to elegant software, and it can help you produce maintainable code - but you will need maturity and self-restraint if you want to be remembered for writing programs that are both elegant and easy to maintain!

C differs from APL in that it has (and needs to have) a full range of constructs to control looping and branching. Because arrays in C have to be manipulated one element at a time, loops are common. The language constructs (linked with appropriate conventions about program layout) mean that loops can be efficiently executed and easily understood.

Another fundamental difference is that C is a compiled language. (C interpreters exist, but lose the efficiency advantage of compiled code). Although C compilers can be fairly fast, they cannot yet yield the same programmer productivity as APL. C Programs can run much faster, however - it takes about 6 seconds to generate a full cross-reference listing for the DIF file interface, whereas an APL cross-reference of a similar sized APL system would take several minutes on a PC.

For those who do not know C, but wish to get a flavour of the language, a listing of one of the functions used in the DIF file interface is included as appendix 1. The function (called `sa__iota`) looks up a string of characters in a vector of strings. It resembles dyadic `iota` in APL.

It has not been possible to list the complete DIF source; it would take up too much space, and in any case a commercial product based on this software is currently undergoing beta-test.

Conclusion

We feel we have achieved considerable benefit through this development. Much has been gained by using the right tool for the right job - we have combined the ease of use of spreadsheet software with the power of APL. We have also been able to integrate package software, tailored C code and APL thanks to the powerful interfacing capability of APL*PLUS/PC.

A further benefit awaits us. When spreadsheets producing DIF files become widespread under UNIX, we will be able to use all the interface software without modification. C is portable across operating systems, and both the APL code and internal data formats are fully compatible with APL*PLUS/UNIX.

Appendix 1 - C function listing

sa_iota(v,s) /* return index of string s in vector of string pointers v.
return -1 if not found. last element of v must be NULL. */

```
char *v[ ], *s;  
{  
    int c;  
    char **p;  
  
    p = v;  
    while (*p) {  
        c = strcmp(s,*p);  
        if (!c) return(p-v);  
        p ++;  
    }  
    return(EOF);  
}
```

Appendix 2 - Sample APL session

```

WRK2APL[]V
[0] WRK2APL pathname;rho;cells;[]IO;char;int;real;tw;trho
[1] A WRK File into APL variables
[2] A
[3] A Calls <if>, <Acmd>. Invokes <WRKDIF.EXE>, <DIF.EXE>.
[4] A
[5] A <nums>, <mask>, <widths> and <chars> are used to contain
[6] A the relevant items of the spreadsheet.
[7] A
[8] A <pathname> is used as the source file;
[9] A <TEMP.DIF> is used as the DIF transfer file, which is erased.
[10] A <TEMP.APL> is used as the APL transfer file, which is erased.
[11] []NUNTIE []NNUMS []IO+1 [] char+82 [] int+163 [] real+645
[12] +err if 0≠Acmd 'WRKDIF ',pathname,' TEMP.DIF' A CONVERT WRK TO DIF
[13] +err if 0≠Acmd 'DIF TEMP.DIF TEMP.APL' A CONVERT DIF TO APL FORMAT
[14] +err if 0≠Acmd 'ERASE TEMP.DIF'
[15] 'TEMP.APL' []NTIE -1
[16] cells+*/rho+[]NREAD -1,int,2 0 A READ SHAPE VECTOR
[17] nums+rhop[]NREAD -1,real,cells A READ NUMERIC VALUES
[18] mask+rhop[]NREAD -1,char,cells A READ MASK
[19] tw+*/widths+[]NREAD -1,int,rho[2] A READ WIDTHS
[20] chars+trhop[]NREAD -1,char,*/trho+rho[1],tw A READ CHARS
[21] 'TEMP.APL' []NERASE -1 [] +0 A ERASE APL TRANSFER FILE
[22] err:'OUCH'

```

```
WRK2APL 'C:\SYMPHONY\RJC\EG1.WRK'
```

```

nums
0      0      0      0      0
0 89000 93000 117000 299000
0 73450 81830 95460 250740
0      0      0      0      0
0 15550 11170 21540 48260
0      0      0      0      0

```

```

chars
Jan 85   Feb 85   Mar 85   Total

```

```
Sales
```

```
Costs
```

```
Profit
```

```

widths
15 9 9 9 9

```

```

mask
ttttt
tvvvv
tvvvv
ttttt
tvvvv
ttttt

```

```

+ \0 -1+nums
0      0      0      0
0 89000 182000 299000
0 73450 155280 250740
0      0      0      0
0 15550 26720 48260
0      0      0      0

```

WHEN DOMINO IS NOT SUFFICIENT

by A.M. Sykes and J.I. Ansell

Introduction

The use of DOMINO to solve linear equations is a particularly pleasant feature of APL to a Statistician as it forms a basis for Regression. However one should be on guard in using any general routine such as DOMINO. Whilst it performs adequately in many contexts, even in comparison to some NAG routines, there are circumstances where it causes concern. On these occasions it would be pleasant to have other general routines.

In this paper the authors outline two main deficiencies they have found using MIP's APL in Regression and suggest remedies. The first problem is numerical stability due to variables in the regression having low variances. The second occurs when fitting many variables, some of which are collinear.

Briefly, the first problem may be overcome fairly simply by use of transformations. The second requires the introduction of forms of Generalised Matrix Inverse or Psuedo-Inverse. The latter leads to a more natural view of regression which facilitates extensions.

Regression

Regression is possibly the most widely used technique in Statistics. It attempts to account for the variance in one variable by another variable or set of variables. In most practical situations the underlying model is taken to be linear of the form;

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_kx_k + e$$

where y is the variable one wishes to predict, $x_1, x_2, \dots, x_k$ is a set of variables used to predict y , and e is the residual. The term residual is used to account for any portion of the variation in y unaccounted for by the variables $x_1, \dots, x_k$. Often e is taken to have a Normal distribution, though this is not a requirement as some practitioners seem to believe, see (1,11,12). (Other forms of generalisation are also mentioned in these papers.) The obvious APL solution for the unknown parameters B would be;

$$Y \overline{X} X$$

where X is the design matrix with columns composed of $x_1, \dots, x_k$ with the first column being a vector of 1's.

The mathematical representation of the solution is not so attractive being;

$$B = (X^T X)^{-1} X^T Y.$$

Hence the main problems revolve around the invertibility of $X^T X$. One can divide the problem into two possible classes; Near Singular and Singular.

Near Singular of $X^T X$ due to small variances.

A simple example, given by Greenfield and Siday (8) shows how small variances cause many statistical packages to fail. The data are presented in Table 1. Applying DOMINO results in a DOMAIN ERROR for even moderate values of n and m , see Table 2. Essentially the routine cannot cope with the variance of the predictive variable. A simple solution to the problem is to strip off the mean and then appropriately scale the variable involved. Fitting can then usually be carried out using DOMINO. After fitting one can then rescale if desired.

Table 1
Greenfield's Data

i	1	2	3	4	5
Y_i	$7 + 10^{-n}$	$7 + 2 \times 10^{-n}$	$7 + 3 \times 10^{-n}$	$7 + 4 \times 10^{-n}$	$7 + 5 \times 10^{-n}$
X_i	$8 + 10^{-m}$	$8 + 3 \times 10^{-m}$	$8 + 2 \times 10^{-m}$	$8 + 4 \times 10^{-m}$	$8 + 5 \times 10^{-m}$

(For this data intercept should be $7 + 0.3 \times 10^{-n} - 7.2 \times 10^{m-n}$,
and the gradient should be $0.9 \times 10^{m-n}$.)

Obviously there is a wide choice of possible scaling procedures. One might scale by the standard deviation though this would entail squaring of already small values. An alternative is to use average of the absolute deviations from the mean, this is the method employed in the algorithm INVERT.

```

V BETA+Y INVERT X;MY;Y1;X1;X2;N;M
[1] Y1+Y-MY+Y/Y+P Y
[2] X1+X1+NpX2+X/(X1+X-NpM+X+N[1])+(N+P X)[1]
[3] N+1+BETA+Y1@1,X1
[4] BETA+(MY+BETA[1]-M/N),N+N+X2
V

```

The algorithm has been fairly successful in practice, the result of analysing data in Table 1 is given in Table 2.

Table 2

Number of correct significant figures when applying DOMINO, INVERT and BJORCK to Greenfield's Data with $n=0$ and $m=1,2, \dots, 13$.

M	DOMINO	INVERT	BJORCK
1	ALL	ALL	ALL
2	8	ALL	9
3	7	ALL	8
4	4	ALL	7
5	DE	9	6
6	DE	8	5
7	DE	7	4
8	DE	6	3
9	DE	5	2
10	DE	4	0
11	DE	3	0
12	DE	2	0
13	DE	0	0

(DE means DOMAIN ERROR and ALL means correct answer given.)

At each pivot-sweep the effect of a given variable is removed not only from the other X variables but also from Y and hence the side-effects are as follows:

- (1) The regression coefficients are produced in the last column.
- (2) Dropping off the last row and column produces the $-(X^T X)$ and this yields the estimators variance covariance matrix.
- (3) The entry in the bottom righthand corner is the residual sum of squares in the usual Normal Theory of regression.

What happens when a pivot is 0? Each pivot is examined before it enters the routine. If below a given threshold, set by the user, then that variable is marked. A marked variable is one which is (almost) totally dependent on previous variables, it therefore affords no additional explanation. For the data presented in Table 3 DOMINO would produce DOMAIN ERROR whereas use of Beaton yields the following solution;

$$Y = 2.6029 - 0.0016X_1 - 0.3061X_2 + 0X_3$$

Obviously it is not unique — an alternative solution is

$$Y = 2.6029 - 0.0995X_1 - 0.1225X_3 + 0X_2$$

In the penultimate section of the paper the authors give some indication of criteria for selecting an appropriate model.

Table 3
Data with Linear Dependence amongst the X variables

N	Y	X1	X2	X3
1	2.6	2.500	1.050	0.625
2	1.3	2.375	1.960	3.000
3	2.5	3.125	1.700	1.750
4	1.5	1.000	0.870	1.375
5	3.0	1.875	1.300	1.750
6	2.2	1.500	1.730	3.125
7	2.2	0.875	1.680	3.500
8	2.4	0.125	1.040	2.500
9	1.7	3.500	1.470	0.875
10	2.2	1.625	1.570	2.625

Bjorck's Algorithm

This is a modified version of the Gram-Schmidt Orthogonalisation procedure. It might be thought of as a sweep procedure since each column sweeps through the matrix X in turn. Hence one removes the effect of a variable from all subsequent variables. It does however result in two matrices usually referred to as Q and R, hence the generic name for this type of procedure — a Q-R technique. R is a square upper triangular matrix whilst Q is an orthogonalization of X.

The APL code for this algorithm is given below:

```

V B←Y BJORCK X;N;J;K;T;S
[1] J←÷/ρR←(N,N+1+ρQ+(ρX)ρ0)ρ0
[2] L:R[J;J]←(X[J;J]+.×X[J;J])×0.5
[3] Q[J;J]←X[J;J]+R[J;J]
[4] +(J=N)/R
[5] R[J;K]←Q[J;J]+.×X[K;J+1N-J]
[6] X[K;J]←X[K;J]-Q[J;J]+.×R[J;K]
[7] +(1<J+1)/L
[8] R←C←Y+.×Q
[9] B←S\ C[T]R[R[T;T+(S+1E-9< 1 1 Q R)/1N]
V

```

Singularity due to Collinearity.

Problems due to collinearity are more complex and result in the singularity of $X^T X$, due to linear dependence between the columns of X . This simply means that one or more of the variables can be expressed in terms of the others for the data in question. This may or may not be due to an intrinsic relationship between the variables. Hence although DOMINO will fail there exists a solution though it will not be unique. It is achieved by removing a dependent subset. The only question to a Statistician in this context would be which variable or set of variables should be removed. The solution depends partially on the context and criteria of efficiency of choice, i.e. in some senses a matter of taste.

The main insight into the Generalized Inverse of a Matrix came from Penrose (13) who reintroduced the concept initially proposed by Moore (10). He was subsequently followed in Statistics by Rao (15) and Wilkinson (18), and summarized both in Chambers (5), and Pringle and Rayner (14). There is a need therefore for an efficient Generalised Matrix Inverse. There are numerous candidates for the role. In this paper we restrict attention primarily to two, but interested readers may consider others, e.g. Given's (6), Householder's (4,9) and Rogers and Wilkinson (16) are worthy of study. The two selected are due to Beaton (2) and Bjorck (3). Each of these can be said to have particular merits; Beaton's is compact; Bjorck is preferred on grounds of accuracy.

Beaton's Algorithm

This routine works directly with the $X^T X$ matrix, and hence is compact. (X is an $m \times n$ matrix with m usually much larger than n , so $X^T X$ is then an $n \times n$ matrix.) Unfortunately this 'squaring' can exacerbate numerical instability when near singular matrices are encountered. Obviously its compactness makes it more appropriate for use when storage or workspace size is limited, or other constraints exist on the size of individual components.

The algorithm applies a sweep-based routine to $X^T X$, which in the context of regression is either the sum of products matrix or the variance-covariance matrix. Given one has selected a particular pivot, a suitable diagonal element, the pivot is used to remove the effect of the associated variable from the matrix. In the case of using the variance-covariance matrix the pivot is the variance of the variable being fitted, and a matrix of the residual variance-covariances results.

The APL code for a single pivot operation is given below and the matrix is inverted by cycling through all the pivotal elements.

```

▽ M←N BEATON K
[1] M←M-N[K;]∘.×N[K;]÷N[K;K]
[2] M[K;]←N[K;]÷N[K;K]
[3] M[;K]←N[;K]÷N[K;K]
[4] M[K;K]←÷N[K;K]
▽

```

In the context of regression it is helpful to consider the following matrix since application of Beaton to it has some pleasant attributes.

$$\begin{array}{cc} X^T X & X^T Y \\ X^T Y & Y^T Y \end{array}$$

Other desired quantities may be calculated easily. For example the residual sum of squares is obtained by

$$+ / (Y - Q + . \times C) * 2$$

Chambers indicates in (5) that it is more accurate to use the above approach to calculate the residual than using the estimates of Y themselves.

The properties of Bjorck's algorithm have been extensively investigated by previous authors (5,16) and found to be very well behaved. As an indication Bjorck's algorithm was applied to Greenfield's data and the results presented in Table 2.

A More Natural View of Regression

The traditional view of regression is through the solution of the Normal Equations, see (7,17). Whilst mathematically sound we have found it has little intuitive appeal even to mathematics students. The idea of 'sweeping' in both Beaton's and Bjorck's algorithms suggests the geometric viewpoint of projecting a new variable onto the present variable space. Also such procedures imply that the order of introducing covariates may be important.

Choosing the Solution in case of Singularity

As indicated before, where no unique solution exists the selection of variables to include is a matter for the individual. First of all, the variables included in the model should be understandable within the physical background of the problem. Secondly, models should be as simple as possible, because they are easier to understand and tend to be more robust. Hence one should include as few variables as possible. Also the variables should themselves be uncomplicated in form.

The selection of variables in these two techniques depends on the order of their consideration, (selection of order of sweeping). This means either the order has to be specified by the user, or some criterion must be enforced by the routine. In Statistics it would be controversial to lay down a definitive solution to this problem. However, the authors feel that the selection based on partial correlation, while not always optimum, might be a suitable way around the problem.

Extensions of the Problem

The paper has discussed some numerical problems associated with standard Regression Analysis, where traditionally one assumes (amongst other things) that the error is Normal and has constant variance. The latter may be relaxed by using weighted least squares, replacing X by WX and Y by WY where W is a diagonal matrix of weights. The amended algorithm forms the core of the iterative solution of such Generalized Linear Models which allows different error distributions to be contemplated, see (11,12). The authors are at present working on an APL version of the Generalized Linear Models (1).

References

- (1) Ansell, J.I., and Sykes, A.M., (1984), 'APLIM — Applied Linear Interactive Models', Tech. Rep. 84/2, Department of Man. Sc. and Stats, University College of Swansea, Wales.
- (2) Beaton, A.E., (1964), 'The use of special matrix operators in statistical calculus', Research Bulletin RB-64-57 Educational Testing Service, Princeton, New Jersey.
- (3) Bjorck, A., (1967), 'Solving linear least squares problems using Gram-Schmidt orthogonalization', BIT, 7, 257-78.
- (4) Bussinger, P.A., and Golub, G.H., (1965), 'Linear least squares solution by Householder transformation', Numer. Math., 7, 269-276.
- (5) Chambers, J.M., (1977), Computational methods for data analysis, Wiley, New York.
- (6) Clarke, M.R.B., (1981), 'Algorithm AS163: A Givens algorithm for moving from one linear model to another without going back to the data', Appl. Stat., 30, 166-8.
- (7) Edwards, A.L., (1979), Multiple regression and the analysis of variance and covariance, W.H. Freeman & Co., San Francisco.
- (8) Greenfield, T., and Siday, S., (1980), 'Statistical computing for business and industry', Statistician, 29, 1, 33-55.
- (9) Lawson, C.L., and Hanson, R.J., (1974), Linear least squares solutions, Prentice-Hall, Englewood Cliffs, NJ.
- (10) Moore, E.H., (1935), 'General analysis', Mem. Amer. Phil. Soc., 1, 197.
- (11) Nelder, J.A., and McCullagh, P., (1983), Generalized Linear Models, Methuen, London.
- (12) Nelder, J.A., and Wedderburn, R.W.M., (1972), 'Generalized Linear Models', JRSS, A, 135, 370-84.
- (13) Penrose, R., (1955), 'A generalized inverse of matrices', Proc. Camb. Phil. Soc., 51, 406-11.
- (14) Pringle, R.M., and Rayner, A.A., (1971), Generalized inverse matrices with applications to statistics, Griffin, London.
- (15) Rao, C.R., (1962), 'A note on a generalized inverse of a matrix with applications to problems in mathematical statistics', JRSS, B, 24, 152-8.
- (16) Rogers, C.E., and Wilkinson, G.N., (1974), 'Regression, curve fitting and smoothing numerical problems in recursive analysis of variance algorithms', IMA Bull., June, 141-143.
- (17) Stoodley, K.D.C., Lewis, T., and Stainton, C.L.S., (1980), Applied Statistical Techniques, Ellis Horwood Ltd, Chichester.
- (18) Wilkinson, G.N., (1958), 'Estimation of missing values for the analysis of incomplete data', Biometrics, 14, 257-86.

INDEX TO ADVERTISERS

APL People Ltd.	Page 163
British Airways	Page 26
The British Computer Society	Page 32
Cocking & Drury	Page 52
Cocking & Drury	Page 140
Dyadic Systems Ltd	Page 74
H.M.W. Programming Consultant Ltd.	Page 86
Inner Product	Page 104
I.P. Sharp Associates	Page 4
Metatechnics Systems Ltd.	Page 62
MicroAPL Ltd.	Page 122

Potential advertisers are reminded that to advertise in the next issue of VECTOR they should contact David Preedy at Metapraxis Ltd., Hanover House, Coombe Road, Kingston-on-Thames, KT2 7AH, or phone 01-541 1696.

Please note the following copy dates for future issues of VECTOR:

VECTOR Vol.2 No. 1 . . . May 31st
VECTOR Vol.2 No. 2 . . . Aug 31st
VECTOR Vol.2 No. 3 . . . Nov 30th
VECTOR Vol.2 No. 4 . . . Feb 29th

The editors would be delighted to receive future contributions on floppy disk. This greatly simplifies the job of proof-reading, and cuts our typesetting costs considerably.

The preferred format is:

IBM PC DOS 2.0 or later

Please save the file unformatted (or print it to disk) so that there are no control characters embedded in it.

Please also include a hardcopy version, so that we can mark it up sensibly for the printers.

We shall of course return your disks!

Thanks.



APL People Ltd.

J. Cross Lane Close,
Orwell,
Nr. Roylton,
Herts. SG8 5QW

APPOINTMENTS

A placement service solely for the APL community.

Use our extensive knowledge, contacts and experience in the APL world.

We have vacancies NOW, both permanent and contract and our confidential advice on career moves is always freely available.

CONSULTANCY

A consultancy service designed to solve your APL problems.

Consultant's Toolkit and Supervision available.

Current projects include:-

- * graphics
- * project management
- * databases
- * system design
- * system 'rescue'
- * documentation
- * VSAPL under VSPC, CMS, TSO
- APL * Plus/PC, Sharp APL

To discuss your requirements: — Call Valerie Lusmore on 04626-77375

Registered employment agency licence SE6440

BRITISH APL ASSOCIATION
Membership Application Form

Please read the membership information in the inside front cover of VECTOR before completing this form. Use photocopies of this form for multiple applications. The membership year runs from 1st May 1985 — 30th April 1986.

Name: _____

Department: _____

Organisation: _____

Address line 1: _____

Address line 2: _____

Address line 3: _____

Address line 4: _____

Post or zip code: _____

Country: _____

Telephone number: _____

Membership category applied for (tick as appropriate):

Non-voting student membership	£ 5
UK private membership	£ 9
Overseas private membership	£ 16
Airmail supplement (not needed for Europe)	£ 8
Corporate membership	£ 75
Sustaining membership	£325

For student applicants:

Name of course: _____

Name and title of supervisor: _____

Signature of supervisor: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK sterling cheque or postal order made payable to "The British APL Association". Corporate or sustaining member applicants should contact the Treasurer in advance if an invoice is required. Please enclose a stamped envelope if you require a receipt.

Send the completed form to the Treasurer at this address:

Mel Chapman, 12 Garden Street, Stafford, ST17 4BT, UK.

THE BRITISH APL ASSOCIATION

The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. It is administered by a Committee of eight officers who are elected by the vote of Association members at the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1984 COMMITTEE

Chairman:	Philip Goacher 01-637 0471	The British Computer Society, 13 Mansfield Street, London W1M 0BD
Secretary:	Anthony Camacho 0727-60130	2 Blenheim Road, St. Albans, Herts AL1 4NR.
Treasurer:	Mel Chapman 0785-53511	12 Garden Street, Stafford, ST17 4BT.
Activities:	Dick Bowman 01-634 7639	CEGB, 85 Park Street, London SE1.
Publicity:	Vacant	
Journal Editor:	Robert Bittlestone 01-541 1696	Metapraxis Ltd. Hanover House, Coombe Road, Kingston KT2 7AH.
Education:	Dick Gray 0476-860483	Horseshoe House, Sproxtton, Melton Mowbray, Leicestershire LE14 4QB
Technical:	Geoff Kemmish 01-638 5372	181 Andrewes House, Barbican, London EC2Y 8BA

ACTIVITIES WORKING GROUP

David Allen
Dick Bowman
Dominic Murphy
David Preedy
Roy Tallis
Stan Wilkinson

JOURNAL WORKING GROUP

Jonathan Barman	01-493 6172
Robert Bittlestone	01-541 1696
David Preedy	01-541 1696
Adrian Smith	0904-53071
David Ziemann	01-493 6172

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. APL stands for "APL Programming Language" - an interactive computer programming language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People

3 Cross Lane Close,
Owells, Nr. Royston, Herts. SG8 5OW
Tel. 04326-77376

APL PLUS Ltd

Aston Science Park, Love Lane,
Birmingham B7 4BL
Tel. 021-639 5096

EXOL Inc. International

Ortel House, 28 The Quadrant,
Richmond, Surrey TW9 1DL
Tel. 01-940 3606

Cooking & Drury Ltd

16 Berkeley Street, London W1X 5AE
Tel. 01-493 6172

Dyadic Systems Ltd

30 Camp Road, Farnborough, Hants.
Tel. 0252 547222

Inner Product Ltd

Eagle House, 76 Clapham Common Southside,
London SW4 9DG
Tel. 01-673 3354

Metatechnics Systems Ltd

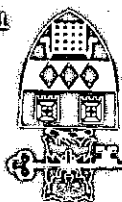
Unit 216, 62 Tilton Road,
London SE24 8DE
Tel. 01-670 7959

MicroAPL Limited

Unit 1F, Nine Elms Industrial Estate,
67 Kirling Street, London SW8 5BP
Tel. 01-622 0895

LP Sharp Associates

10 Dean Farer Street,
London SW14 0DX
Tel. 01-222 7083



The British Computer Society, 19 Mansfield Street, London W1M 0BD